CURS 1 - INTRODUCERE

Sisteme Grafice - clasificare

- de sinteză a imaginilor
    - crearea unei imagini ale unor obiecte/scene reale sau virtuale, statice sau dinamice (animate)
    - domeniul de studiu: Computer Graphics
- de prelucrare și analiză a imaginilor / sisteme cu viziune
    - extragerea de informații din imagini digitale produse de dispozitive de achiziție (camere video, scanner)
    - domeniul de studiu: Computer Vision
- de reconstrucție și vizualizare a unor obiecte/fenomene reale pornind de la date volumetrice
    - domeniul de studiu: Volume Graphics (combină algoritmi de CG și CV)

Modelul 2D/3D al imaginii

- inclus în programul de aplicație
- alcătuit din
    - primitive grafice 2D, 3D (linii, poligoane) sau obiecte complexe - alcătuiesc scena 2D/3D redată într-un cadru imagine
    - proprietăți ale suprafețelor obiectelor din scenă (culoare, textură, proprietăți de reflexie/refracție a luminii)
    - transformări de aplicat obiectelor în scopul compunerii, vizualizării scenei și animării obiectelor scenei
    - surse de lumină (poziție, caracteristici de culoare/luminozitate)
    - poziția observatorului virtual în scena 3D și direcția privirii
- orice modificare trebuie să aibă ca efect modificarea în timp real a imaginii afișate

Operații de sinteză a imaginilor grafice

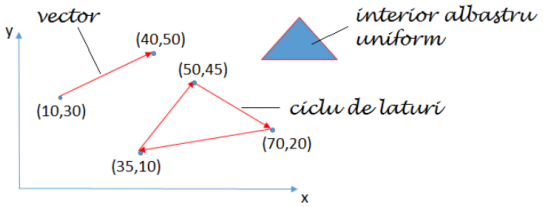
- efectuarea transformărilor care se aplică obiectelor pentru modelarea scenei
- efectuarea transformărilor de proiecție din spațiul 3D în spațiul 2D
- eliminarea obiectelor nevizibile din imaginea unei scene 3D
- eliminarea fețelor nevizibile (auto-obturate) ale obiectelor
- decuparea primitivelor la marginile volumului vizual (field of view)
- rasterizarea - descompunerea primitivelor grafice în fragmente care se afișează în pixeli

- calculul culorii fiecărui fragment (în fragment shader) folosind modele de reflexie și refracție a luminii, calcul de umbre, texturi
- operații la nivel pixel (operații raster) - test vizibilitate a fragmentelor în imagine, combinarea culorii fragmentului cu cea a pixelului în care se afișează

CURS 2 - TRANSFORMĂRI GEOMETRICE 2D

Introducere

- obiectele 2D/3D sunt reprezentate prin
  - date geometrice - coordonatele vârfurilor, raportate la un sistem de coordonate carteziene 2D sau 3D
  - atribute topologice, care definesc modul în care sunt conectate vârfurile (laturi, ciclul de laturi al unei fețe)
  - atribute de aspect - culoarea, tipul de interior pentru suprafețele 2D, atribute de material pentru suprafețele 3D (exemplu: reflexia/refracția luminii de către suprafață)



- transformările geometrice se aplică (coordonatelor) vârfurilor și nu afectează atributele sale
- transformările geometrice sunt folosite pentru
  - redarea desenelor la diferite mărimi
  - compunerea desenelor sau a scenelor 3D
  - realizarea animației
  - transformarea obiectelor dintr-un spațiu de coordonate logic, în care sunt definite, în spațiul fizic de afișare
- transformările geometrice sunt **transformări ale obiectelor definite în sistemul de coordonate carteziene XOY (obiecte 2D)**; fiecare vârf al unui obiect 2D => (x, y)

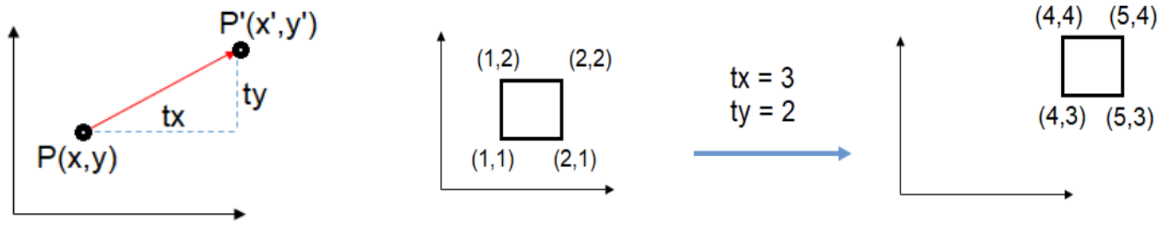
Transformări geometrice elementare

- translația, scalarea față de origine, rotația față de origine, forfecarea față de origine, oglindire față de axele principale și față de origine
- orice altă transformare se obține prin compunerea a două sau mai multe transformări elementare, de exemplu
  - scalarea/rotația față de un punct oarecare din plan
  - scalarea compusă cu translație

Translația

- definită printr-un vector T[tx,ty]

$$x' = x + tx$$
$$y' = y + ty$$



- **compunerea transformărilor** se realizează prin **înmulțirea matricelor**
- un punct din plan P(x,y) se reprezintă în coordonate carteziene printr-un vector [x,y] sau $\begin{bmatrix} x \\ y \end{bmatrix}$
- nu există o matrice M de 2x2 astfel încât să se exprime translația prin

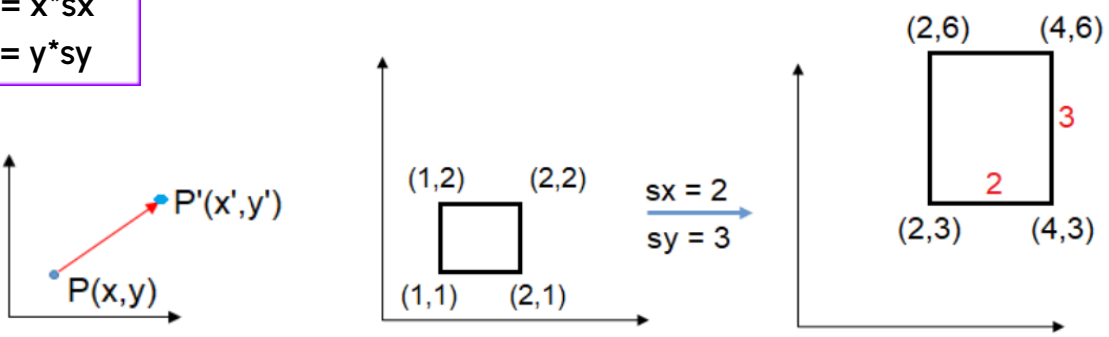
$$[x', y'] = [x, y] * M \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = M * \begin{bmatrix} x \\ y \end{bmatrix}$$

- sunt păstrate lungimile, unghiurile

Scalarea față de origine

- definită prin 2 numere reale, de regulă pozitive
  - sx - scalarea pe axa OX
  - sy - scalarea pe axa OY

$$x' = x*sx$$
$$y' = y*sy$$



- efecte
  - dacă sx sau sy sau ambii sunt >1 => obiectul se mărește
  - dacă sx sau sy sau ambii sunt <1 => obiectul se micșorează
  - sx = sy => scalare uniformă
  - dacă sx ≠ sy => scalare neuniformă, nu se păstrează lungimile/unghiurile

- **reprezentarea matriceală**

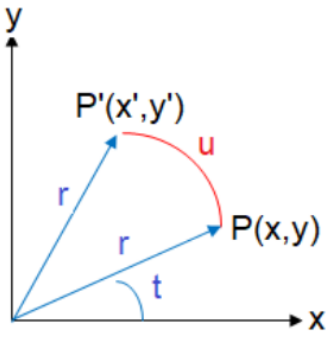
$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Rotirea față de origine

relația dintre coordonatele carteziene și coordonatele polare ale unui punct P(x,y)

x = r*cos(t)
y = r*sin(t)
x' = r*cos(t+u) = r*(cos(t)*cos(u) – sin(t)*sin(u))
= x*cos(u) – y*sin(u)
y' = r*sin(t+u) = r*(cos(t)*sin(u) + sin(t)*cos(u))
= x*sin(u) – y*cos(u)



rotația față de origine

x' = x*cos(u) - y*sin(u)
y' = x*sin(u) + y*cos(u)

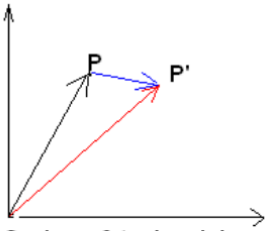
$$[x' \;\; y'] = [x \;\; y] * \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) \\ \sin(u) & \cos(u) \end{bmatrix} * \begin{bmatrix} x \\ y \end{bmatrix}$$

Scalarea față de un punct oarecare din plan

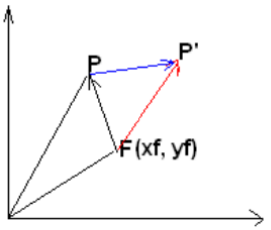
- punctul fix al transformării este un punct oarecare F(xf,yf) - coordonatele sale nu se modifică prin aplicarea transformării
- P(x,y) este punctul transformat
- scalarea se aplică vectorului FP [x-xf, y-yf]
  x' - xf = sx*(x - xf)
  y' - yf = sy*(y - yf)

  rezultă
  x' = x*sx + xf - xf*sx
  y' = y*sy + yf - yf*sy



Scalarea fata de origine

x' - 0 = sx (x - 0)
y' - 0 = sy (y - 0)



Scalarea fata de un punct F din plan

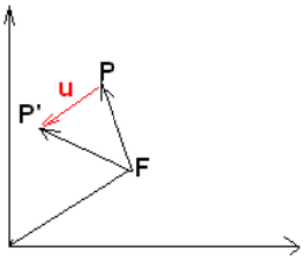
x' - xf = sx (x - xf)
y' - yf = sy (y - yf)

- nu poate fi exprimată în coordonate carteziene printr-o matrice de 2x2

Rotația față de un punct oarecare din plan

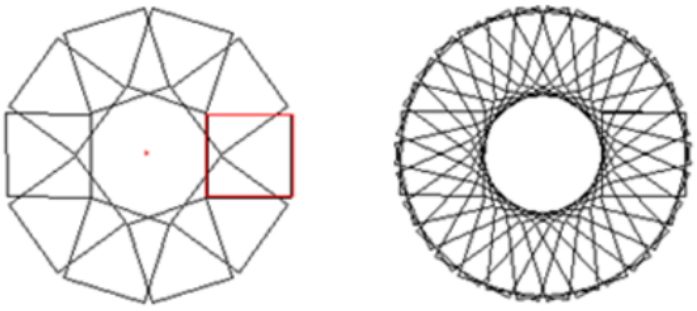
- Punctul fix al transformării este F(xf,yf)
- P(x,y) este punctul transformat
- rotația se aplică vectorului FP [x-xf,y-yf] în jurul lui F
  x' - xf = (x - xf)*cos(u) - (y - yf)*sin(u)
  y' - yf = (x - xf)*sin(u) + (y - yf)*cos(u)

  rezultă
  x' = x*cos(u) - y*sin(u) + xf - xf*cos(u) + yf*sin(u)
  y' = x*sin(u) + y*cos(u) + yf - xf*sin(u) - yf*cos(u)



Rotatia fata de un punct F din plan

- nu poate fi exprimată în coordonate carteziene printr-o matrice de 2x2


Rotatia unui patrat in jurul unui punct din plan

Compunerea transformărilor geometrice 2D

- necesară pentru a putea aplica **o singură transformare care înglobează o secvență de transformări elementare**, în locul aplicării unei succesiuni de transformări elementare
- de exemplu, se aplică tuturor vârfurilor o transformare care înglobează scalare, rotație și translație în loc să se aplice fiecărui vârf secvența de transformări elementare
  Pt = M*P în loc de P' = S*P; P'' = R*P'; P''' = T*P'', unde M = T*R*S
- matricea unei transformări compuse se obține prin înmulțirea matricelor transformărilor elementare

$$RS = R*S = \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix} \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix}$$

RS # SR

$$SR = S*R = \begin{bmatrix} s_x & 0 \\ 0 & s_y \end{bmatrix} \begin{bmatrix} \cos(u) & \sin(u) \\ -\sin(u) & \cos(u) \end{bmatrix}$$

- **translația nu poate fi reprezentată în coordonate carteziene printr-o matrice** => impune reprezentarea transformărilor în **coordonate omogene**

Coordonate omogene

- un punct din plan P(x,y) se reprezintă în coordonate omogene printr-un vector [xw, yw, w] sau $\begin{bmatrix} xw \\ yw \\ w \end{bmatrix}$, **w - orice număr real**

(x,y) -> (xw, yw, w) **xw = x*w; yw = y*w**
(xw,yw,w) -> (x,y) **x = xw/w; y = yw/w**

- exemplu: P(2,0.5) => [2, 0.5, 1], [4, 1, 2], [20, 5, 10]
- **transformarea din coordonate omogene în coordonate carteziene**

[xw yw w] → P(x, y), unde:

- **pentru w #0, x = xw/w, y = yw/w**
- **pentru w =0, P este un punct la infinit**

Reprezentarea transformărilor elementare în coordonate omogene

- Translația

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ t_x & t_y & 1 \end{bmatrix} \quad \text{sau} \quad \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Scalarea față de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Rotirea față de origine

$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} \cos(u) & \sin(u) & 0 \\ -\sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(u) & -\sin(u) & 0 \\ \sin(u) & \cos(u) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformările inverse ale transformărilor elementare

- T(tx,ty) - matricea translației, în coordonate omogene
- S(0,0,sx,sy) - matricea scalării față de origine, în coordonate omogene
- R(0,0,u) - matricea rotației față de origine, în coordonate omogene

Transformările inverse sunt

$$T(tx,\ ty)^{-1} = T(-\ tx, -\ ty)$$

$$S(0, 0, sx, sy)^{-1} = S(0, 0, 1/sx, 1/sy)$$

$$R(0, 0, u)^{-1} = R(0, 0, -\ u)$$

Transformări geometrice 2D compuse

- exemple: scalarea/rotația față de un punct oarecare din plan
- expresiile matematice ale scalării și rotației față de un punct oarecare din plan se pot obține prin compunerea următoarelor transformări
    - translația prin care punctul fix al transformării ajunge la origine: T(-xf,-yf)

- scalarea/rotația față de origine: S(0,0,sx,sy)/R(0,0,u)
- translația inversă primei translații: T(xy,yf)

- **Scalarea față de punctul F(xf,yf)**

$$[x'\ y'\ 1] = [x\ y\ 1]\underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ -x_f & -y_f & 1 \end{bmatrix}}_{T(-xf,\ -yf)}\underbrace{\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{S(0,\ 0,\ sx,\ sy)}\underbrace{\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ x_f & y_f & 1 \end{bmatrix}}_{T(xf,\ yf)}$$

Matricea transformarii compuse:

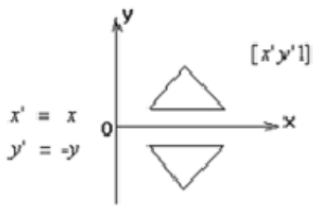
M = T(-xf, -yf) * S(0, 0, sx, sy) * T(xf, yf)

Aplicarea transformarii compuse:

[x' y' 1] = [x y 1] * M

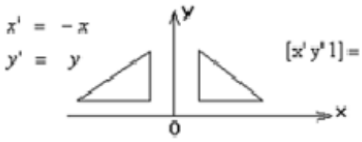
Alte transformări geometrice

- **Oglindirea**
  - față de axa OX



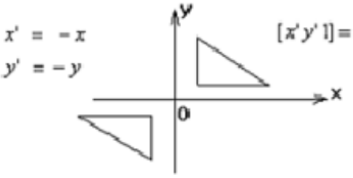
$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

  - față de axa OY



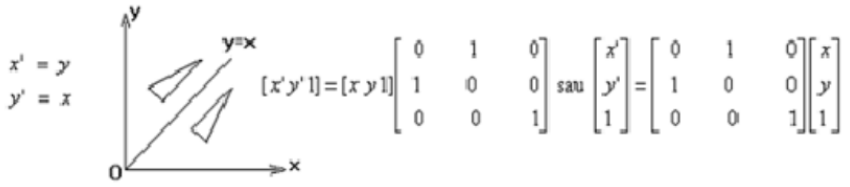
$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

  - față de origine



$$[x'\ y'\ 1] = [x\ y\ 1]\begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ sau } \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- față de dreapta x=y



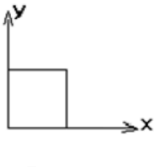
- **Oglindirea față de o dreaptă oarecare** - se exprimă ca transformare compusă prin înmulțirea matricelor care exprimă următoarele transformări
  a. o translație, astfel încât dreapta să treacă prin origine **T(-x1,-y1)**
  b. o rotație față de origine, astfel încât dreapta să se suprapună peste una dintre axele principale (OX sau OY) **R(-u)**
  c. oglindirea față de axa principală peste care a fost suprapusă dreapta (OX sau OY) **Ox sau Oy (oglindire față de OX/OY)**
  d. rotația inversă celei de la subpunctul b **R(u)**
  e. translația inversă celei de la subpunctul a **T(x1,y1)**

  În notație matricală:

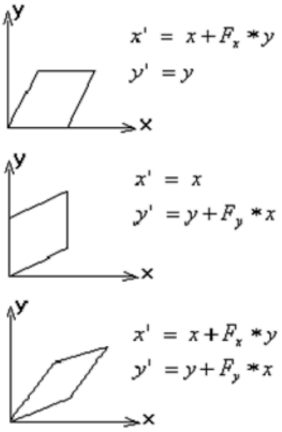
  M = T * R* O* R^{-1} *T^{-1} (folosind vectori linie)

  M = T^{-1} * R^{-1} *O *R *T (folosind vectori coloană)

- **Forfecarea**
  - definită prin 2 numere reale
    - Fx - factorul de forfecare pe axa OX
    - Fy - factorul de forfecare pe axa OY
  - față de un punct oarecare din plan (xf,yf), exprimată ca transformare compusă
    - translație prin care punctul (xf,yf) ajunge în origine
    - forfecarea față de origine
    - translația inversă primei translații



Forfecarea fata de origine



- Matricea pentru forfecarea față de axa OX

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & f_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Matricea pentru forfecarea față de axa OY

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ f_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Matricea pentru forfecarea față de ambele axe

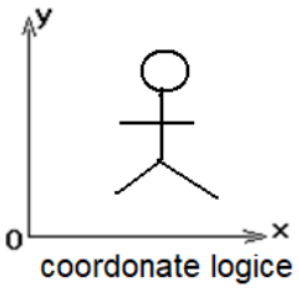
$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & f_x & 0 \\ f_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Transformări afine

- transformările care **conservă liniile drepte și paralelismul liniilor**
- translația, scalarea și rotația sunt transformări afine

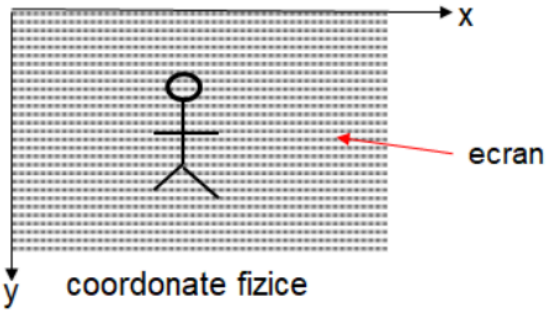
CURS 3 - TRANSFORMAREA DE VIZUALIZARE 2D

Coordonate logice și coordonate fizice



(metri, milimetri, viteza, timp, etc.)

Sistemul de coordonate utilizator

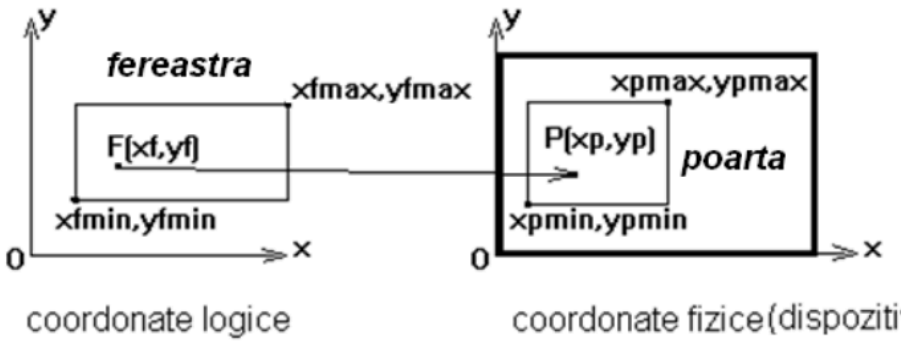


(pixeli, milimetri, s.a.)

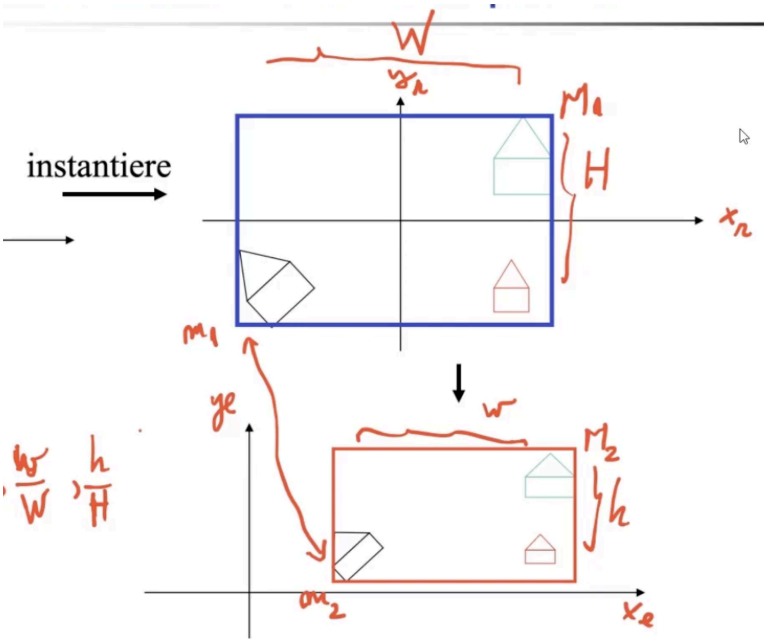
Sistemul de coordonate dispozitiv

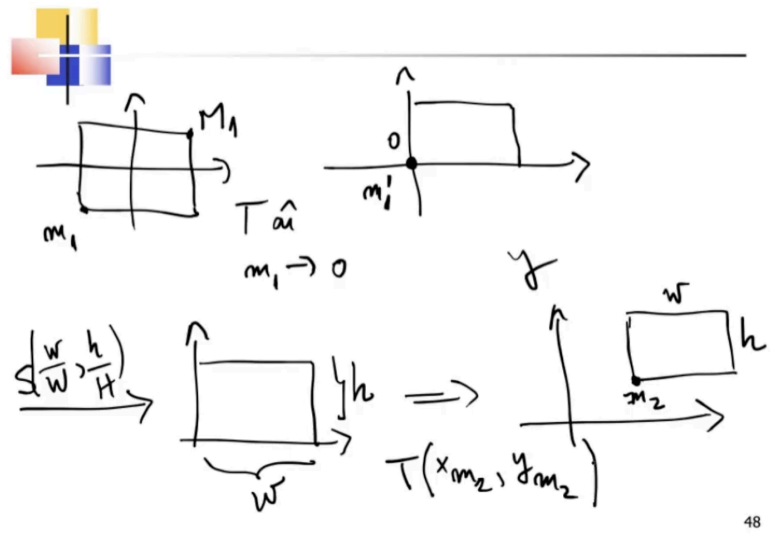
Transformarea "fereastră-poartă"

- fereastra - zona din spațiul coordonatelor logice al cărei conținut se va afișa (laturile paralele cu axele sistemului de coordonate real)
- poarta - zona din spațiul coordonatelor fizice în care se va afișa conținutul ferestrei (laturile paralele cu axele sistemului de coordonate dispozitiv)



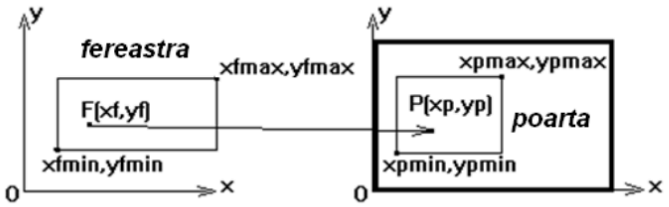
- transformarea e definită folosind 2 dreptunghiuri - **fereastra (de vizualizare), poarta (de afișare)**
- F - un punct din fereastră
  P - punctul în care se transformă F prin transformarea fereastră-poartă
- transformarea trebuie să asigure că poziția relativă a lui P în poarta de afișare e aceeași cu poziția relativă a lui F în fereastră





48

- pași pe exemplul de mai sus
  - translația lui m1 către origine
  - scalarea astfel încât să se ajungă la dimensiunile porții (de la W la w și de la H la h)
  - translația din origine cu (xm2,ym2) ca să se ajungă cu colțul din stânga jos la m2



$$\frac{xp - xp\min}{xp\max - xp\min} = \frac{xf - xf\min}{xf\max - xf\min}$$

$$\frac{yp - yp\min}{yp\max - yp\min} = \frac{yf - yf\min}{yf\max - yf\min}$$

$$sx = \frac{xp\max - xp\min}{xf\max - xf\min}$$

$$sy = \frac{yp\max - yp\min}{yf\max - yf\min}$$

- sx, sy reprezintă factorii de scalare ai transformării
- tx, ty depind de pozițiile celor 2 ferestre față de originea sistemului de coord. în care sunt definite

și $tx = xpmin - sx*xfmin$ $ty = ypmin - sy*yfmin$

$$xp = xf * sx + tx$$
$$yp = yf * sy + ty$$

← Exprimarea matematică a transformării fereastră – poartă, considerând o aceeași orientare a axelor celor 2 sisteme de coordonate.

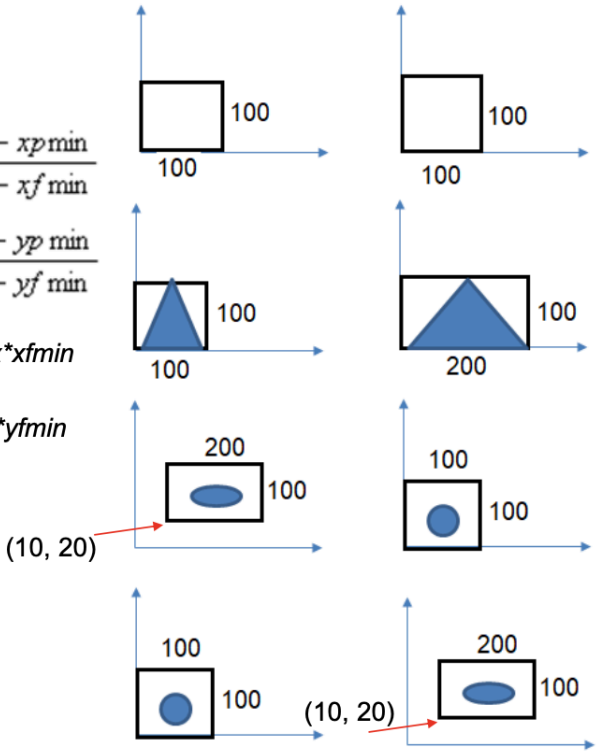
- exemple de scalare

$$sx = \frac{xp\max - xp\min}{xf\max - xf\min}$$

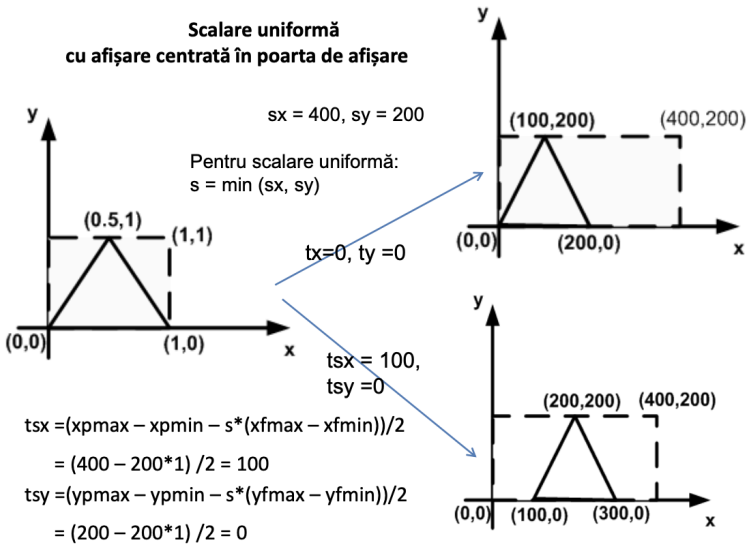
$$sy = \frac{yp\max - yp\min}{yf\max - yf\min}$$

*tx = xpmin- sx*xfmin*

*ty= ypmin- sy*yfmin*



- scalare uniformă cu afișare centrată în poarta de afișare



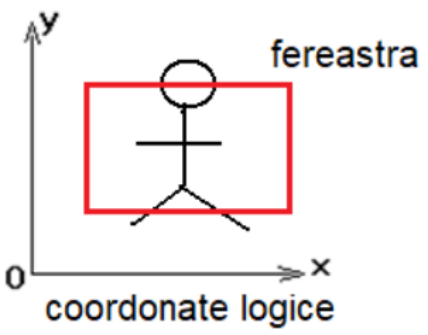
- efectele transformării
  - mărire/micșorare, în funcție de dimensiunile ferestrei și ale porții
  - deformare - fereastra și poarta nu sunt triunghiuri asemenea
  - **pentru scalare uniformă sx,sy = min(sx,sy)**
  - **afișare centrată în poartă => translație suplimentară pe axa OX sau pe axa OY**

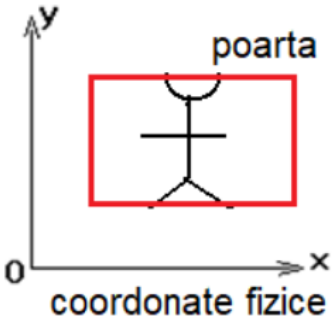
$$tx' = \frac{xp\max - xp\min - sx*(xf\max - xf\min)}{2}$$

$$ty' = \frac{yp\max - yp\min - sy*(yf\max - yf\min)}{2}$$

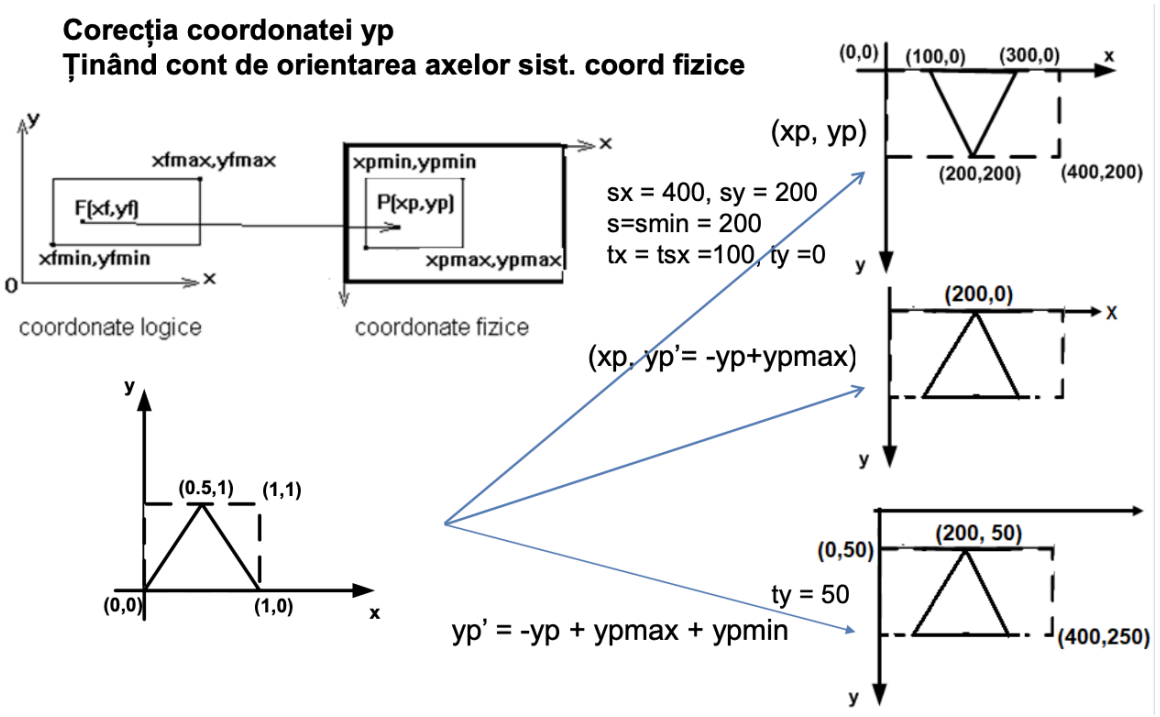
translația totală: tx + tx', ty + ty'

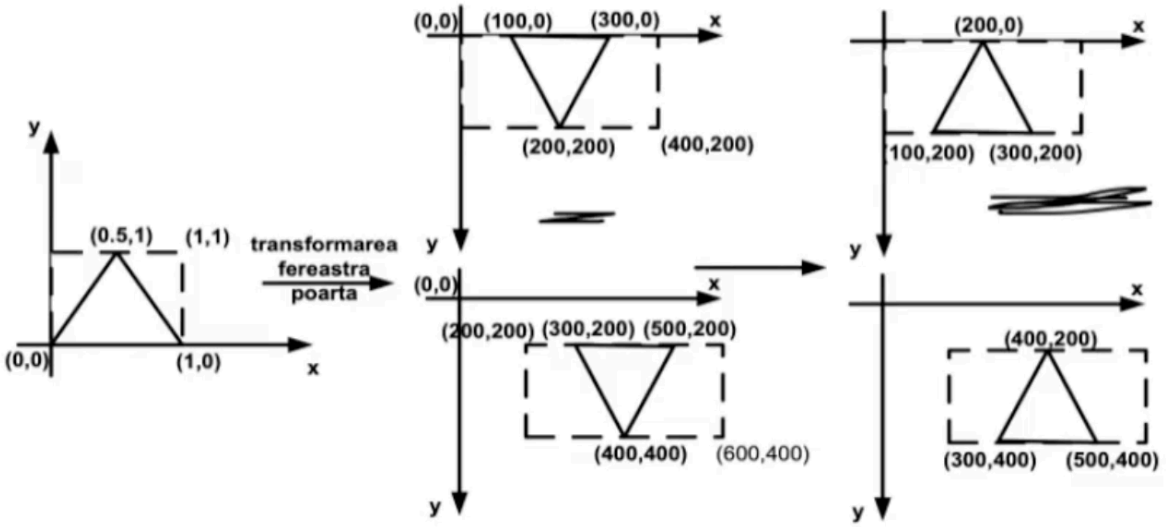
  - decuparea primitivelor aflate în afara ferestrei vizuale





- corecția coordonatei yp, ținând cont de orientarea axelor sistemului de coordonate fizice

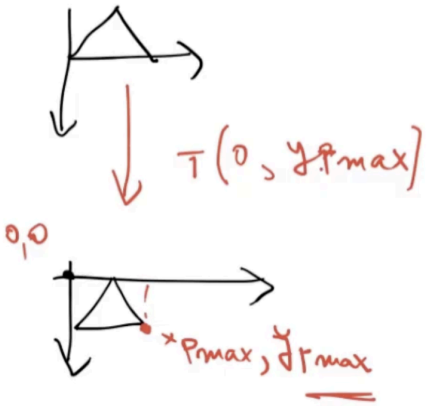




$$xp = xf * sx + tx$$

$$yp = yp\min + yp\max - (yf * sy + ty)$$

- pentru a ajunge la desenul subliniat (ultimul de pe prima linie) va trebui să realizăm o oglindire față de OX, iar apoi o translație pentru a aduce înapoi obiectul în poartă, o translație T(0, ypmax)



- formulele transformării fereastră-poartă atunci când axa OY a sistemului de coordonate fizice este orientată în jos

  xp = xf*sx + tx
  yp = -(yf*sy + ty) + ypmin + ypmax = -yf*sy + ypmin + ypmax - ty

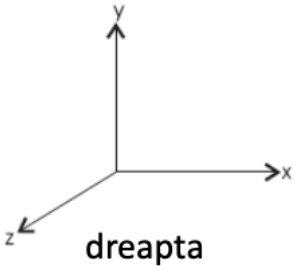
- exprimarea matriceală a transformării fereastră-poartă în cazul corecției coordonatei y

$$\begin{bmatrix} xp \\ yp \\ 1 \end{bmatrix} = M * \begin{bmatrix} xf \\ yf \\ 1 \end{bmatrix}$$

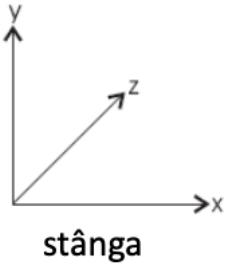
$$M = \begin{bmatrix} sx & 0 & tx \\ 0 & -sy & yp\min + yp\max - ty \\ 0 & 0 & 1 \end{bmatrix}$$

CURS 4 - TRANSFORMĂRI GEOMETRICE 3D

Sisteme de coordonate carteziene 3D



dreapta



stânga

- i, j, k - versorii direcțiilor axelor sistemului de coordonate
  - dreapta: k = (i x j) / |i x j|
  - stânga: k = (j x i) / |j x i|
- scena într-o aplicație OpenGL e descrisă în sistemul de coordonate carteziene 3D dreapta

Transformările geometrice 3D elementare

- translația, scalarea față de origine, rotațiile în jurul axelor principale ale sistemului de coordonate, oglindirea față de un plan al sistemului de coordonate, forfecarea față de originea sistemului de coordonate
- considerăm punctele din spațiu reprezentate prin vectori coloană, pentru a respecta convenția din OpenGL

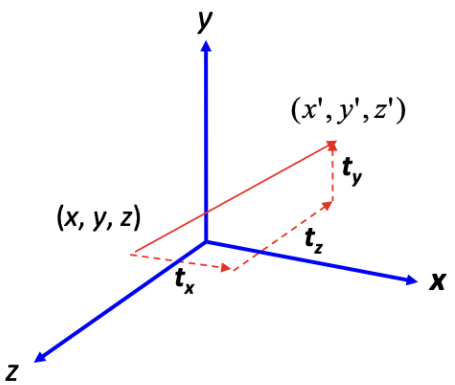
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

M este matricea de transformare a punctului (x, y, z), în coordonate omogene

Translația

- definită printr-un vector T[tx,ty,tz]

x' = x + tx
y' = y + ty
z' = z + tz



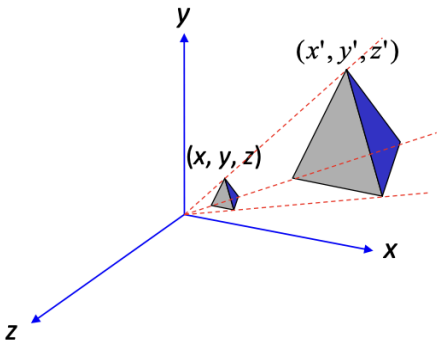
- matricea de translație T(tx',ty',tz')

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$T(t_x, t_y, t_z)$

Scalarea față de origine

- definită prin 3 numere reale sx, sy, sz - factorii de scalare pe cele 3 axe
- **sx = sy = sz => scalare uniformă**, altfel este neuniformă



x' = x*sx
y' = y*sy
z' = z*sz

- matricea de scalare S(sx,sy,sz)

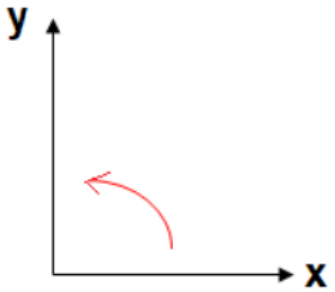
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$S(s_x, s_y, s_z)$

Rotația pozitivă (trigonometrică) în jurul axei OZ

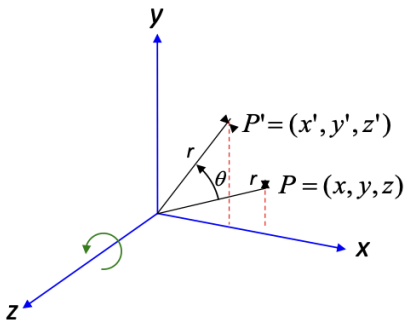
- este o rotație într-un plan de **z constant => prin rotație nu se modifică coordonata z**

- **rotația în planul XOY**



$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$
$$z' = z = 0$$

- **rotația punctului P(x,y,z) în jurul axei OZ cu unghiul θ**



$$x' = x\cos\theta - y\sin\theta$$
$$y' = x\sin\theta + y\cos\theta$$
$$z' = z$$

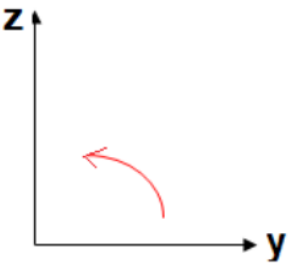
- matricea de rotație Rz(θ)

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & 0 \\ \sin\theta & \cos\theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

$R_z(\theta)$

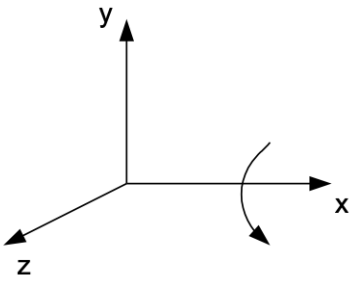
Rotația pozitivă în jurul axei OX

- **rotația în planul YOZ**



$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$
$$x' = x = 0$$

- rotația punctului P(x,y,z) în jurul axei OX cu unghiul **θ**



$$x' = x$$
$$y' = y\cos\theta - z\sin\theta$$
$$z' = y\sin\theta + z\cos\theta$$

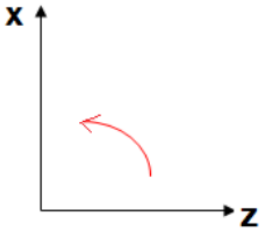
- matricea de rotație Rx(θ)

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta & 0 \\ 0 & \sin\theta & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_x(\theta)$

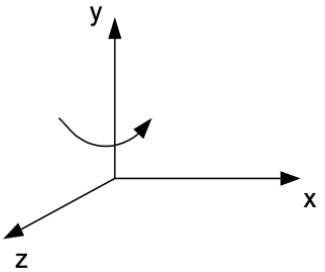
Rotația pozitivă în jurul axei OY

- rotația în planul ZOX



$$z' = z\cos\theta - x\sin\theta$$
$$x' = x\cos\theta + z\sin\theta$$
$$y' = y = 0$$

- rotația punctului P(x,y,z) în jurul axei OY cu unghiul **θ**



$$x' = x\cos\theta + z\sin\theta$$
$$y' = y$$
$$z' = -x\sin\theta + z\cos\theta$$

- matricea de rotație Ry(θ)

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin\theta & 0 & \cos\theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R_y(\theta)$

Alte transformări geometrice 3D elementare

- **Oglindirea față de un plan al sistemului de coordonate**
  - matricea de oglindire Oz

$$[O_z] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

- **Forfecarea față de origine**
  - **de-a lungul axei OZ - modifică x și y proporțional cu z**

x' = x + Fx*z
y' = y + Fy*z
z' = z



  - analog pentru forfecarea de-a lungul axei OX și a axei OY

  - **cazul general (forfecarea pe toate cele 3 axe)**

x' = x + y*d + z*g
y' = x*b + y + z*i
z' = x*c + y*f + z

$$[x'\ \ y'\ \ z'\ \ 1] = [x\ \ y\ \ z\ \ 1] * \begin{vmatrix} 1 & b & c & 0 \\ d & 1 & f & 0 \\ g & i & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Transformări geometrice 3D compuse

- **Scalarea față de un punct oarecare F(xf,yf,zf)**

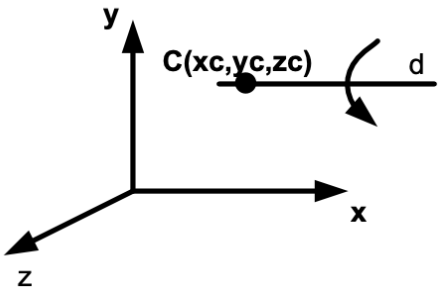
  T(xf,yf,zf)*S(sx,sy,sz)*T(-xf,-yf,-zf)

- **Rotația în jurul unei axe paralele cu o axă a sistemului de coordonate**
  - translația obiectului astfel încât axa de rotație să se suprapună peste o axă a sistemului de coordonate
  - rotația obiectului în jurul axei sistemului de coordonate
  - translația inversă primei
  - rezultă

  M = T(xd,yd,zd)*R(u)*T(-xd,-yd,-zd)
  u = unghiul de rotație
  xd, yd, zd = un punct de pe axa de rotație

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = M * \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

  - **rotația în jurul unei drepte paralele cu axa OX**



$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \mathbf{T(xc, yc, zc)\ R_X(u)\ T(-xc, -yc, -zc)} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Rotația în jurul unei drepte oarecare

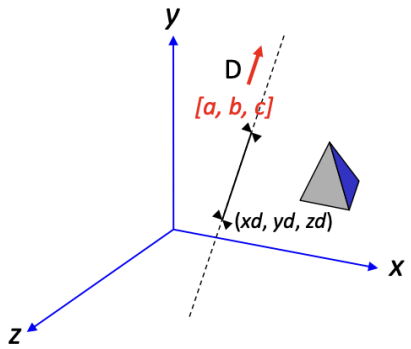
Se consideră **dreapta** dată printr-un **punct (xd,yd,zd)** și **direcția sa D[a, b, c]**

1. translație prin care dreapta va trece prin origine: **T(-xd,-yd,-zd)**
2. alinierea dreptei cu una dintre axele principale, de exemplu cu OZ

a. rotație în jurul axei OX, cu un unghi ux, prin care dreapta ajunge în planul XOZ: **Rox(ux)**
b. rotație în jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: **Roy(uy)**

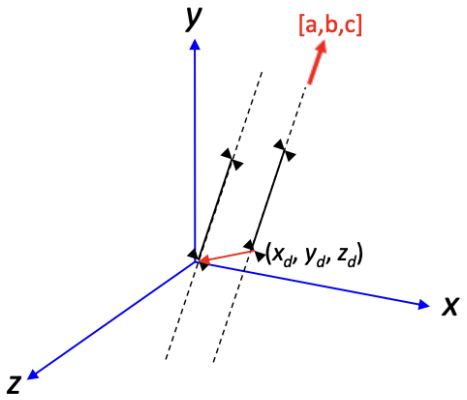
3. rotația cu unghiul dat, u, în jurul axei pe care s-a aliniat dreapta - rotație în jurul axei OZ: **Roz(u)**
4. transformarea inversă celei din pasul 2
   a. rotație în jurul axei OY. cu unghiul -uy: **Roy(-uy)**
   b. rotație în jurul axei OX, cu unghiul -ux: **Rox(-ux)**
5. transformarea inversă celei de la pasul 1: **T(xd,yd,zd)**

Exemplu



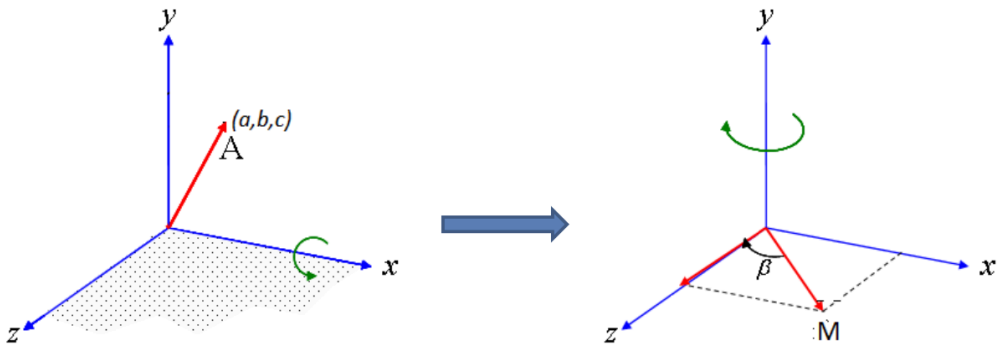
1. Translație astfel încât dreapta să treacă prin origine
2. Aliniere dreaptă cu una din axe
3. Rotația propriu-zisă
4. Transformarea inversă de la punctul 2
5. Transformarea inversă de la punctul 1

1. Translație



$$T = \begin{bmatrix} 1 & 0 & 0 & -xd \\ 0 & 1 & 0 & -yd \\ 0 & 0 & 1 & -zd \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

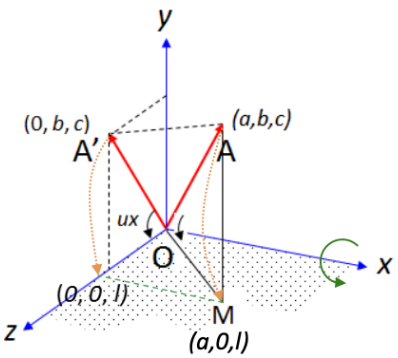
2. Aliniere *D* cu axa OZ



Rotație în jurul axei OX cu un unghi ux, prin care dreapta ajunge în planul XOZ: Rox(ux)

Rotație în jurul axei OY, cu un unghi uy, prin care dreapta se suprapune pe axa OZ: Roy(uy)

2.1. Rotația în jurul axei OX



$R_x(ux)$

A' – proiecția lui A în planul YOZ

Lungime (OA) = L; lungime (OA') = l;

$$L = \sqrt{a^2 + b^2 + c^2}$$

$$l = \sqrt{b^2 + c^2}$$

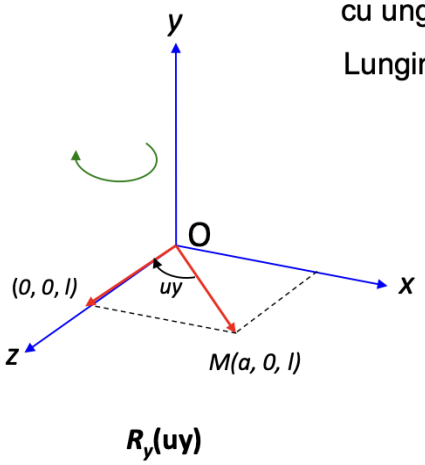
b = l*sin(ux)
c = l*cos(ux)

$$\sin(ux) = \frac{b}{l}$$

$$\cos(ux) = \frac{c}{l}$$

Prin rotația lui OA în jurul axei Ox cu unghiul ux, acesta va ajunge în planul XOZ

2.2. Rotația în jurul axei OY



$R_y(uy)$

M este punctul obținut prin rotația lui A în jurul axei Ox, cu unghiul ux.

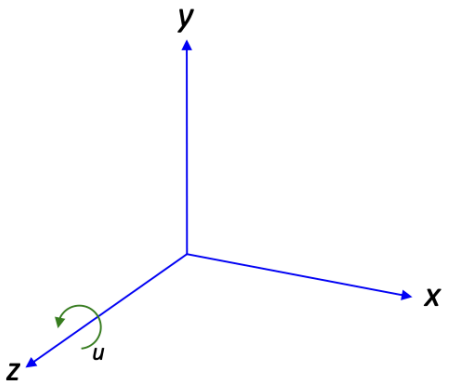
Lungime (OM) = Lungime (OA) = L

a = L*sin(-uy)

l = L*cos(-uy)

$$\sin(-uy) = a/L -> \sin(uy) = -a/L$$
$$\cos(-uy) = \cos(uy) = l/L$$

3. Rotația în jurul axei OZ cu unghiul dat



$$R_z(u) = \begin{bmatrix} \cos(u) & -\sin(u) & 0 & 0 \\ \sin(u) & \cos(u) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

CURS 5 - PROIECȚII

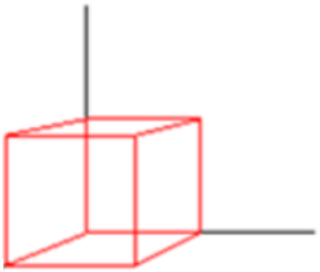
Definiție

Transformări dintr-un spațiu n-dimensional într-un spațiu k-dimensional, k < n

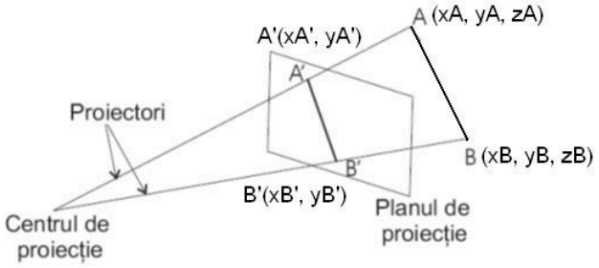
Proiecții R^3 -> R^2

- transformări din spațiul tridimensional într-un spațiu bidimensional
- se aplică vârfurilor obiectelor
- **nu modifică topologia (legăturile dintre vârfuri)**

- exemplu
  - proiecția unui cub în planul XOY
  - legăturile dintre vârfurile 2D sunt aceleași cu legăturile dintre vârfurile 3D proiectate

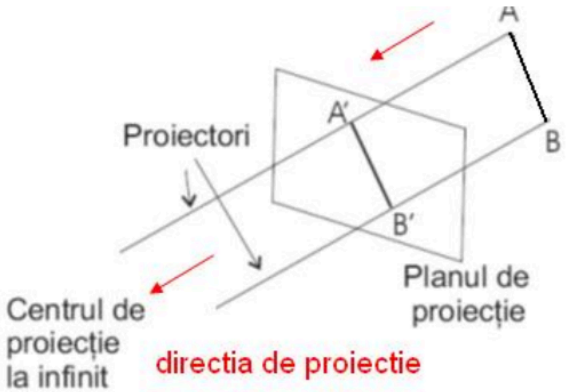


- proiecția unui vârf 3D într-un plan 2D este **punctul de intersecție** dintre **plan** și **proiectorul care pleacă din centrul de proiecție și trece prin vârf**



Segmentul A'-B' este proiecția segmentului A-B în planul de proiecție XOY (z=0).

- 2 clase de proiecții
  - **proiecții perspectivă** - centrul de proiecție este situat la distanță finită față de planul de proiecție (ca în figura de mai sus); proiectorii sunt drepte convergente în centrul de proiecție
  - **proiecții paralele** - centrul de proiecție este infinit; proiectorii sunt linii paralele care trec prin vârfurile obiectului proiectat și au direcția specificată **(vectorul direcție de proiecție)**

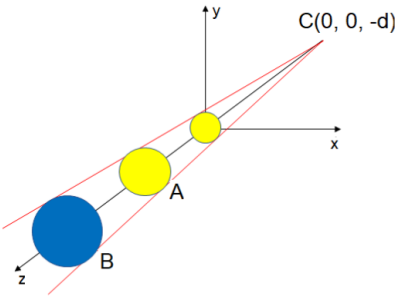


Proiecții perspectivă

- produc imagini asemănătoare celor obținute cu aparatele de fotografiat
- caracteristici
    - **efectul de micșorare** - dimensiunea obiectului în proiecția 2D este invers proporțională cu distanța de la centrul de proiecție la obiect

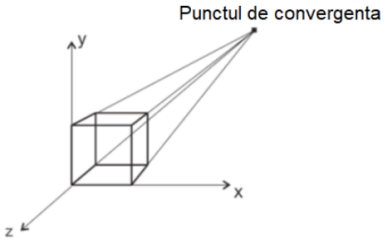
      A și B sunt 2 sfere, cu centrele pe axa OZ, care se proiectează în același disc în planul XOY

      raza sferei B = 2*raza sferei A
      Dist(centru_B-C) = 2*Dist(centru_A-C)



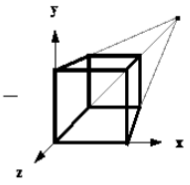
    - **modifică unghiurile dintre dreptele care nu sunt paralele cu planul de proiecție** - proiecțiile liniilor paralele care nu sunt paralele cu planul de proiecție converg către un punct din plan, numit punct de convergență

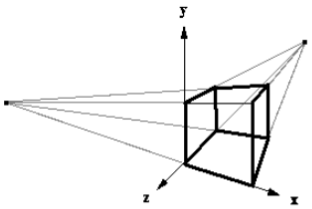
- punctul către care converg liniile paralele cu una dintre axele principale - **punct de convergență principal**

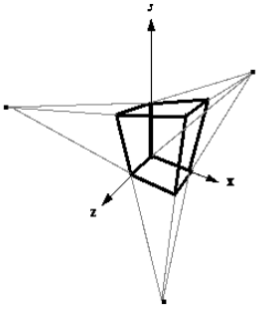


Proiecția perspectivă în planul XOY a unui cub cu fetele paralele cu planele principale

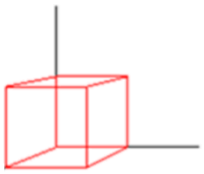
- pot fi efectuate cu unul, două sau trei puncte de convergență principale
- numărul de puncte de convergență principale = numărul de axe principale intersectate de planul de proiecție



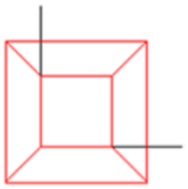




- exemple de proiecții perspectivă
    - proiecții ale unui cub cu latura egală cu 1, fețele paralele cu planele principale ale sistemului de coordonate și colțul de (xmin,ymin,zmin) în originea sistemului de coordonate

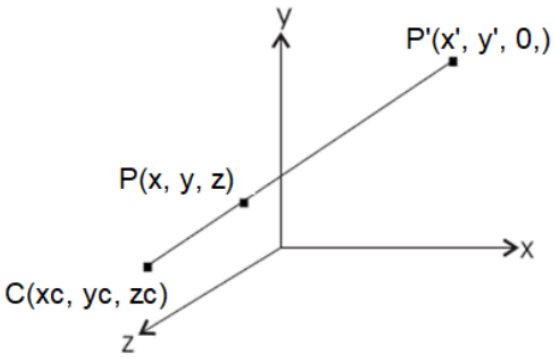


Proiecție perspectivă cu centrul în (5,2,8)



Proiecție perspectivă cu centrul în (0.5,0.5,2)

- **proiecția perspectivă în planul XOY**



C – centrul de proiecție
P – punctul proiectat
P' – proiecția lui P în XOY

ecuațiile parametrice ale proiectorului care trece prin P
x = xc + (x - xc)*t
y = yc + (y - yc)*t 0 <= t <=1 pentru punctele aflate pe segmentul C-P
z = zc + (z - zc)*t

proiecție în planul XOY => z = 0 => 0 = zc + (z - zc)*t' => t' = -zc/(z - zc)

=> x' = (xc*z - zc*x)/(z - zc)
y' = (yc*z - zc*y)/(z - zc)

Reprezentarea matriceală a transformării de proiecție

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix} \qquad Pperspectiva = \begin{vmatrix} \frac{-zc}{z-zc} & 0 & \frac{xc}{z-zc} & 0 \\ 0 & \frac{-zc}{z-zc} & \frac{yc}{z-zc} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Matricea proiecției depinde de coordonata z a punctului proiectat:

→ nu poate fi folosită pentru toate vârfurile aceluiași obiect

$$\begin{vmatrix} xw \\ yw \\ zw \\ w \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix} \qquad Pperspectiva = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z

yw = -zc*y + yc*z

zw = 0

w = z – zc

Coordonatele carteziene ale punctului P'
- împărțirea perspectivă -

x' = xw/w = (-zc*x + xc*z)/(z-zc)

y' = yw/w = (-zc*y + yc*z)/(z-zc)

z' = zw/w = 0

Reprezentarea matriceală a transformării de proiecție

$$\begin{vmatrix} xw \\ yw \\ zw \\ w \end{vmatrix} = Pperspectiva \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix} \qquad Pperspectiva = \begin{vmatrix} 1 & 0 & \frac{-xc}{zc} & 0 \\ 0 & 1 & \frac{-yc}{zc} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{-1}{zc} & 1 \end{vmatrix}$$

Coordonatele omogene ale punctului P'

xw = x + (-xc/zc)*z

yw = y + (-yc/zc)*z

zw = 0

w = (-1/zc)*z +1 = -(z – zc)/zc

Coordonatele carteziene ale punctului P'

x' = xw/w = -(x + (-xc/zc)*z)*zc/(z – zc)

= (-zc*x + xc*z)/(z-zc)

y' = yw/w = (-zc*y + yc*z)/(z-zc)

z' = zw/w = 0

Coordonatele omogene ale punctului obținut prin proiecție perspectiva au w ≠ 1.

Proiecția perspectivă în planul XOY(4)

Pentru conservarea informației de adâncime (necesară pentru eliminarea din imagine a părților nevizibile ale scenei 3D) se folosește matricea :

Coordonatele omogene ale punctului P'

xw = -zc*x + xc*z → x' = (-zc*x + xc*z)/(z-zc)

yw = -zc*y + yc*z → y' = (-zc*y + yc*z)/(z-zc)

zw = z → z' = z/(z – zc)

w = z – zc

$$Pperspectiva = \begin{vmatrix} -zc & 0 & xc & 0 \\ 0 & -zc & yc & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & -zc \end{vmatrix}$$

Proiecția perspectivă standard

Xc = 0, Yc = 0, Zc = - d

xw = x → x' = x*d/(z+d)

yw = y → y'= y*d/(z+d)

zw = z → z' = z/(z+d)

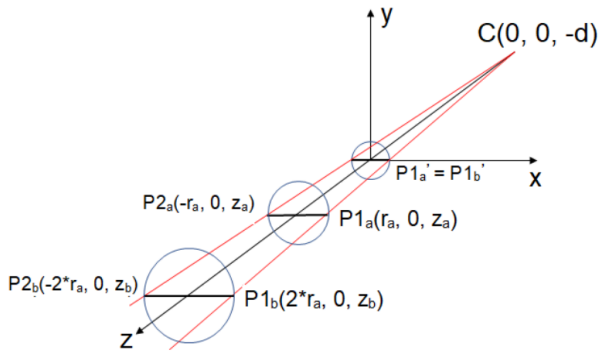
w = z/d + 1 = (z+d)/d

$$Ppers_standard = \begin{vmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & \frac{1}{d} & 1 \end{vmatrix}$$

Proiecția perspectivă în planul XOY - exemplu

A şi B sunt 2 sfere cu centrele pe axa OZ, A de rază r_a şi B de rază $2*r_a$.

Distanța de la centrul sferei B la centrul de proiecție C este de 2 ori mai mare decât distanța de la centrul sferei A la C.



Proiecția perspectivă standard

xw = x → x' = x*d/(z+d)

yw = y → y'= y*d/(z+d)

zw = z → z' = z/(z+d)

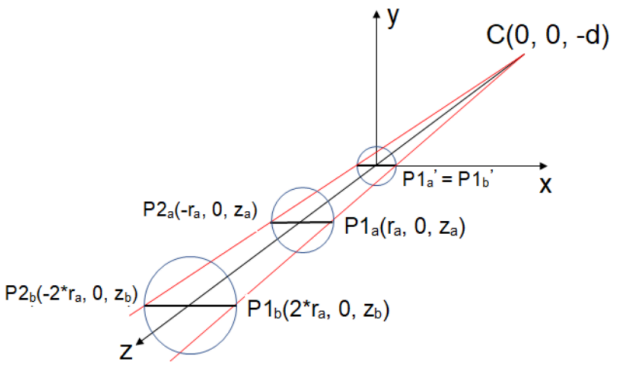
w = z/d + 1 = (z+d)/d

$x_P1_a' = r_a*d/(z_a + d)$; $y_P1_a' = 0 = y_P1_b'$

$x_P1_b' = 2*r_a*d/(z_b + d) = 2*r_a*d/(2*(z_a + d)) = r_a*d/(z_a + d) = x_P1_a'$

Analog se demonstrează că $P2_b' = P2_a'$

Utilizarea coordonatei de adâncime în eliminarea suprafețelor nevizibile din imagini



Proiecția perspectivă standard

zw = z → z' = z/(z+d)

$z_P1_a' = z_a/(z_a + d)$

$z_P1_b' = z_b/(z_b + d)$

Considerăm:

d = 50; z_a =100;

$z_b = 2(z_a + d)-d = 250$

$z_P1_a' = 100/150 = 0.666$;

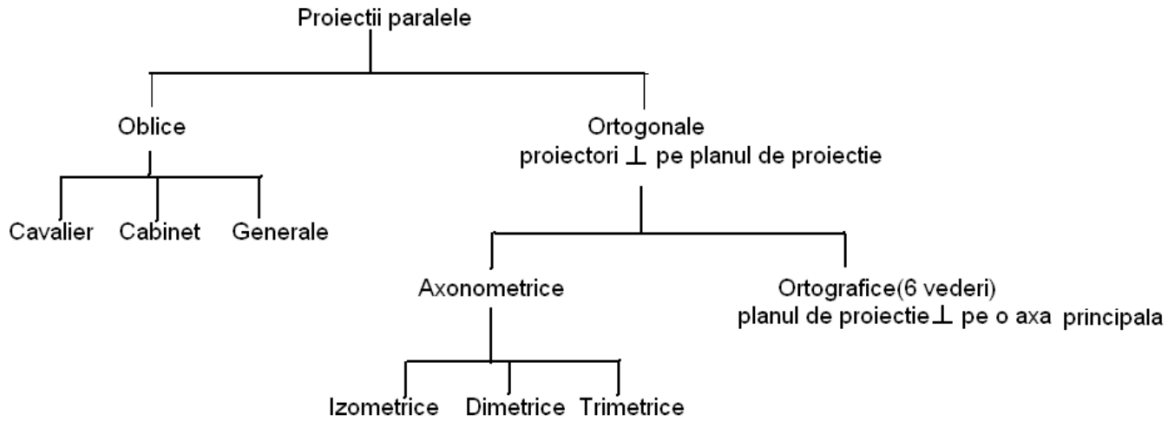
$z_P1_b' = 250/300 = 0.833$

Daca 2 puncte aparținând la 2 suprafețe 3D se proiectează în același punct, se va reda în imagine culoarea punctului mai apropiat de observator (centrul de proiecție):

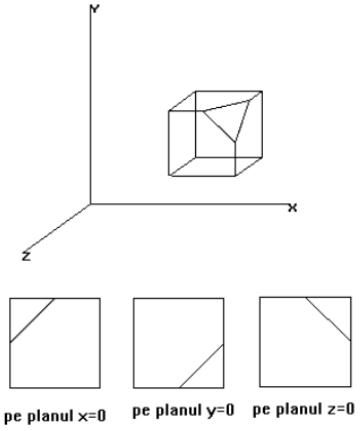
$z_P1_a' < z_P1_b'$ → discul în care se proiectează cele 2 sfere va fi afișat în culoarea sferei A

Proiecții paralele

- caracteristici
  - **proiectorii sunt drepte paralele pe direcție dată - direcția de proiecție**
  - **sunt transformări afine (conservă paralelismul liniilor)**
  - **unghiurile se conservă doar pentru fețele obiectului paralele cu planul de proiecție**
- clasificare după unghiul dintre proiectori și planul de proiecție
  - **proiecții ortogonale** - proiectorii sunt **perpendiculari pe planul de proiecție**
    - **ortografice** - planul de proiecție este || cu un plan principal (⊥ pe o axă principală)
    - **axonometrice** - planul de proiecție este oarecare
  - **proiecții oblice** - proiectorii **nu sunt perpendiculari** pe planul de proiecție; cazuri particulare:
    - proiecții Cavalier (unghiul dintre proiectori și plan = 45 grade)
    - proiecții Cabinet (unghiul dintre proiectori și plan = 63.43 grade)



- **Proiecții ortogonale - ortografice**
  Proiecții în planele sistemului de coordonate (planul din proiecție ⊥ pe o axă principală) => numite și **vederi**
  - în YOZ - vederea din stânga, vederea din dreapta
  - în XOZ - vederea de sus, vederea de jos
  - în XOY - vederea din față, vederea din spate

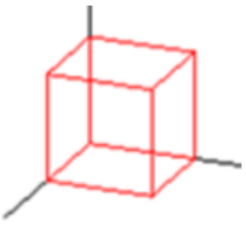


Proiectii ortografice ale unui cub cu un colt taiat

  - **conservă lungimile laturilor și unghiurile dintre laturi**
  - utile în desenul tehnic

- **Proiecții ortogonale - axonometrice**
  - planul de proiecție nu este perpendicular pe niciuna din axele sistemului de coordonate
  - redau mai multe fețe ale obiectului proiectat (ca și cele perspectivă) - alegând planul de proiecție se poate controla scalarea laturilor



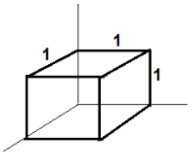


  - în funcție de unghiurile pe care planul de proiecție le face cu axele sistemului de coordonate
    - **proiecții izometrice**, planul face unghiuri egale cu toate cele 3 axe => prin proiecție, laturile sunt scalate cu factori egali pe cele 3 axe: **sx = sy = sz**
    - **proiecții dimetrice**, planul face unghiuri egale cu 2 dintre axe => laturile sunt scalate cu factori egali pe 2 axe: **sx = sy sau sx = sz sau sy = sz**
    - **proiecții trimetrice**, unghiurile dintre cele 3 axe și plan sunt diferite => factorii de scalare ai laturilor pe cele 3 axe sunt diferiți: **sx ≠ sy ≠ sz**

- Proiecții oblice
  - planul de proiecție este perpendicular pe o axă principală
  - direcția de proiecție nu este perpendiculară pe planul de proiecție
  - fețele || cu planul de proiecție se proiectează fără modificarea unghiurilor și a mărimilor laturilor
  - alegând direcția de proiecție se obțin cazurile particulare

❑ **Proiecții Cavalier**

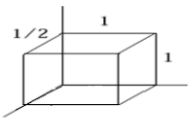
(unghiul proiectorilor cu planul de proiecție = 45 grade)



- Lungimea proiecției unei laturi perpendiculare pe plan este egală cu lungimea laturii 3D (conservă lungimea laturilor perpendiculare pe planul de proiecție)

❑ **Proiecții Cabinet**

(unghiul proiectorilor cu planul de proiecție = 63.43 grade)



- Lungimea proiecției unei laturi perpendiculare pe plan este egală cu jumătate din lungimea laturii 3D

- Proiecții paralele în planul XOY

Reprezentarea matriceală a proiecțiilor paralele:

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pparalela \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

$$Pparalela = \begin{vmatrix} 1 & 0 & \frac{-a}{c} & 0 \\ 0 & 1 & \frac{-b}{c} & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

Ca și în cazul proiecțiilor perspectivă, pentru conservarea informației de adâncime se folosește matricea →

$$Pparalela = \begin{vmatrix} 1 & 0 & \frac{-a}{c} & 0 \\ 0 & 1 & \frac{-b}{c} & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{vmatrix}$$

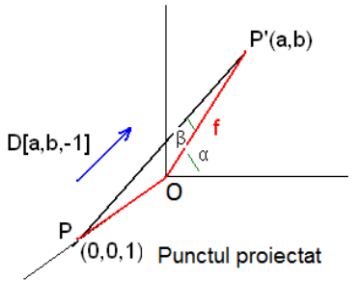
Cazuri particulare

1. Proiecția ortografică în XOY (proiectorii perpendiculari pe XOY):

D[a, b, c]
a= 0, b = 0 →

$$P_{O\text{-}XOY} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2. Proiecţii oblice:



c = -1 → $P_{OBI} = \begin{bmatrix} 1 & 0 & a & 0 \\ 0 & 1 & b & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

$P_{OBI} = \begin{bmatrix} 1 & 0 & f*\cos(\alpha) & 0 \\ 0 & 1 & f*\sin(\alpha) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$

OP de lungime 1 se proiectează în OP' de lungime f: → f este factorul de scalare al laturilor perpendiculare pe XOY (OP este perpendicular pe XOY).

tg(β) = 1/f

f = 0 → $\beta=90^0$: proiectie ortografica

f = 1 → $\beta=45^0$: proiectie Cavalier

f = 0.5 → $\beta=63.43^0$: proiectie Cabinet

α este un parametru liber

in mod uzual, $\alpha= 30^0$ sau 45^0

Exemple de proiecţii oblice





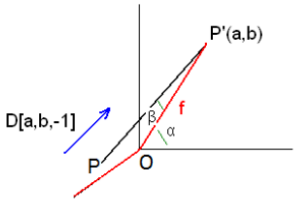
Proiecție Cavalier: f=1, beta=45, alfa=45

Direcția de proiecție: [cos(45),sin(45),-1]

Proiecție Cabinet: f=0.5, beta=63.43, alfa=45

Direcția de proiecție:

[0.5*cos(45),0.5*sin(45),-1]



3. Proiecţii axonometrice (1)

2 posibilități de exprimare matematică:

- Proiecție ortogonală într-un plan oarecare
  - obiect fix in spațiu, alegere plan astfel încât să obținem cele 3 cazuri (izometrice, dimetrice, generale):

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Pax(N,R) \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

- Transformare geometrică a obiectului (TA) urmată de proiecție ortografică în XOY (Po-xoy)– cu efect echivalent – plan de proiecție fix (XOY), poziționare obiect in spațiu:

$$\begin{vmatrix} x' \\ y' \\ z' \\ 1 \end{vmatrix} = Po\text{-}xoy * TA \begin{vmatrix} x \\ y \\ z \\ 1 \end{vmatrix}$$

TA constă din rotații în jurul axelor OX si OY. Particularizând unghiurile de rotație obținem cazurile de proiecții axonometrice: izometrice, dimetrice, generale.

Proiecţii axonometrice (2)

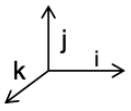
Exemplu:

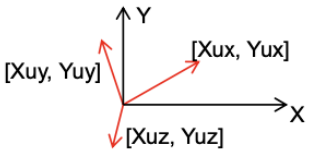
TA = Rx(ux) * Ry(uy) PA = Po-xoy* TA =

$$\begin{bmatrix} \cos(uy) & 0 & \sin(uy) & 0 \\ \sin(ux)^*\sin(uy) & \cos(ux) & -\cos(uy)^*\sin(ux) & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Se aplică transformarea PA versorilor axelor principale:

$$\begin{bmatrix} Xux & Xuy & Xuz \\ Yux & Yuy & Yuz \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} = PA \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$





Factorii de scalare a laturilor pe cele 3 axe sunt lungimile proiecțiilor celor trei vectori unitate:

$$sx = (Xux^2 + Yux^2)^{0.5}$$
$$sy = (Xuy^2 + Yuy^2)^{0.5}$$
$$sz = (Xuz^2 + Yuz^2)^{0.5}$$

Proiecții axonometrice (3)

Se deduc unghiurile de rotație în jurul axelor pentru obținerea celor 2 cazuri particulare de proiecții axonometrice: izometrice și dimetrice.

1. **Proiecții izometrice:**

- Se impune condiția: sx = sy = sz
- Rezultă: ux = +/- 35.26 grade ; uy = +/- 45 grade

2. **Proiecții dimetrice:** doi dintre factorii de scalare sunt egali

- Daca se impune condiția: sx = sy, rezultă:

  $ux = \arcsin(+/- sz/\sqrt{2}), \quad uy = \arcsin(+/- sz/\sqrt{(2 - sz^2)})$

  - sz poate fi ales intre 0 si 1;
  - pentru fiecare valoare a lui sz exista 4 proiecții dimetrice; de exemplu:
  - pentru sz = 0.5 : ux = +/- 20.705 grade, uy = +/- 22.208 grade

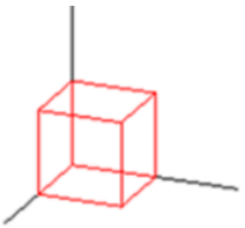
Exemple de proiecții axonometrice



Proiecție izometrică:

P_ort_XOY * R_Ox(35.26)* R_Oy(45)

sx = sy = sz



Proiecție dimetrică:

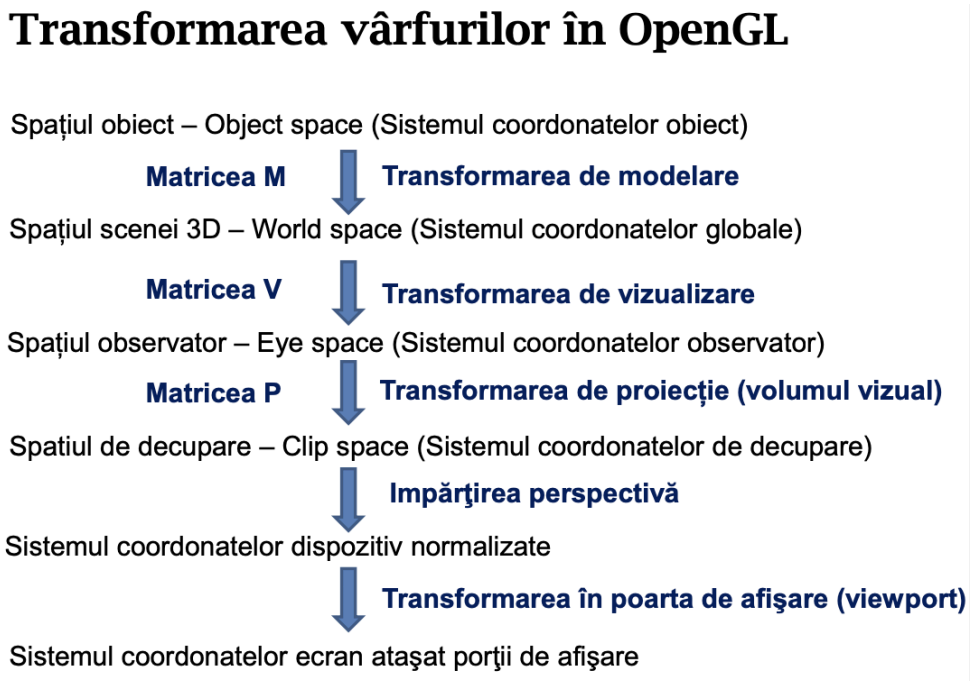
P_ort_XOY * R_Ox(20.705)* R_Oy(22.208)

sx = sy # sz

CURS 6 - TRANSFORMAREA DE VIZUALIZARE DIN SPAȚIUL 3D

Introducere

- scop = **obținerea unei vederi a modelului unei scene 3D pe o suprafață de afișare**
- poate fi comparată cu transformarea efectuată de o cameră foto asupra lumii reale pentru a se obține imaginea dintr-o fotografie; imaginea obținută depinde de
  - conținutul din lumea reală
  - poziția camerei (observatorului)
  - orientarea camerei
  - ceea ce vede ochiul prin vizorul camerei foto (volumul virtual)
- transformare **compusă din mai multe transformări**, între care și o proiecție R^3 -> R^2
- transformările se efectuează asupra vârfurilor obiectelor din scena 3D
- include și **operația de decupare (clipping)** a primitivelor grafice la marginea volumului vizual **(eliminarea părților din scena 3D care sunt în afara volumului vizual)**



Matricele transformării vârfurilor obiectelor 3D

- M - matricea de modelare: diferită de la obiect la obiect
  V - matricea de vizualizare: aceeași pentru întreaga scenă vizualizată
  P - matricea de proiecție: aceeași pentru întreaga scenă vizualizată

  $MV = V \cdot M$ (matricea "model-view")

  $MVP = P \cdot V \cdot M = P \cdot MV$ (matricea "model-view-projection")

- transformarea vârfurilor din coordonate obiect în coordonate de decupare (clip)

  $[xc, yc, zc]^T = MVP \cdot [xo, yo, zo]^T$

  Matricile sunt calculate în programul OpenGL folosind functii ale bibliotecii **glm** şi transferate la GPU

  Vârfurile sunt transformate în programul ***Vertex Shader executat pe GPU,***

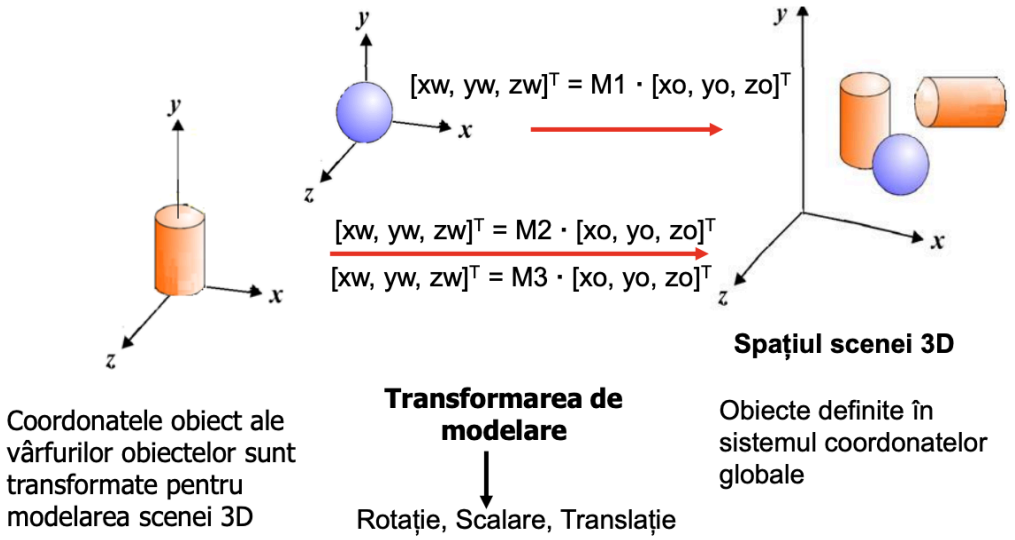
  folosindu-se matricea MVP

  Dupa asamblarea primitivelor, folosindu-se vârfurile transformate şi topologia transmisă la GPU de programul OpenGL, primitivele sunt decupate la frontiera volumului vizual.

Transformarea de modelare

- **Spațiul obiect (Sistemul coordonatelor obiect)**
  - în mod uzual, fiecare obiect este definit în propriul său sistem de coordonate (sistemul coordonatelor locale - coordonate obiect)
  - exemple: cubul, sfera, scheletul unui personaj animat, etc.
  - de regulă, obiectul e definit centrat în originea sistemului de coordonate
  - avantaje: ușurința modelării și reutilizarea modelelor obiectelor 3D
- **Spațiul scenei 3D - World Space (Sistemul coordonatelor globale)**
  - spațiul în care e compusă scena 3D
  - raportat la un sistem de referință global - sistem de coordonate carteziene 3D "dreapta"
- scop = dimensionarea și poziționarea obiectelor scenei 3D într-un cadru imagine

- este o **transformare geometrică compusă**



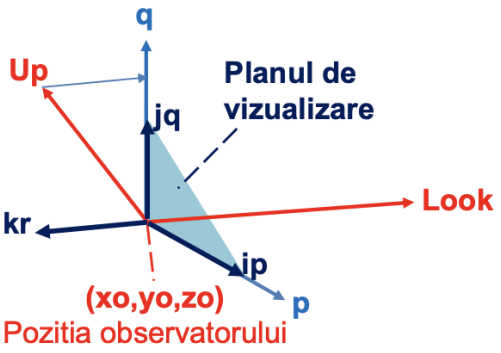
Transformarea de vizualizare

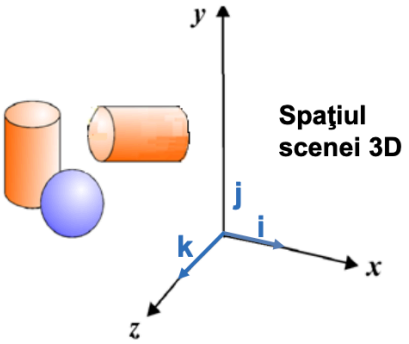
din sistemul de coordonate globale => în sistemul de coordonate observator

- o imagine este o vedere obținută dintr-un punct al spațiului 3D, similară unei fotografii
- depinde de
  - **poziția camerei** (observatorului) în spațiul lumii reale
  - **orientarea camerei** - direcția în care privește observatorul și rotația camerei față de axa sa

- **spațiul observator** = spațiul 3D văzut din poziția observatorului pe direcția în care acesta privește
- **obiectele 3D** din acest spațiu = raportate la un **sistem de coordonate cu originea în poziția observatorului (camerei virtuale)**
- sistemul de coordonate observator este **definit prin**
  - **originea sa** - poziția camerei virtuale în spațiul coordonatelor globale (xo,yo,zo)
  - **direcția în care privește observatorul** (spre scena 3D) - vectorul Look, reprezintă normala la planul de vizualizare (N)
  - **direcția "sus" a planului de vizualizare** (vectorul Up), care **determină direcția axei OY a sistemului de coordonate observator**: reprezintă rotația camerei în jurul axei privirii (Look)

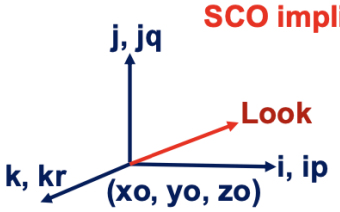
Sistemul de coordonate observator





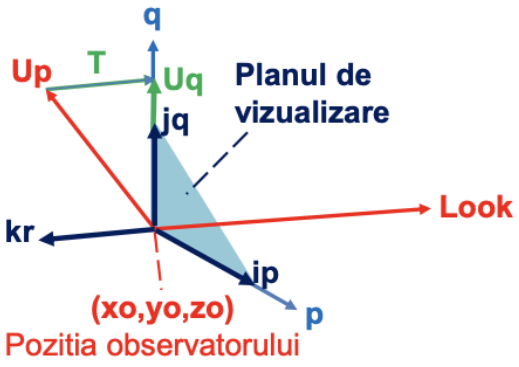
- (xo, yo, zo): originea sistemului de coordonate observator (SCO)
- Look – normala la planul de vizualizare
- axa q: axa verticală a planului de vizualizare – are direcția data de proiecția vectorului UP în planul de vizualizare
- (ip,jq,kr) formează un sistem de coordonate 3D dreapta

SCO implicit



ip, jq, kr: versorii axelor sistemului de coordonate observator coincid cu cei ai sist. coordonatelor globale

Determinarea axelor sistemului de coordonate observator



kr = - Look/|Look|

Uq: proiectia ortogonala a vectorului Up in planul de vizualizare

Uq dă direcţia axei verticale a planului de vizualizare

Uq = Up + T

T || (-kr) → T = s*(-kr) = - s*kr (s un scalar oarecare) → Uq = Up - s*kr

Uq perpendicular pe kr → Uq·kr = 0 → (Up - s*kr)·kr =0 → Up·kr - s*kr·kr = 0 → s = Up·kr

Uq = Up – (Up·kr)·kr

jq = Uq/|Uq| este versorul axei verticale a planului de vizualizare

ip = (jq x kr) / |(jq x kr)| este versorul axei orizontale a planului de vizualizare

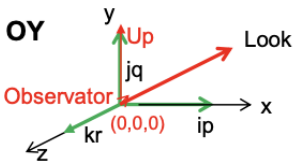
(ip, jq, kr) formează un sistem de coordonate carteziene 3D dreapta

Calculul matricei transformării de vizualizare

Sistemul de coordonate observator implicit în OpenGL

- poziţia observatorului: **originea sistemului de coordonate globale**
- direcţia în care priveşte observatorul: **direcţia negativă a axei OZ**
- direcţia sus a planului de vizualizare: **direcţia pozitivă a axei OY**

Cu aceste valori implicite, transformarea de vizualizare este transformarea identica.



Sistemul de coordonate observator implicit în OpenGL

Pentru calculul matricei transformarii de vizualizare poate fi folosita functia **lookAt:**

glm::mat4 **V** = glm::lookAt (glm::vec3 (xo,yo,zo), glm::vec3 (Px,Py,Pz), glm::vec3 (Upx, Upy, Upz))

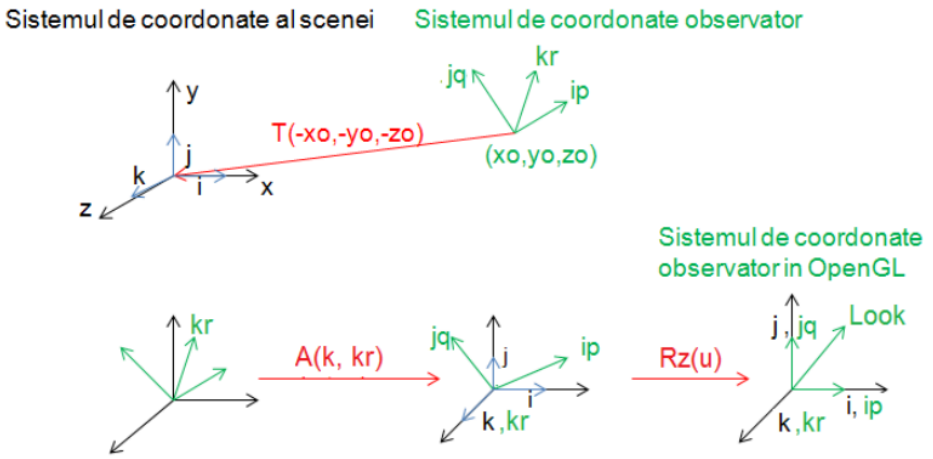
(xo, yo, zo): **poziţia observatorului**

(Px,Py,Pz): punctul din scena 3D către care privește observatorul;

vectorul (Px,Py,Pz) - (xo,yo,zo) reprezintă **vectorul Look**

(Upx, Upy, Upz): **vectorul « sus » al planului de vizualizare (Up)**

Matricea transformării de vizualizare



- A(k,kr) este o transformare prin care se aliniază axele z ale celor 2 sisteme de coordonate: constă din 2 rotații – o rotație în jurul axei X și una în jurul axei Y.
- Prin rotația în jurul axei z (Rz) se aliniază celelalte 2 axe.

Matricea transformării de vizualizare: V = Rz(u) * A(k,kr) * T(-xo,-yo,-zo)

Transformarea de modelare și vizualizare

din sistemul coordonate obiect => în sistemul de coordonate observator

- transformare compusă (model-view) $MV = V \bullet M$
  M - matricea de modelare folosită pentru transformarea unui obiect în spațiul scenei
  V - matricea de vizualizare a scenei

- transformările vârfurilor din coordonate obiect în coordonate observator (eye)

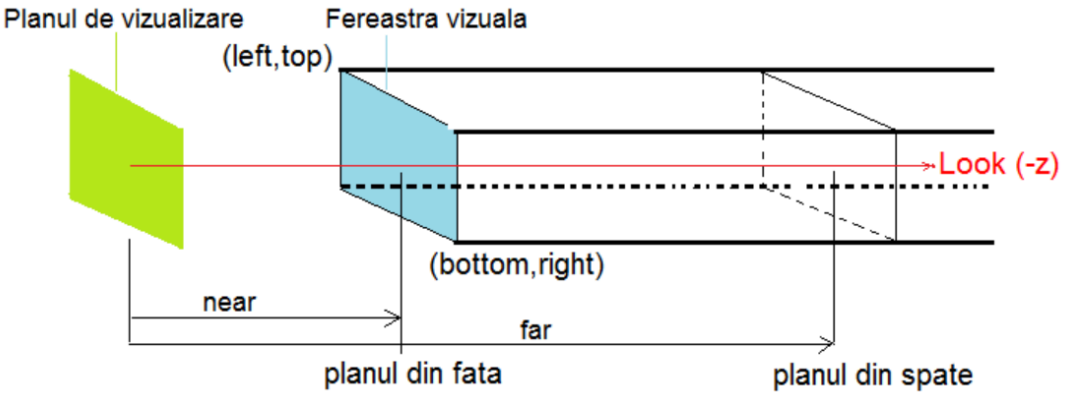
  $$[xe\ ye\ \ ze\ we]^T = MV \bullet [xo\ yo\ zo\ wo]^T$$

- pentru calculele de iluminare a obiectelor scenei 3D se folosesc vectori normali atașați vârfurilor obiectelor; calculele de iluminare se fac, de regulă, în sistemul coordonatelor observator (sau global)

Transformarea de proiecție

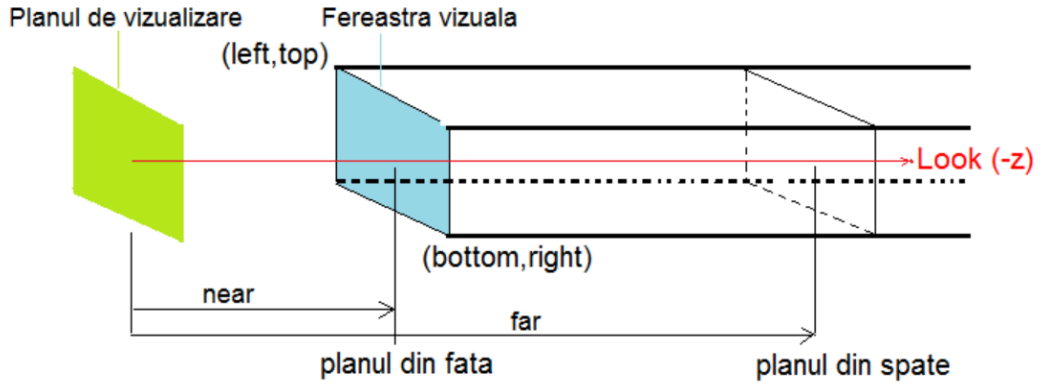
- **determinată de volumul vizual definit de programator**

- **Volumul vizual al proiecției ortografice**: paralelipiped dreptunghic cu laturile paralele cu axa z a sistemului de coordonate observator (SCO), delimitat în adâncime de planul din față (near) și planul din spate (far) al vederii; near și far sunt distanțe (pozitive) față de planul de vizualizare măsurate pe axa z negativă a SCO

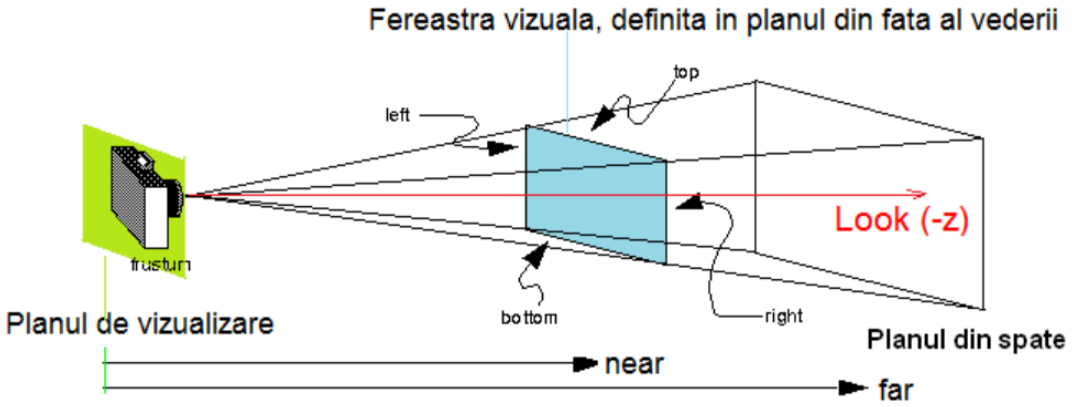


  - **coordonatele colțurilor ferestrei vizuale** sunt **raportate la sistemul de coordonate ale planului din față**, care are **axele paralele cu cele ale planului de vizualizare**

- **Proiecția ortografică**: se efectuează **în fereastra vizuală a obiectelor cuprinse în volumul vizual**; primitivele grafice **situate complet în afara** volumului vizual sunt **eliminate** iar cele care **intersectează volumul** sunt **decupate la frontiera volumului**, creându-se **noi primitive**



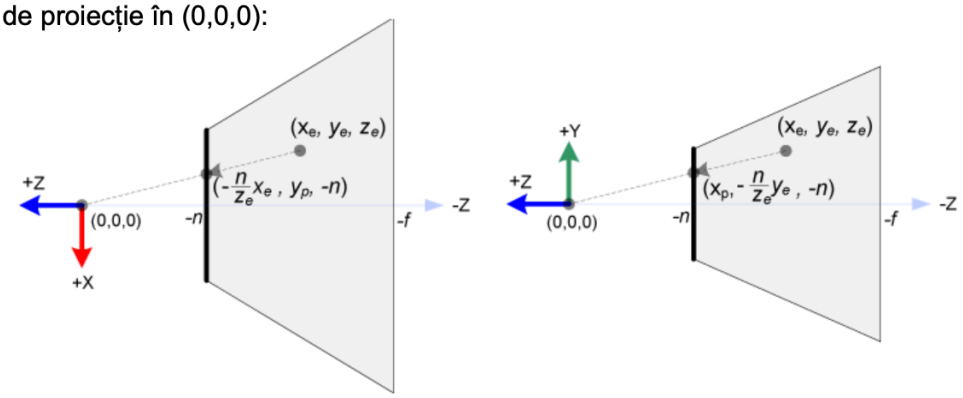
- **Volumul vizual al proiecției perspectivă (View frustum)**: trunchi de piramidă **delimitat în adâncime de planul din față** (near) și **planul din spate** (far) al vederii, cu **vârful în poziția observatorului**



  - se efectuează proiecția perspectivă în planul din față (în fereastra vizuală), cu **centrul de proiecție în poziția observatorului**, a obiectelor cuprinse în volumul vizual; obiectele situate **în afara volumului vizual** sunt **eliminate sau decupate la frontiera volumului vizual**

- Proiecția se efectuează în fereastra vizuală din planul din față: planul z = -near (în SCO)

  Proiecția perspectivă a unui punct din SCO, (xe,ye,ze), pe planul din față al vederii, cu centrul de proiecție în (0,0,0):



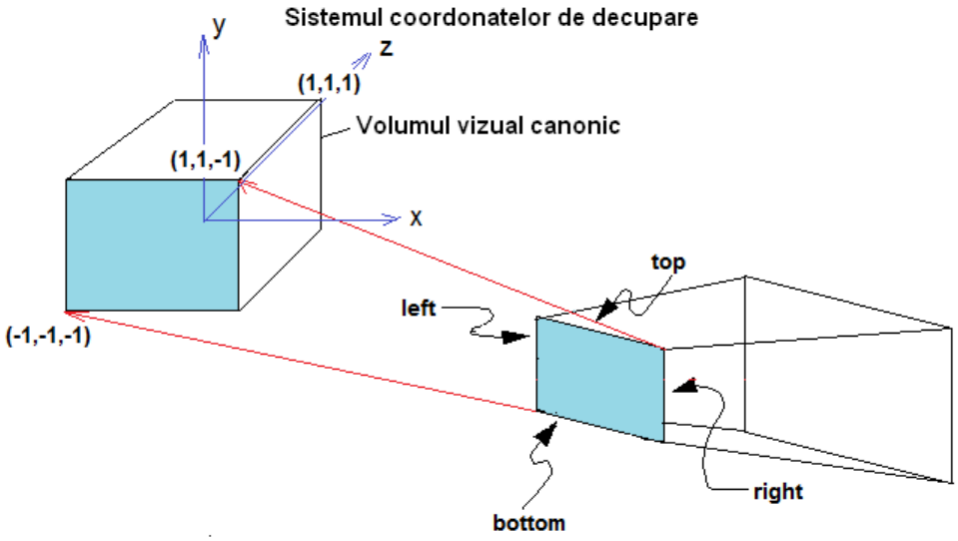
  xp = (-near/ze)*xe, yp =(-near/ze)*ye – se pot obține aplicând formulele proiecției perspectivă cu centrul de proiecție în (0,0,0) și planul de proiectie este z=-near.

  Proiectia ortografică a unui punct din sist. coordonatelor observator, (xe,ye,ze), în fereastra vizuală:

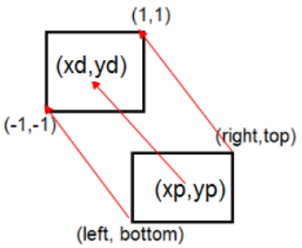
  xp = xe, yp = ye, zp = -near.

- transformarea de proiecție este însoțită de operația de decupare a primitivelor grafice la marginile volumului vizual
- pentru simplificarea calculelor de decupare, volumul vizual definit de programator este transformat în **volumul vizual canonic**: cub cu latura de 2 unități, centrat în originea sistemului coordonatelor de decupare, cu laturile paralele cu axele
- transformarea prin care volumul vizualizat este transformat în volumul vizual canonic se numește **transformarea de normalizare**
- după transformarea vârfurilor prin transformarea de proiecție, vârfurile sunt unite pe baza topologiei definite de programator, formând primitive grafice 3D => acestea sunt decupate la marginile volumului vizual canonic

- Normalizarea
  - **sistemul coordonatelor de decupare** - sistem de coordonate carteziene 3D stânga
  - prin transformarea de normalizare, colțurile ferestrei 2D din planul de proiecție, (left,bottom,-near) și (right,top,-near) sunt mapate pe colțurile stânga-jos și dreapta-sus ale ferestrei 2D din sistemul coordonatelor de decupare, adică (-1,-1,-1) și (1,1,-1)
    planul z = -near se mapează pe z = -1, iar z = -far se mapează pe z = 1



- Impunând condiția (left,bottom) → (-1,-1) și (right,top) → (1,1), rezultă transformarea punctelor din planul din față, (xp,yp), în sistemul coordonatelor de decupare – este o transformare freastră-poartă:



xd = 2*xp/(right – left) – (right + left)/(right – left)

yd = 2*yp/(top – bottom) – (top + bottom)/(top-bottom)

Înlocuind xp și yp rezultate din proiecția perspectivă în planul -near:

xd = (xe*2*near/(right – left) + ze*(right + left)/(right – left))/(-ze)

yd = (ye*2*near/(top – bottom) + ze*(top + bottom)/(top-bottom))/(-ze)

Pentru transformarea coordonatelor z se impun condițiile: -near → -1, -far → 1; rezultă:

zd = (-ze*(far+near)/(far-near) – 2*far*near/(far-near)) / (-ze)

(xd,yd,zd) sunt coordonate carteziene în sistemul coordonatelor de decupare (clip coordinates); folosind matricea transformării de proiecție rezultă coordonate omogene:

[xc, yc, zc, w] → xd = xc/w; yd = yc/w; zd = zc/w ; w = - ze

Obs: w trebuie sa fie ≠ 0 → punctul proiectat nu poate fi în planul camerei (centrul de proi.)

Matricea transformării de proiecție pentru proiecția perspectivă:

$$P = \begin{bmatrix} \frac{2*near}{right-left} & 0 & A & 0 \\ 0 & \frac{2*near}{top-bottom} & B & 0 \\ 0 & 0 & C & D \\ 0 & 0 & -1 & 0 \end{bmatrix} \quad A = \frac{right+left}{right-left} \quad B = \frac{top+bottom}{top-bottom}$$

$$C = -\frac{far+near}{far-near} \quad D = -\frac{2*far*near}{far-near}$$

Matricea transformării de proiecție pentru proiecția ortografică:

xp=xe si yp=ye → xc = 2*xe/(right – left) – (right + left)/(right – left)

yc = 2*ye/(top – bottom) – (top + bottom)/(top-bottom)

$$P = \begin{bmatrix} \frac{2}{right-left} & 0 & 0 & -A \\ 0 & \frac{2}{top-bottom} & 0 & -B \\ 0 & 0 & \frac{-2}{far-near} & C \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$[xc\ yc\ \ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$

Matricile de proiecție pot fi obținute apelând următoarele funcții din biblioteca glm:

glm::ortho(float left, float right, float bottom, float top, float zNear, float zFar);

glm::frustum (float left, float right, float bottom, float top, float zNear, float zFar);

glm::perspective (float fov, float aspect, float znear, float zfar);

Crează o matrice de proiecție pentru un volum perspectivă simetric:

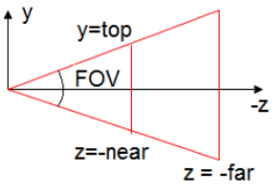
- fov: Field of View

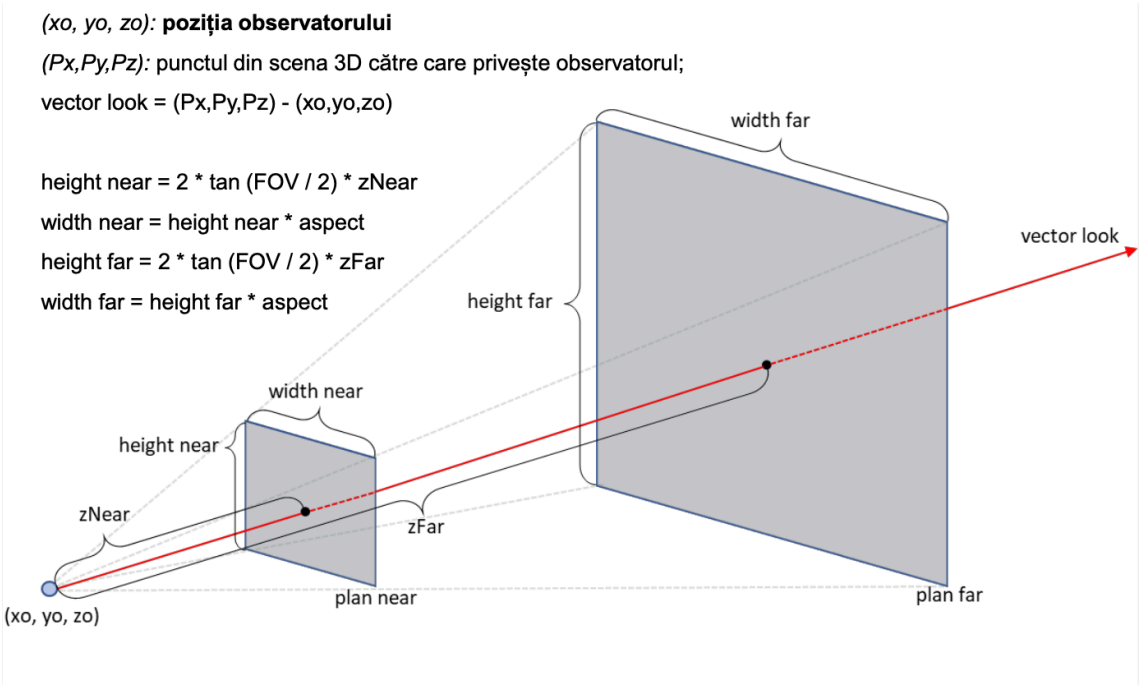
  tan(FOV/2) = top/near

- aspect: aspect ratio

  aspect ratio = (right-left)/(top-bottom)

  = raportul dintre lățimea și înălțimea ferestrei vizuale





- **din coordonate observator => în coordonate de decupare**

  $[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$ (clip coordinates) ← eye coordinates)

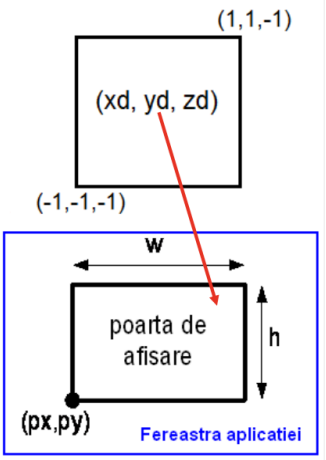
  - împărțirea perspectivă
    coordonate omogene => coordonate carteziene: xd = xc/w, yd = yc/w, zd = zc/w
    (xd,yd,zd) coordonate dispozitiv normalizate
  - vârfurile rezultate prin împărțirea perspectivă sunt asamblate în primitive grafice care sunt decupate la frontiera volumului vizual canonic

Transformarea de modelare-vizualizare-proiecție

- prin înmulțirea matricelor transformărilor de modelare, vizualizare și proiecție se obține matricea numită **"model-view-projection"**, astfel încât asupra vârfurilor definite în sistemul coordonatelor obiect poate fi efectuată o singură transformare

$$MVP = P \bullet V \bullet M \quad \text{(matricea ModelViewProjection)}$$

Transformarea în poarta de afișare



- este o **transformare fereastră-poartă** în care
  - **fereastra** este fereastra 2D din sistemul coordonatelor de decupare, având colțurile în (-1,-1,-1) - (1,1,-1)
  - **poarta** este definită folosind funcția

    void **glViewport**(GLint Px, GLint Py, GLsizei width, GLsizei height);

    (*Px, Py*) reprezintă coordonate relative la colțul stânga-jos al ferestrei aplicației (în pixeli). Valorile implicite : (0,0).
    width, height reprezintă lăţimea / înălţimea porţii de afişare.
    Valorile implicite sunt: lăţimea şi înălţimea ferestrei aplicației.

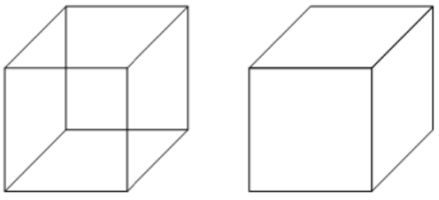
- este definită astfel

  xv = (width/2)*xd + ox , ox = px+ width/2
  yv = (height/2)*yd + oy, oy = py + height/2

  zv = ((f-n)/2)*zd + (n+f)/2 // conservă coordonata z pentru eliminarea părților nevizibile
  n (near) şi f (far) au valorile implicite 0.0 respectiv 1.0, pentru care 0<=zv<=1
  n și f pot fi modificate la valori cuprinse în intervalul [0,1] folosind funcţia ***DepthRange()***

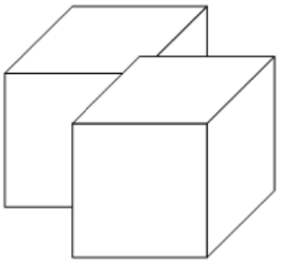
CURS 7 - ELIMINAREA PĂRȚILOR NEVIZIBILE ALE SCENELOR 3D DIN IMAGINI

Introducere



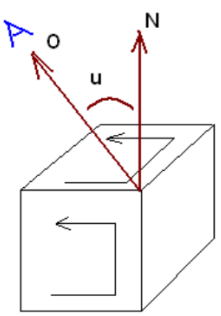
- **vizibilitatea obiectelor** dintr-o scenă 3D depinde de
  - poziția observatorului și direcția în care privește
  - volumul vizual
  - poziționarea obiectelor din scenă unul față de celălalt
- **părțile unei scene 3D nevizibile într-o imagine sunt**
  - părți aflate total sau parțial în afara volumului vizual

    - fețe auto-obturate
    - părți obturate (ascunse) de obiecte ale scenei, aflate în fața lor în raport cu observatorul



- **efectuată la nivel de**
    - **obiecte sau grupuri de obiecte - Object culling**
        - **frustum culling** - eliminarea din banda grafică a obiectelor sau grupurilor de obiecte care sunt **complet în afara volumului vizual**
        - **occlusion culling** - eliminarea din banda grafică a obiectelor sau grupurilor de obiecte care sunt **complet obturate de alte obiecte**
        - nu e efectuată de OpenGL/GPU
        - poate fi efectuată prin algoritmi implementați într-o bibliotecă folosită de aplicația 3D (motorul grafic)
    - **fețe ale obiectelor - Back face culling**: eliminarea din banda grafică **a fețelor auto-obturate ale obiectelor** (este efectuată pe GPU)
    - **primitive grafice (poligoane, linii)**
        - eliminarea primitivelor aflate **total sau parțial în afara volumului vizual prin operația de decupare (clipping)** efectuată de GPU
        - eliminarea părților obturate (ascunse) ale primitivelor grafice: hidden surface/hidden line removal
        - poate fi efectuată prin algoritmi implementați într-o bibliotecă folosită de aplicația grafică 3D (motorul grafic): BSP, Pictorului, etc.
        - este efectuată de GPU, ca operație raster la nivel de fragment (algoritmul Z-buffer)

Eliminarea fețelor auto-obturate pentru poliedre convexe - Back face culling



- poate reduce substanțial numărul de poligoane rasterizate
- **algoritmul**
    - observatorul și obiectul sunt raportați la același sistem de coordonate

- conturul fiecărei fețe a obiectului este orientat în sens trigonometric atunci când poliedrul este văzut din exterior
- observatorul este situat în afara obiectului

- fața e **vizibilă** dacă **0 <= u < 90 de grade sau 0 < cos(u) <= 1**
  N - normala la față
  O - vectorul orientat către observator

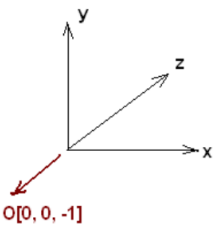
  Produsul scalar: N•O = |N| * |O| * cos(u) → cos(u) = N•O / (|N| * |O|) = Nu•Ou

  Condiția de vizibilitate: Nu • Ou > 0 (produsul scalar al versorilor)

Determinarea fețelor auto-obturate pentru poliedre convexe

- **Determinarea fețelor auto-obturate poate fi efectuată**
  1. în sistemul coordonatelor globale (în care este definit și observatorul - camera virtuală)
  2. în sistemul de coordonate observator
  3. în sistemul coordonatelor de decupare (după aplicarea transformării de proiecție)

În cazul 3



Sistemul coordonatelor de decupare

Nu • Ou = [nx, ny, nz] • [0, 0, -1] = - nz

Condiția de vizibilitate devine: nz <0

După transformarea de proiecție poziția observatorului este la infinit, pe axa z negativă.

Vectorul orientat către observator este același pentru orice punct al unui obiect

- Calculul normalei la o față a poliedrului

  Fie V1(x1, y1, z1), V2(x2, y2, z2), V3(x3, y3, z3), 3 varfuri succesive ale conturului feței.
  Normala la față se poate obține calculând produsul vectorial: (V2-V1)x(V3-V1)

$$\mathbf{N} = (V2-V1)\times(V3-V1) = \begin{vmatrix} i & j & k \\ x2-x1 & y2-y1 & z2-z1 \\ x3-x1 & y3-y1 & z3-z1 \end{vmatrix} = [(y2-y1)*(z3-z1) - (z2-z1)*(y3-y1)]*\mathbf{i} - [(x2-x1)*(z3-z1) - (z2-z1)*(x3-x1)]*\mathbf{j} + [(x2-x1)*(y3-y1) - (y2-y1)*(x3-x1)]*\mathbf{k}$$

  unde **i, j, k** sunt versorii direcțiilor axelor sistemului de coordonate carteziene 3D

  - normalele
    - pot fi calculate în sistemul coordonatelor obiect și atașate vârfurilor în OpenGL => sunt transformate în sistemul coordonatelor de decupare, în vertex shader

        - nu sunt atașate vârfurilor: sunt calculate pe GPU după asamblarea primitivelor, folosind vârfurile transformate în sistemul coordonatelor de decupare
- pentru **eliminarea fețelor auto-obturate** se apelează următoarele funcții OpenGL

  glCullFace(GL_FRONT / GL_BACK / GL_FRONT_AND_BACK)
  - eliminare fețe: din față / din spate / toate (nu vor fi afișate fețe, ci numai alte primitive - puncte, linii)

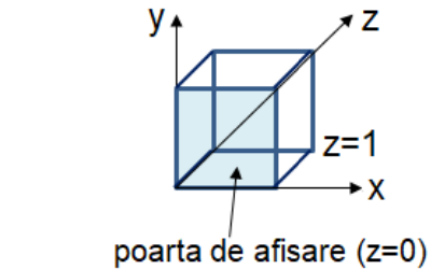
  glFrontFace(GL_CCW/GL_CW)
  - specifică orientarea fețelor din față: trigonometrică / sensul acelor de ceas

  - dacă ciclul de vârfuri al fiecărei fețe este orientat trigonometric atunci când obiectul este privit din exteriorul său
    - atunci când **observatorul** este **în spațiul exterior obiectului** se va apela **glFrontFace(GL_CCW)**
    - atunci când **observatorul** este **în spațiul interior obiectului** se va apela **glFrontFace(GL_CW)** - văzute din interior, fețele obiectului au conturul orientat anti-trigonometric

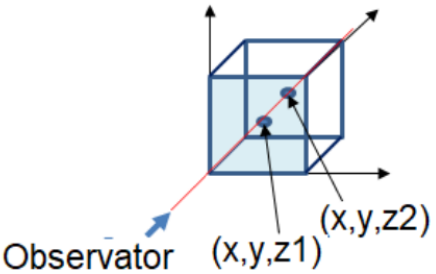
Algoritmul z-buffer

- **algoritm de eliminare a fragmentelor nevizibile ale primitivelor grafice**
- executat de GPU, fiind **integrat în procesul de rasterizare a primitivelor**

- **observatorul este la infinit**, pe axa z negativă
- asupra primitivelor se efectuează **proiecția ortografică în planul XOY (poarta de afișare)**



- dacă două fragmente aflate pe același proiector, rezultate din rasterizarea a două primitive diferite, au **coordonata z diferită => fragmentul având coordonata z mai mică va fi afișat în pixelul (x,y)**



- primitivele grafice (triunghiuri, linii) în care a fost descompusă scena 3D sunt rasterizate în ordinea în care au fost transmise din programul de aplicație (nu sunt sortate)

- **buffer-ul imagine (frame buffer)** este actualizat pe măsura rasterizării primitivelor, astfel:

  if (fragmentul curent se proiectează în pixelul (x,y) && z-fragment < z-fragment afișat în pixelul (x,y))
  actualizează z-fragment afișat în pixelul (x,y), ia valoarea z a fragmentului curent
  actualizează culoarea pixelului (x,y) în buffer-ul imagine, ia culoarea fragmentului curent

=> este necesar să se memoreze coordonatele z ale fragmentelor afișate în pixelii imaginii

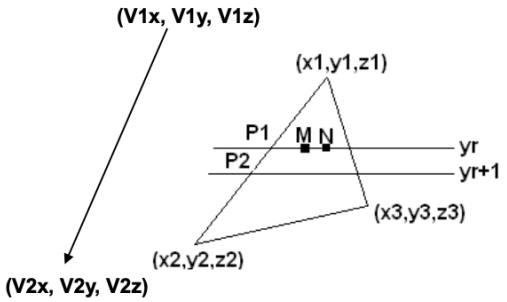
=> **Z-buffer** - tablou bidimensional cu număr de elemente egal cu numărul de pixeli ai suprafeței de afișare; în memoria GPU

Pași algoritm

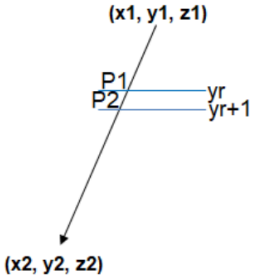
- inițializează buffer-ul imagine la culoarea de fond
- inițializează Z-buffer la coordonata z maximă (z = 1)
- pentru fiecare fragment f(x,y,z) rezultat din rasterizarea unei primitive
  - calculează culoarea fragmentului (în fragment shader)

```
// testul de vizibilitate (adâncime)
daca z < Z-buffer[y][x] atunci
    {
        Z-buffer[y][x] = z
        *actualizează culoarea pixelului (x,y) in buffer-ul imagine folosind culoarea
        fragmentului f
    }
```

- calculul coordonatei z a unui fragment
  coordonatele z ale fragmentelor rezultate la rasterizarea vectorilor și a suprafețelor triunghiulare se obțin prin calcul incremental, în algoritmul de rasterizare



Calculul coordonatei z la rasterizarea vectorilor



x = x1 + t(x2-x1)
y = y1 + t(y2-y1) Ecuațiile parametrice ale vectorului
z = z1 + t(z2-z1)

yr, yr+1: ordonatele a 2 linii raster (ecran) care intersectează vectorul

y(P1) = yr = y1 + t'(y2-y1) → t' = (yr-y1)/(y2-y1)

z(P1) = z1 + (yr-y1)(z2-z1)/(y2-y1) = z1 + (yr-y1)*K,
K este o constantă la rasterizarea vectorului

z(P2) = z1 + (yr+1-y1)*K = z1 + (yr-y1)*K + K

Z(P2) = Z(P1) + K → coordonatele z ale fragmentelor de pe vector se obțin prin calcul incremental.

Calculul coordonatei z la rasterizarea poligoanelor

P1(xP1, yr, zP1), P2(xP2, yr+1, zP2)

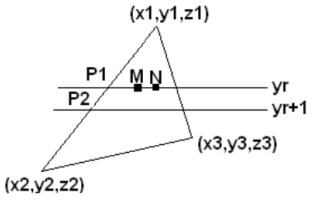
m- panta laturii (x1,y1,z1) – (x2,y2,z2)

xP2 = xP1 + 1/m (vezi curs - rasterizare)

A*x + B*y + C*z + D = 0 ec. planului poligonului

C*z = -A*x - B*y - D

zP1 = (-A*xP1 - B*yr –D)/C



zP2 = (-A*xP2 - B*(yr +1) – D)/C = (-A(xP1 + 1/m) - B*yr –B – D)/C = zP1 + (-A/m –B) → **zP2 = zP1 + K1**

M(xM, yr, zM), N(xM + 1, yr, zN)

zM = (-A*xM – B*yr – D)/ C

zN = (-A*(xM + 1) – B*yr – D)/ C = zM + (-A/C) → **zN = zM + K2**

Observație: atunci când un vector se suprapune pe suprafața unui poligon pot apărea defecte datorită modului diferit în care se calculează coordonatele z ale fragmentelor care se afișează în aceiași pixeli!

Aprecieri asupra algoritmului Z-buffer

- numărul de comparații de valori z pentru un pixel = numărul de fragmente care se proiectează în acel pixel
  timpul de calcul tinde să devină independent de numărul de poligoane - în medie, numărul de pixeli acoperiți de un poligon este invers proporțional cu numărul de poligoane
- "depth complexity" - numărul de suprascrieri ale unui pixel în buffer-ul imagine, la generarea unui cadru imagine
  calcul culoare fragment - model iluminare, utilizare texturi
- pentru o precizie bună, este necesară o reprezentare pe 32 de biți a valorilor din z-buffer

Algoritmul BSP - Binary Space Partitioning

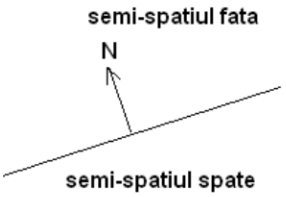
Introducere

- inițial, arborele BSP a fost folosit numai pentru operația de eliminare a obiectelor (grupurilor de primitive) nevizibile din banda grafică (object culling) => ulterior a devenit algoritm pentru reprezentarea și afișarea scenelor 3D cu "eliminarea" părților nevizibile
- eficient pentru eliminarea părților nevizibile ale poligoanelor (hidden surface removal) dar poate fi folosit și pentru eliminarea părților nevizibile ale liniilor dintr-o scenă compusă din poligoane și linii

Intrarea: lista primitivelor care compun scena 3D (nu are importanță din care obiect face parte fiecare primitivă)

- scena 3D este reprezentată printr-un **arbore binar: arborele BSP**
- **arborele BSP al scenei este independent de poziția observatorului (reprezintă scena în sistemul coordonatelor globale)**

Construirea arborelui BSP al unei scene



- fiecare nod al arborelui corespunde unui plan de partiționare al spațiului 3D (de regulă, planul unui poligon al scenei)
- fiecare plan de partiționare împarte spațiul în 2 semi-spații
  - cel din fața planului (de aceeași parte cu normala la plan)
  - cel din spatele planului

1. **se începe cu un poligon oarecare din listă (de regulă primul), pentru care se creează nodul rădăcinei al arborelui**
    a. **primitivele din semi-spațiul "față" formează "lista-față", care va genera subarborele "față" al nodului**
    b. **primitivele din semi-spațiul "spate" formează "lista-spate", care va genera subarborele "spate" al nodului**
2. **se alege un poligon din lista-față și se creează nodul rădăcină al subarborelui "față", etc.**

Afișarea arborelui BSP al unei scene

- ține cont de poziția observatorului
- afișare "front-to-back" și "back-to-front"
- afișare "back-to-front" - se începe cu primitiva cea mai îndepărtată de observator
    - se pornește din rădăcina arborelui și se avansează în arbore până la frunze, în funcție de poziția observatorului față de planul atașat fiecărui nod
    - în fiecare nod - se avansează în subarborele nodului care este opus observatorului față de plan

Eliminarea obiectelor aflate în afara volumului vizual (frustum culling)

- dacă planul de partiționare al unui nod nu intersectează volumul vizual, atunci numai subarborele aflat de aceeași parte cu volumul vizual va fi afișat, celălalt subarbore fiind eliminat din banda grafică - la fel și primitivele atașate nodului (fiind în afara volumului vizual)

Aprecieri

- arborele nu trebuie să fie construit pentru fiecare cadru imagine: avantaj pentru scenele statice
- modificarea poziției observatorului => execuția funcției de afișare (dacă scena nu s-a modificat)
- permite eliminarea din banda grafică a unui număr mare de poligoane care sunt în afara volumului vizual (Object culling)
- pentru obiectele dinamice se pot face diverse adaptări ale algoritmului, de exemplu:
    - pentru fiecare cadru imagine se inserează în arborele BSP cu obiecte statice, nodurile corespunzând obiectelor dinamice

- se transmite arborele BSP static nemodificat și obiectele dinamice vizibile, eliminarea părților obturate de acestea fiind realizată în algoritmul z-buffer (la rasterizare)
- **dacă primitivele grafice sunt trimise în banda grafică în ordinea "din spate în față", nu mai este necesar testul de adâncime la rasterizarea primitivelor (fragmentele care se proiectează în același pixel sunt suprascrise)**
- testul de adâncime efectuat asupra fragmentelor la rasterizare poate fi activat/dezactivat:
  glEnable(GL_DEPTH_TEST) / glDisable(GL_DEPTH_TEST)
- dacă se folosește algoritmul BSP cu afișare «back-to-front» împreună cu testul de adâncime, numărul de suprascrieri ale pixelilor este mai mare decât în cazul unei ordini aleatorii a poligoanelor (algoritmul z-buffer fără BSP)
- dacă se folosește BSP cu afișare «front-to-back» împreună cu testul de adâncime, afișarea poate fi mai rapidă decât folosind BSP «back-to-front» fără testul de adâncime, în funcție de amplasarea poligoanelor în scenă
- construirea și afișarea arborelui BSP sunt funcții de management al scenei 3D, implementate de regulă într-un motor grafic 3D

Algoritmul pictorului

Introducere

- algoritm "hidden surface removal"
- intrare: lista poligoanelor care alcătuiesc scena 3D, transformate în spațiul de afișare

Ideea: eliminarea părților nevizibile se rezolvă prin afișare "din spate în față" , la fel ca în pictură

- nu este necesar testul de adâncime la rasterizarea poligoanelor

Forma generală a algoritmului

1. se calculează "extensia" fiecărui poligon din listă pe axele OX, OY, OZ (xmin, ymin, zmin - xmax, ymax, zmax): paralelipipedul încadrator al poligonului, cu fețele paralele cu planele principale ale sistemului de coordonate
2. se ordonează poligoanele crescător după coordonata zmin a fiecărui poligon: primul din listă va fi cel mai apropiat de observator

3. se descompun poligoanele ale căror extensii pe axa OZ se suprapun, astfel încât extensiile lor pe axa OZ să fie disjuncte
    a. sunt multe aplicații în care acest pas nu e necesar, poligoanele fiind amplasate în plane de Z-constant: cartografie, generarea straturilor circuitelor imprimate, etc.
    b. poate fi optimizat, știind că nu întotdeauna atunci când extensiile pe axa OZ se suprapun este necesară descompunerea poligoanelor; descompunerea nu este necesară dacă
        i. extensiile pe axa OX nu se suprapun
        ii. extensiile pe axa OY nu se suprapun
        iii. proiecțiile poligoanelor nu se suprapun în imagine
    c. testele se efectuează progresiv, în funcție de complexitatea calculelor presupuse
4. se transmit în banda grafică poligoanele începând cu ultimul din listă (cel mai îndepărtat de observator)

- **principalul efort de calcul - sortarea listei de poligoane și descompunerea, dacă este necesară**

CURS 8 - ELIMINAREA PĂRȚILOR NEVIZIBILE ALE SCENELOR 3D DIN IMAGINI - STRUCTURI DE DATE DE ACCELERARE

Tehnici de eliminare a obiectelor nevizibile - Frustum culling

- scopul: eliminarea din banda grafică a obiectelor sau grupurilor de obiecte care sunt complet în afara volumului vizual
- se bazează pe reprezentarea scenei printr-o structură de date spațială

Structuri de date spațiale pentru managementul scenei 3D

- ierarhie de volume încadratoare (BVH)
- arbore BSP
- arbore AA-BSP
- arbore octal

- managementul scenei este realizat în cadrul unui motor grafic 3D, implementat peste OpenGL / Direct 3D
- structurile de date sunt utile și pentru alte operații: mai sunt numite **structuri de date de accelerare**

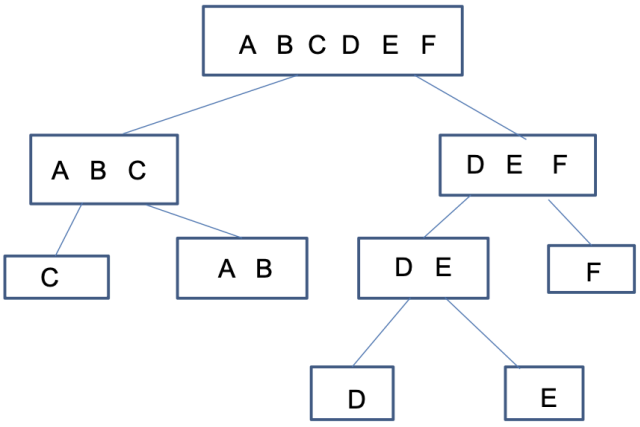
Scena - Ierarhie de volume încadratoare (Bounding Volume Hierarchy - BVH)

- scena este reprezentată printr-un arbore care are în noduri **volume încadratoare de obiecte sau grupuri de obiecte**: sfera, paralelipipedul aliniat cu axele (Axis-Aligned minimum Bounding Box: AABB), poliedrul convex minimal încadrator, cilindrul, elipsoidul

Construirea arborelui

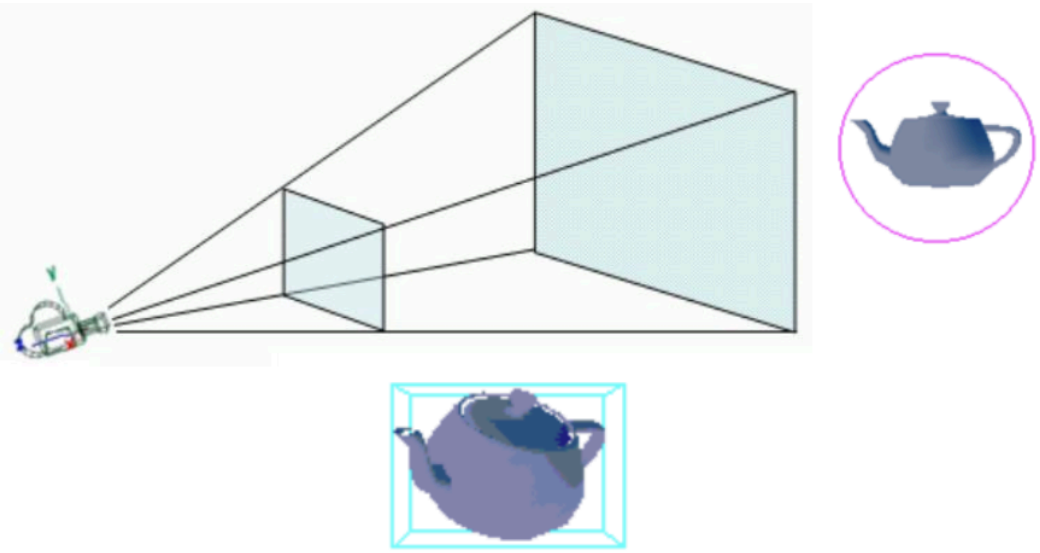
1. **Top-down** - rădăcina arborelui este asociată cu întreaga scenă
   a. Se partiționează setul de obiecte în 2 sau mai multe subseturi și se alocă fiecare subset unui volum încadrator - rezultă primul nivel al arborelui
   b. Se continuă procedura recursiv la nivelul fiecărui grup de pe nivelul anterior până când se ajunge la un singur obiect (sau grup de obiecte) pe nod - nod frunză
   c. Metoda este simplă dar, în general, nu produce arbori optimi.
2. **Bottom-up**
   a. Se pleacă de la setul de obiecte și se definește un volum încadrator pentru fiecare obiect sau grup de obiecte – nodurile frunză.
   b. Se grupează 2 sau mai multe volume nod frunză, rezultând alte noduri ale arborelui.
   c. Se continuă gruparea până când totul este grupat într-un singur volum – nodul rădăcină

- **construirea bottom-up produce arbori mai eficienți decât cea top-down**



Afișarea scenei 3D folosind arborele BVH

- dacă volumul încadrator al unui nod este **în afara volumului vizual** atunci **obiectele din subarborele său NU sunt trimise în banda grafică**



- testele de vizibilitate încep cu rădăcina arborelui
- dacă **volumul încadrator al nodului curent** este **complet în afara volumului vizual**
    - atunci **se revine pe nivelul anterior al arborelu**i (întregul grup de obiecte din subarborele nodului curent este exclus din banda grafică)
    - **altfel se coboară în arbore**
- dacă se ajunge la **o frunză al cărei volum încadrator nu este în afara volumului vizual**
    - atunci **grupul de primitive (obiectul) asociat frunzei** este **trimis în banda grafică**; se revine pe nivelul anterior al arborelui

- pentru simplitate și eficiență, **testele de vizibilitate se pot face în spațiul coordonatelor de decupare**, transformând volumele încadratoare prin matricea MVP și testând intersecția cu volumul vizual canonic (cub cu laturile paralele cu axele)

- structura BVH este frecvent utilizată pentru detecția coliziunilor dintre obiectele unei scene dinamice

Eficiența structurii BVH privind eliminarea obiectelor nevizibile din banda grafică

- depinde mult de felul în care sunt grupate obiectele în volume încadratoare; poate fi eficientă pentru anumite poziții ale observatorului și ineficientă pentru alte poziții
- alegerea tipului de volum încadrator pentru un nod:
    - să aibă o formă simplă, astfel încât testele de intersecție cu volumul vizual să fie simple și rapide
    - să încadreze minimal obiectul/obiectele

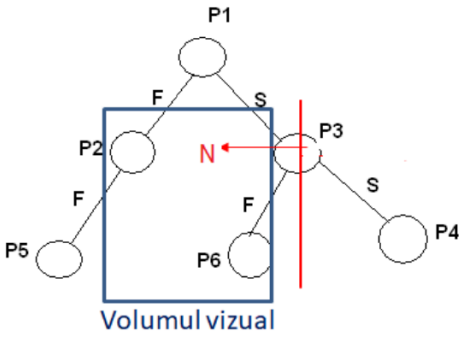
Proprietăți dorite ale ierarhiei de volume încadratoare

- volumele nodurilor dintr-un subarbore trebuie sa fie apropiate spațial, cu atât mai apropiate cu cât sunt situate mai jos în arbore
- volumul fiecărui nod să încadreze minimal obiectul/grupul de obiecte corespunzător
- volumul încadrator al întregii scene trebuie să fie minimal
- suprapunerea volumelor nodurilor vecine să fie minimală

Scena - Arbore BSP (Polygon Aligned BSP tree)

- planele de partiționare sunt planele poligoanelor (Polygon Aligned - PA BSP tree)
- **arborele scenei se construiește ca în algoritmul BSP pentru eliminarea suprafețelor nevizibile**
- eliminarea grupurilor de primitive nevizibile

- Dacă **planul de divizare atașat unui nod nu intersectează volumul vizual**, atunci **subarborele atașat nodului care este situat de partea opusă a planului față de volumul vizual este complet nevizibil și poate fi eliminat din procesul de redare**; la fel și primitivele atașate nodului.



Eliminarea se efectuează **în spațiul coordonatelor globale**, folosind volumul vizual definit de aplicație (trunchi de piramidă sau paralelipiped dreptunghic)

Scena - Axis Aligned BSP Tree

- planele de partiționare sunt aliniate cu axele sistemului de coordonate 3D - perpendiculare pe cele 3 axe: Axis-Aligned BSP Tree (AA-BSP)
- AA-BSP tree este un k-d tree (k-dimensional tree): arbore binar în care fiecare nod corespunde unei coordonate într-un spațiu k-dimensional

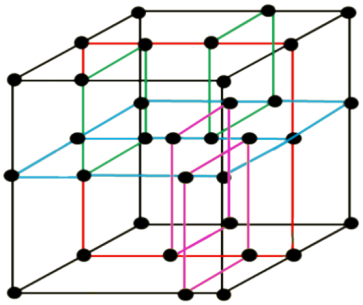
  Pentru k = 3, un nod va corespunde uneia dintre coordonatele x, y sau z; de exemplu, dacă planul de partiționare al unui nod este perpendicular pe axa OX, atunci nodul este asociat coordonatei x a planului

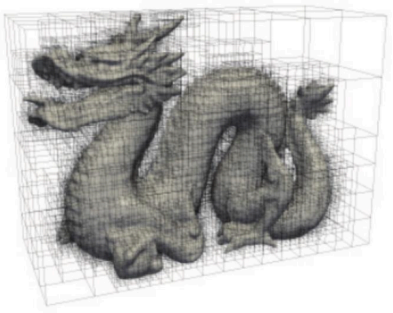
Construirea arborelui

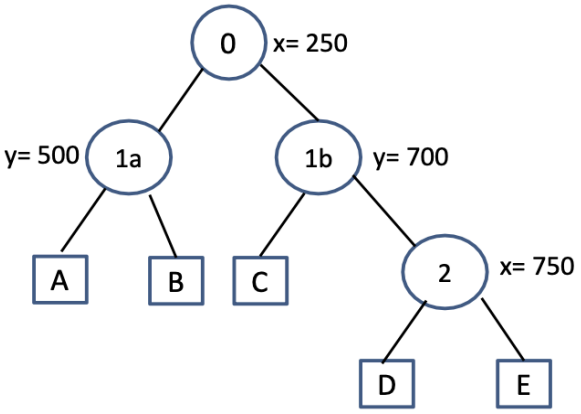
- scena este divizată recursiv prin plane paralele cu planele sistemului de coordonate, într-o ordine fixă, ținând cont și de geometria scenei; de exemplu, în pasul 1 se alege un plan perpendicular pe OX, în pasul 2 un plan perpendicular pe OY, în pasul 3 un plan perpendicular pe OZ, în pasul 4 un plan perpendicular pe OX, etc.

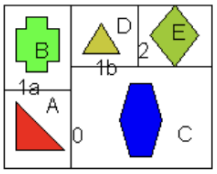
Alegerea planelor

- numărul de obiecte din fiecare semispațiu să fie același sau diferit cu 1, pentru ca arborele să fie echilibrat
- numărul de intersecții dintre planele de partiționare și obiectele scenei să fie minimizat









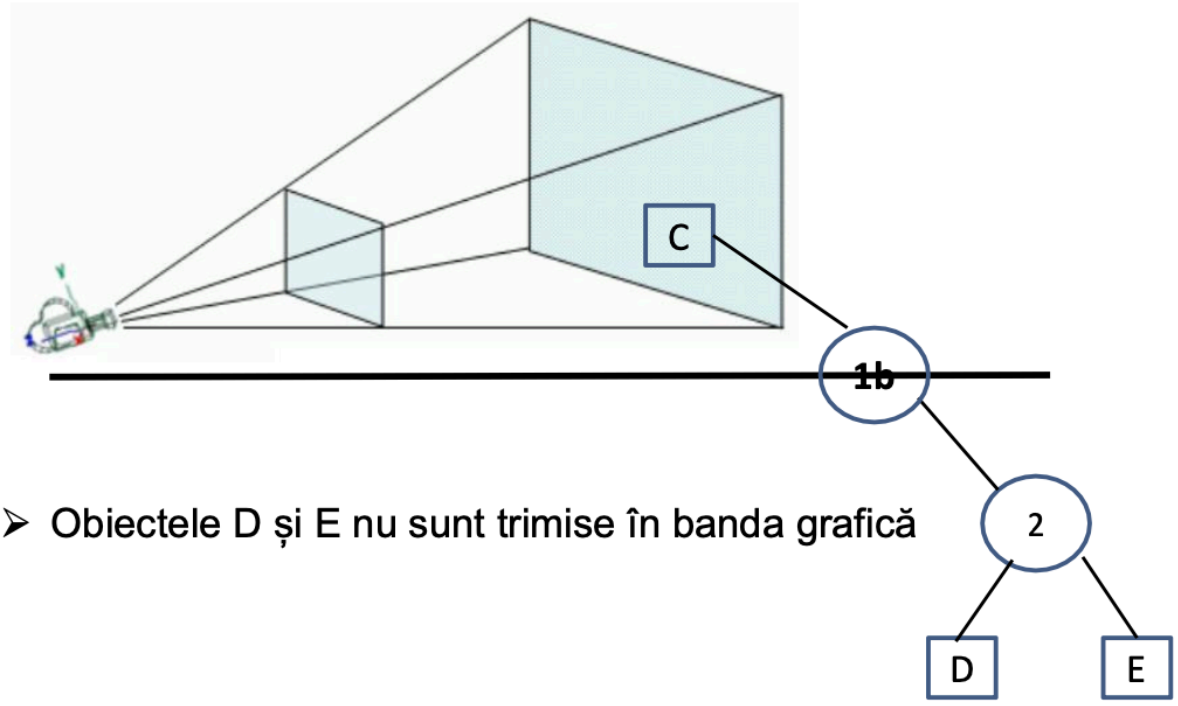
Exemplu de divizare
cu plane aliniate cu axele, în 2D:

- în fiecare pas, planul de divizare este ales astfel încât să se evite intersecția sa cu obiectele scenei

Eliminarea obiectelor nevizibile - la fel ca în cazul arborelui PA - BSP: dacă planul de partiționare al unui nod nu intersectează volumul vizual, întregul grup de obiecte din subarborele aflat de partea opusă volumului este eliminat din banda grafică

Afișarea scenei 3D folosind arborele AABSP

- dacă planul atașat unui nod nu intersectează volumul vizual atunci întregul grup de obiecte din subarborele aflat de partea opusă volumului NU este trimis în banda grafică

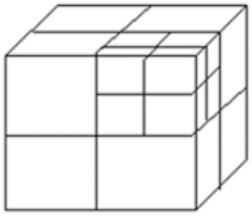


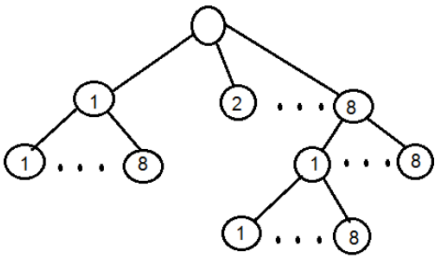
➢ Obiectele D și E nu sunt trimise în banda grafică

Scena - Arbore octal

Construirea arborelui octal al scenei

- se pleacă de la volumul încadrator al scenei, aliniat cu axele, care se asociază nodului rădăcină al arborelui
- se divizează volumul încadrator în 8 sub volume egale
- se împart primitivele între cei 8 octanți





- se continuă cu divizarea recursivă a sub volumelor și asocierea primitivelor cu sub volumele;
  fiecare primitivă este asociată cu cel mai mic octant care o încadrează complet
- un subvolum nu este divizat dacă s-a ajuns la un nivel de divizare prestabilit sau subvolumul conține suficient de puține primitive sau nicio primitivă
- rezultă un arbore în care fiecare nod intern (care nu este frunză) are 8 copii.
- fiecare nod al arborelui octal are atașat un volum încadrator

Eliminarea obiectelor nevizibile folosind arborele octal al scenei - la fel ca în cazul BVH

- se efectuează testând intersecțiile subvolumelor arborelui cu volumul vizual (cub, în sistemul coordonatelor de decupare), începând cu nodul rădăcină
- dacă volumul asociat nodului curent intersectează volumul vizual, se coboară în arbore, testând intersecția dintre fiecare copil al său și volumul vizual
- dacă sub volumul asociat unui nod este în afara volumului vizual, atunci se revine în arbore pe nivelul anterior (se elimină din banda grafică toate obiectele din subarborele nodului)

Eficiența arborelui octal al scenei

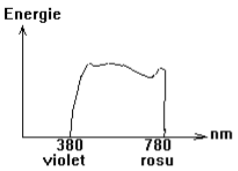
- subdivizarea strictă a spațiului împreună cu metoda de asociere a fiecărei primitive cu cel mai mic octant care o încadrează complet, poate conduce la ineficiență; de exemplu: dacă o primitivă este situată în apropierea centrului volumului încadrator al scenei, primitiva va fi asociata cu acest volum, chiar dacă este foarte mică în raport cu volumul
- o alternativă: o primitivă poate fi asociată cu 2 sau mai mulți octanți vecini care împreună o încadrează complet și formează un volum minimal (crește însă complexitatea construirii arborelui)

CURS 9 - PROPRIETĂȚILE LUMINII - MODELE DE CULOARE

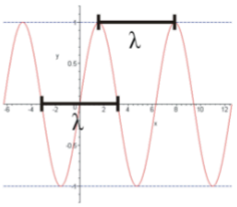
Lumina

Introducere

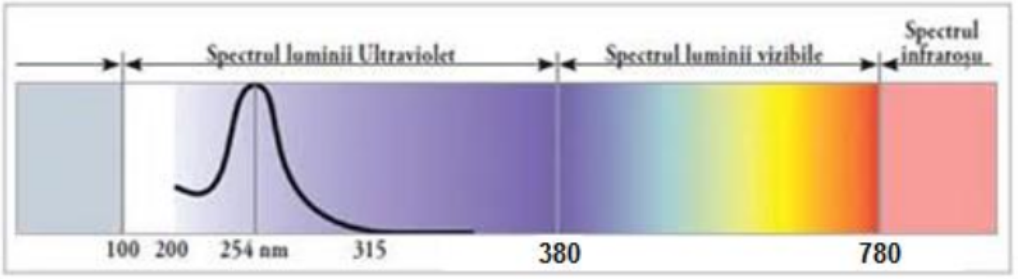
- lumina este energie electromagnetică: spectrul vizibil
- spectrul vizibil este alcătuit din undele cu lungimi cuprinse între 380-780 nanometri



Spectrul vizibil

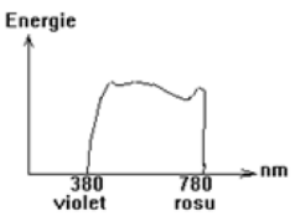


Lungimea de undă: distanța dintre două puncte din spațiu între care unda a parcurs o perioada de repetiție.

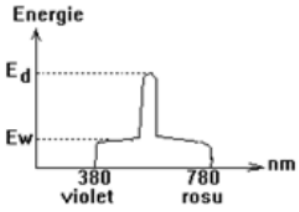


Proprietățile luminii

- atunci când cade pe suprafața unui obiect, lumina poate fi: **absorbită, reflectată sau transmisă**
- sistemul vizual uman percepe
    - lumina provenită direct de la o sursă
    - lumina reflectată sau transmisă de obiectele din mediu.



Lumina acromatica



Lumina cromatica

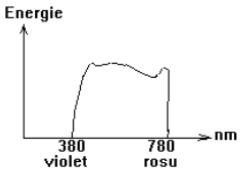
- lumina care conține toate lungimile de undă din spectrul vizibil în proporții aproximativ egale se numește **acromatică**
- lumina în care predomină anumite lungimi de undă se numește **cromatică**

- lumina acromatică provenită de la o sursă este percepută ca albă
- lumina acromatică provenită de la un obiect este percepută ca albă, neagră sau ca o nuanță de gri în funcție de proprietățile fizice ale suprafeței obiectului
- obiectele care reflectă acromatic mai mult de 80% din lumina incidentă albă apar ca albe; cele care reflectă acromatic mai puțin de 3% din lumina incidentă apar ca negre

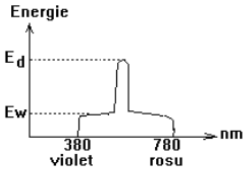
- culoarea unui obiect, percepută de ochiul uman, depinde atât de distribuția lungimilor de undă în lumina care cade pe obiect cât şi de caracteristicile fizice ale suprafeței obiectului: reflectă/absoarbe anumite lungimi de undă
- energia electromagnetică nu are culoare
- **culoarea este rezultatul unui proces psiho-fiziologic**

- **definiția psiho-fiziologică a unei culori cuprinde: nuanța, luminozitatea (strălucirea), saturația**
    - **nuanța**: roşu, galben, verde, etc. - determinată de lungimea de undă dominantă a distribuției spectrale a luminii.
    - **luminozitatea sau strălucirea** - reprezintă intensitatea luminii.
        - luminozitatea: caracteristică a unui obiect emițător de lumină

        - strălucirea: caracterizează un obiect ne-emițător, care reflectă lumina
    - **saturația sau puritatea** - o măsură a amestecului de alb într-o culoare pură
        - permite să se facă distincție între roșu şi roz, între albastru și bleu, etc
        - o culoare pură are saturația 100%
        - lumina acromatică are saturația 0%



Lumina acromatica



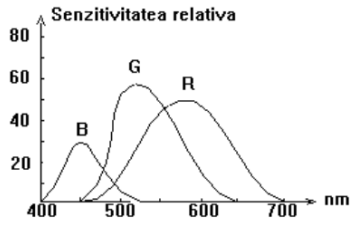
Lumina cromatica

- **Culoarea** este data de **lungimea de undă dominantă**
- Ew = 0 - culoarea are puritatea 100% : monocromatică (lărgime de banda de un nanometru)

- Cea mai cunoscută dintre teoriile privind formarea culorilor în sistemul ochi-creier este aceea conform căreia **în retina ochiului uman există trei tipuri de conuri, fiecare tip fiind sensibil la una dintre culorile roşu, verde şi albastru**: **teoria tricromaticității (Tristimulus theory)**

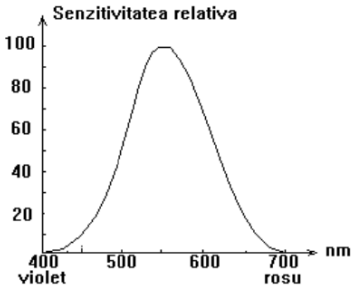
Teoria tricromaticității

- culorile **roșu**, **verde** și **albastru** se numesc **culori primare**
- conform teoriei tricromaticității, **orice culoare care poate fi percepută de ochiul uman se obține printr-un amestec aditiv al celor 3 culori primare**
- prin amestecul lor în proporții egale se obține alb



Raspunsul ochiului la cele 3 culori primare

Senzitivitatea maximă corespunde luminii cu lungimea de undă în jur de 550 nm, percepută ca galben-verde.



Curba de luminozitate

Suma celor 3 curbe de senzitivitate

- două culori care **prin amestec produc lumină albă** se numesc **complementare** => culorile complementare culorilor primare sunt: cyan (Cyan), magenta (Magenta), galben (Yellow)
- amestecând două culori primare în proporții egale se obține culoarea complementară celei de-a treia:

  R G B

  C M Y

  de exemplu, albastru+verde=cian, roşu+verde=galben, roşu+albastru= magenta

- culorile roșu, verde și albastru se numesc și **"primitive aditive"** deoarece ele permit formarea de culori prin adunarea lor în diferite proporții; acest mod de definire a culorilor corespunde echipamentelor emițătoare de lumină (dispozitive de vizualizare cu ecran)

- culorile cyan, magenta, galben se numesc **"primitive extractive"**
  => se obțin prin extragerea culorilor primare din lumina albă:
  cian = alb-roșu, magenta = alb-verde,
  galben = alb-albastru

  R G B

  C M Y

  => se folosesc pentru a defini culorile reflectate de documentele imprimate: pigmenții existenți în cernelurile tipografice absorb din lumina incidentă componentele complementare acelora ale pigmenților

  exemplu: pigmentul de culoare magenta absoarbe din lumina incidentă lumina verde, iar cel de culoare galben, lumina albastra.
  - o suprafață care conține pigmenți magenta și galben va reflecta (sau transmite) lumină roșie
- C, M, Y permit specificarea de nuanțe prin extragerea culorilor primare în diferite proporții din alb; scăzându-le în proporții egale din alb se obțin diferite nuanțe de gri

Modele de culoare folosite în sinteza imaginilor

Orientate către echipamente

- se bazează pe culorile primare folosite de echipamente pentru redarea culorilor: RGB, CMY
- YIQ: pentru transmisia imaginilor TV color

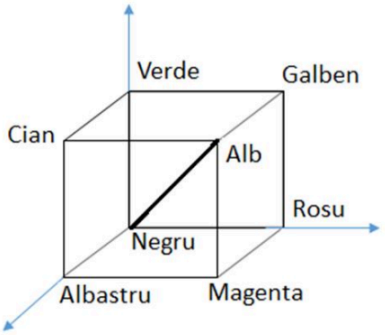
Orientate către utilizator

- se bazează pe proprietățile psiho-fiziologice ale culorilor : HSV și HSL

Definiție

- un model de culoare specifică un sistem de coordonate 3D și un subspațiu al culorilor în sistemul de coordonate respectiv
- fiecare culoare se reprezintă printr-un punct în subspațiul culorilor

Modelul RGB (Red, Green, Blue)



Roșu: 1,0,0

Verde: 0,1,0

Albastru: 0,0,1

Alb: 1,1,1; Negru: 0,0,0

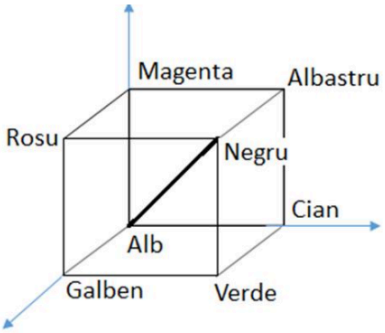
Cian: 0, 1, 1 - culoarea complementară culorii Roșu !

Pe diagonala Negru-Alb: nuanțe de gri – obținute prin amestec în proporții egale între R,G,B

Modelul CMY (Cyan, Magenta, Yellow)

CMYK (Cyan, Magenta, Yellow, Key) → key = negru





Cian: 1,0,0

Magenta: 0, 1, 0

Galben: 0,0,1

Alb: 0,0,0

Negru: 1,1,1 - (R,G,B) sunt extrase în proporție maximă din lumina alba

Roșu: 0, 1, 1 - Complementara culorii Cian!

Unele dispozitive de imprimare includ culoarea negru între culorile C,M,Y: CMYK

Conversia RGB ←→ CMY

[C M Y] = [1 1 1] - [R G B]
[R G B] = [1 1 1] - [C M Y]

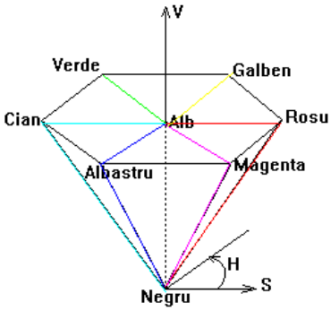
Culorile reflectate (RGB):
Cian: 1,1,1 – 1,0,0 = 0,1,1
Magenta: 1,1,1 – 0,1,0 = 1,0,1
Negru: 1,1,1 – 1,1,1 = 0,0,0

Modelul HSV (Hue, Saturation, Value)

- Model de culoare orientat către utilizator



- Artiştii specifică culorile prin tente, nuanţe şi tonuri



0<= V<= 1, 0<= S <= 1, 0<= H <= 360

V=1 - baza hexaconului

S=0 - pe axa hexaconului

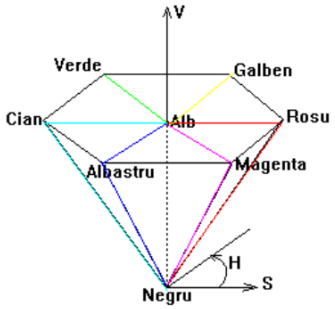
S=1 - pe frontiera bazei hexaconului: culorile pure

Roşu: (0, 1, 1)

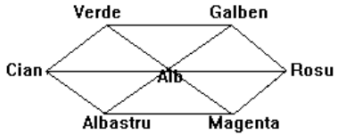
Culorile complementare – la 180 grade pe baza hexaconului

Cian: (180, 1, 1)

- Hexaconul din figură reprezintă subspaţiul culorilor în modelul HSV



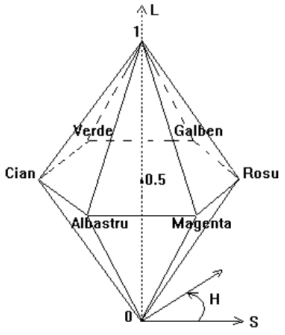
- Saturaţia într-un punct este definită prin distanta de la axa hexaconului la punctul respectiv.
- Combinaţia (S=0,V=1) corespunde culorii alb iar prin (S=0, 0<=V<=1) se reprezintă nivelurile de gri.
- Pentru S=0 valoarea lui H este nesemnificativă.
- Culorile pure sunt reprezentate prin V=1 şi S=1.



- Baza hexaconului corespunde suprafeţei văzute atunci când se priveşte cubul RGB dinspre vârful (1,1,1) de-a lungul diagonalei principale.

- Fiecare plan de V constant din spaţiul HSV corespunde vederii unui sub cub din cubul RGB.
- Diagonala principală a cubului RGB devine axa principală în spaţiul HSV.
- Această interpretare sta la baza algoritmilor de conversie între modelele RGB şi HSV.

Modelul HSL (Hue, Saturation, Lightness)



Modelul HLS

Culorile primare cu saturație maximă şi complementarele lor sunt reprezentate prin S=1, L=0.5.

Conversii RGB → HSV/ HSL

max = sup(R, G, B) min = inf(R, G, B)

V = max
S= (max-min)/max daca max ≠ 0
  0 daca max =0

H = 0 daca (max – min) = 0
  (G – B) / (max – min) daca R = max
  (B – R) / (max – min) + 2 daca G = max
  (R – G) / (max – min) + 4 daca B = max
H = H x 60
daca H<0 H = H + 360

RGB → HSV

L = (max-min)/2
S = (max – min)/(max + min) daca L<=0.5
  = (max – min)/ (2 – max – min) daca L > 0.5

H = 0 daca (max – min) = 0
  (G – B) / (max – min) daca R = max
  (B – R) / (max – min) + 2 daca G = max
  (R – G) / (max – min) + 4 daca B = max
H = H x 60
daca H<0 H = H + 360

RGB → HSL

Interpolarea în spațiul culorilor

- **Rezultatul interpolării între două culori depinde de modelul de culoare în care sunt specificate**
- Fie două culori specificate în modelul RGB, C1=(1,0,0) şi C2=(0,1,0) - le interpolăm cu ponderi egale în modelele RGB și HSV
  - Interpolare în modelul **RGB**:
    C=t*C2 + (1-t)*C1 unde t=0.5, se obține
    C=(0.5, 0.5, 0), care convertită în HSV ne dă **(60, 1, 0.5)**
  - Interpolare în modelul **HSV**:
    C1 se reprezintă în HSV prin (0, 1, 1) iar C2 prin (120, 1, 1)
    C= 0.5*(0,1,1) + 0.5*(120,1,1) = **(60, 1, 1)**

CURS 10 - MODELE DE REFLEXIE

Modele empirice pentru calculul reflexiei luminii

- caracteristicile **luminii reflectate într-un punct al suprafeței unui obiect** depind de
  - **lungimile de undă** conținute în lumina incidentă
  - **proprietățile materialului** din care este construită suprafața
  - **direcția** și **geometria** sursei luminii incidente
  - **orientarea suprafeței** în punctul respectiv

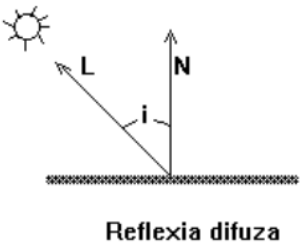
- expresia care modelează **intensitatea luminii reflectate într-un punct al unei suprafețe** este definită pentru **o suprafață necolorată** și **o lumină incidentă monocromatică**, deci caracterizată printr-o anumită lungime de undă, **λ**

- în cazul general, lumina reflectată nu este monocromatică, de aceea pentru calculul său expresia ar trebui să fie evaluată continuu pe întregul domeniu al spectrului de modelat
- **în practică, expresia se evaluează separat pentru cele trei culori primare, R, G, B**

Modelarea reflexiei difuze a luminii provenite de la o sursă

Aproximarea reflexiei difuze într-un punct al unei suprafețe 3D

- Lumina reflectată difuz de o suprafață este dispersată regulat în toate direcțiile
- **Legea lui Lambert** definește reflexia luminii provenite de la o sursă punctiformă, de către un difuzor perfect



Reflexia difuza

Id = Isursa * kd * cos(i) 0<= i <= π/2
N - normala la suprafață în punctul considerat
Isursa - intensitatea luminii incidente (provenită de la sursa de lumină)
Kd - coeficientul de difuzie a luminii incidente, dependent de materialul suprafeței: suprafața reflectă/absoarbe/transmite lumina în anumite proporții: 0 <= kd <= 1
Id - intensitatea luminii reflectate difuz de suprafață

- Dacă **i este mai mare ca π/2**, suprafața nu primește lumină de la sursă (sursa de lumină se află în spatele suprafeței).

Modelarea reflexiei luminii ambientale

Lumina ambientală

- **modelează lumina provenită de la celelalte obiecte ale scenei 3D**: este o sursă de lumină distribuită uniform în spațiu
- lumina ambientală este **reflectată la fel în orice punct al unei suprafețe**; **intensitatea sa** depinde de **intensitatea luminii ambientale și de proprietățile suprafeței**

reflexia luminii ambientale este modelată prin:

Ia*ka

Ia - intensitatea luminii ambientale

0 <= **ka** <= 1 coeficientul de reflexie a luminii ambientale, dependent de materialul suprafeței

atunci:

Id - intensitatea luminii reflectate difuz într-un punct al unei suprafețe

Id = reflexia luminii ambientale + reflexia difuză a luminii de la sursa = Ia * ka + Isursa * kd* cos(i)

0 <= i <= π/2

Modelarea atenuării luminii de la o sursă

Sursă direcțională = sursa aflată la distanță foarte mare (infinit) de scena 3D

- vectorul L este același în orice punct al suprafeței
- pentru 2 suprafețe paralele, cu aceleași proprietăți de material, rezultă aceeași Id → dacă proiecțiile suprafețelor se suprapun în imagine, ele nu se vor distinge

Sursă pozițională = sursa poziționată la o distanță specifică față de scena 3D

Intensitatea luminii descrește proporțional cu inversul pătratului distanței de la sursa de lumină la suprafață

Id = Ia* ka + fat * Isursa* kd* cos(i), unde fat = $1/d^2$ este funcția de atenuare a luminii de la o sursă

d este distanța de la sursă la punctul de pe suprafață considerat.

- corecția nu satisface cazurile în care sursa este foarte îndepărtată: atenuare prea mare
- dacă sursa este la distanță foarte mică de scenă, atenuările obținute pentru două suprafețe aflate la distanțe puțin diferite de sursă, vor fi mult diferite
- în practică se folosește **factorul de atenuare**

> **fat = min(1/(c1 + c2*d + c3*d2), 1)**
> **c1, c2 şi c3 sunt trei constante care se asociază sursei de lumină**

- constanta c1 se alege astfel încât numitorul să nu devină prea mic atunci când sursa este foarte apropiată; **valoarea funcției este limitată la 1 pentru a se asigura atenuarea**

Culoarea luminii reflectate difuz

- lumina incidentă poate conține mai multe lungimi de undă → reflectate în mod diferit de o suprafață
  Csursa = [IsursaR, IsursaG, IsursaB] – culoarea luminii sursei
  Ca = [IaR, IaG, IaB] – culoarea luminii ambientale

- o suprafață reflectă **în mod diferit componentele R, G, B ale luminii incidente**
  exemplu: o suprafață care absoarbe din lumina incidentă componentele de verde (G) și albastru (B),
  - Dacă **lumina incidentă** este **albă**, **suprafața** va fi văzută ca **roșie**:
    [1,1,1] – [0,1,1] = [1,0,0]
  - Dacă **lumina incidentă** este **roșie**, **suprafața** va fi văzută tot ca **roșie**:
    [1,0,0] – [0,1,1] = [1,0,0] (componentele G și B au valoarea zero – nu pot fi extrase)
  - Dacă **lumina incidentă** este **verde**, **suprafața** va fi văzută ca **neagră** (componenta verde a fost absorbită):
    [0,1,0] – [0,1,1] = [0,0,0]
- **rezultă: coeficienții de difuzie a luminii de către o suprafață depind de componentele luminii incidente și de proprietățile suprafeței**

- ka = [kaR, kaG, kaB] – coeficienții de difuzie a componentelor R, G, B din lumina ambientală
- kd = [kdR, kdG, kdB] – coeficienții de difuzie a componentelor R, G, B din lumina provenită de la sursă

➢ Expresia luminii reflectate difuz se evaluează separat pentru cele 3 componente ale luminii incidente, R,G,B:

Idλ = Iaλ*kaλ + fat*Isursaλ *kdλ *cos(i), unde λ: lungimea de undă

sau:

❖ Cd = [IdR, IdG, IdB] - culoarea luminii reflectate difuz

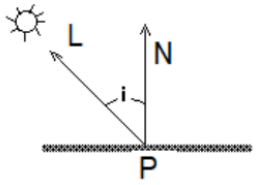
Cd = [IaR *kaR + fat*IsursaR *kdR *cos(i),

IaG *kaG + fat*IsursaG *kdG *cos(i),

IaB *kaB + fat*IsursaB *kdB *cos(i)]

❖ ka (kaR, kaG, kaB) și kd (kdR, kdG, kdB) sunt caracteristici ale materialului suprafeței, numite: culoarea ambientală și culoarea difuză a materialului.

Modelarea reflexiei difuze



Reflexia difuza in punctul P

- Știind că

cos(i)=L·N / (|L|·|N|) = Lu·Nu

unde: N este normala la suprafață în P

L este vectorul din P către sursa de lumină

(L·N) – produsul scalar al vectorilor L, N

- Rezultă ecuația care modelează lumina reflectată difuz:

❖ Idλ = Iaλ*kaλ + fat*Isursaλ *kdλ* (Lu · Nu)

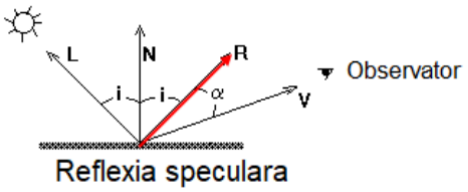
Cd = [IdR, IdG, IdB] – culoarea luminii reflectate difuz

Pentru a include și cazul în care i> π/2 (lumina de la sursă nu ajunge în punctul considerat):

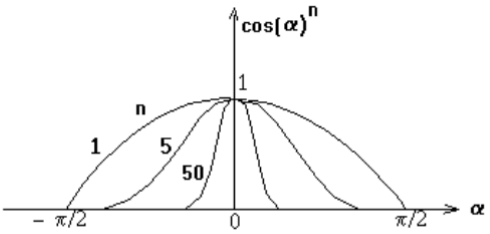
Iλ = Iaλ*kaλ + fat*Isursaλ *kdλ* max((Lu · Nu), 0)

Modelarea reflexiei speculare

- un **reflector perfect**, de exemplu o oglindă, **reflectă lumina numai într-o singură direcție**, R, care este simetrică cu L față de normala la suprafață → **numai un observator situat exact pe direcția respectivă va percepe raza reflectată**



Reflexia speculara



Functia de distributie spatiala a luminii reflectate specular

- pentru **materialele imperfect reflectante** cantitatea de lumină care ajunge la observator depinde de **distribuția spațială a luminii reflectate specular**:
  - la suprafețele netede (exemplu: metale) distribuția este dreaptă şi focalizată
  - la suprafețele cu rugozități (exemplu: hârtia) ea este dispersată
  - **se aproximează prin $cos(\alpha)^n$ (modelul Bui-Tuong Phong) unde n este exponentul de strălucire (shininess) al materialului**

Modelul Phong pentru aproximarea reflexiei speculare într-un punct al unei suprafețe 3D

$$I_{s\lambda} = I_{sursa\lambda} * w(i,\lambda) * cos(\alpha)^n$$

w(i, λ) -funcția de reflectanță
i - unghiul de incidență
λ - lungimea de undă a luminii incidente

- În practică, w(i, λ) este înlocuit cu o constantă determinată experimental, numită **coeficientul de reflexie specular al materialului**, notat **$k_{s\lambda}$**

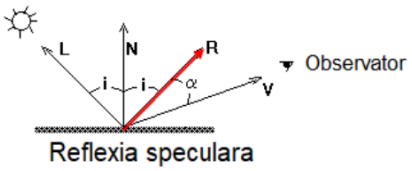
❖ Modelul practic al reflexiei speculare:

$I_{s\lambda} = I_{sursa\lambda} * k_{s\lambda} * \cos(\alpha)^n$

$\cos(\alpha) = R \cdot V / (|R| \cdot |V|) = Ru \cdot Vu$

Rezultă:

$I_{s\lambda} = I_{sursa\lambda} * fat * k_{s\lambda} * (Ru \cdot Vu)^n$



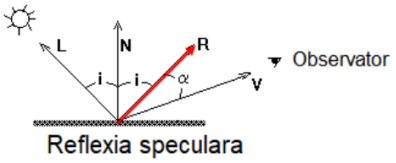
Pentru a include și cazul în care α >= 90:

$I_{s\lambda} = I_{sursa\lambda} * fat * k_{s\lambda} * \max((Ru \cdot Vu)^n, 0)$

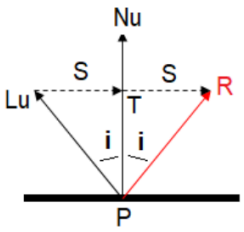
Reflexia specular nu poate avea loc dacă în punctul considerat nu se primește lumină de la sursă: $I_{s\lambda} = lum * I_{sursa\lambda} * fat * k_{s\lambda} * \max((Ru \cdot Vu)^n, 0)$

$lum = 1$ daca $(Lu \cdot Nu) > 0$ $(0 <= i < \pi/2)$

$= 0$ altfel



Calculul direcției reflexiei speculare



• Vectorul R este simetricul vectorului L față de N.

• Proiecția scalară a vectorului Lu pe direcția normalei este mărimea segmentului PT: $|Lu|\cos(i) = \cos(i) = Lu \bullet Nu$

• Proiecția vectorială a lui Lu pe direcția normalei este **vectorul PT**, care are lungimea segmentului PT și direcția normalei:

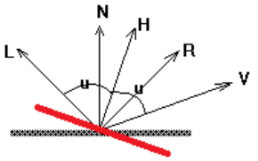
$(Lu \bullet Nu) * Nu / |Nu| = (Lu \bullet Nu) * Nu$

$R = Lu + 2 * S$, $PT = Lu + S \rightarrow S = PT - Lu$

$R = Lu + 2(PT - Lu) = Lu + 2 * (Lu \bullet Nu) * Nu - 2 * Lu = 2 * (Lu \bullet Nu) * Nu - Lu$

$R = 2 * (Lu \bullet Nu) * Nu - Lu$ **sau în glsl: R = *reflect*(-Lu, Nu)** 😎

- o altă formulare a modelului Phong pentru reflexia specular folosește vectorul median, notat cu H în figură; el face unghiuri egale cu L și cu V
- dacă suprafața ar fi orientată astfel încât **normala să aibă direcția lui H**, atunci **observatorul ar percepe lumina specular maximă** (deoarece ar fi pe direcția razei reflectate specular).



❖ Termenul care exprimă reflexia specular este în acest caz:

$I_{s\lambda} = I_{sursa\lambda} * fat * k_{s\lambda} * (Nu \bullet Hu)^n$, unde $Hu = (Lu + Vu) / |(Lu + Vu)|$

- atunci când sursa de lumină şi observatorul sunt la infinit, utilizarea termenului Nu · Hu este avantajoasă deoarece Hu este constant (același în toate punctele unei suprafețe)

Modelul de iluminare locală - lumina reflectată într-un punct al unei suprafețe - Modelul de reflexie Phong

- combină culorile datorate:
  - reflexiei luminii ambientale - **culoarea_ambientală**
  - reflexiei difuze a luminii de la o sursă / surse - **culoarea_difuză**
  - reflexiei speculare a luminii de la o sursă / surse - **culoarea_specular**ă

- **reflexia luminii într-un punct al unei suprafețe determină culoarea sa în acel punct**

$$I\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda\,[kd\lambda * \max((Lu \bullet Nu),0) + lum * ks\lambda * \max((Hu \bullet Nu)^{n},0)]$$

$lum = 1$ daca $(Lu \bullet Nu) > 0$ $\quad (0 <= i < \pi/2)$

$= 0$ altfel

- dacă scena 3D este iluminată de m surse de lumină

$$I\lambda = Ia\lambda * ka\lambda + \sum_{1<=i<=m} fat_i * Isursa\lambda_i * [kd\lambda * \max((Lu_i \bullet Nu),0) + lum_i * ks\lambda * \max((Hu_i \bullet Nu)^{n},0)]$$

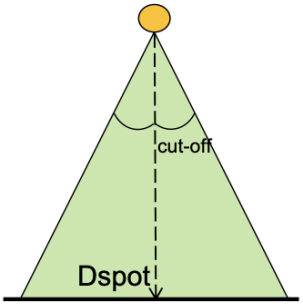
- la cele 3 componente se poate adăuga culoarea_emisă de suprafață

Culoarea într-un punct al unei suprafețe
= culoarea_emisa + culoarea_ambientala + culoarea_difuza + culoarea_speculara

- culoarea emisă
  - lumina emisă de o suprafață
  - independentă de sursele de lumină
  - aceeași în orice punct al suprafeței
  - nu luminează obiectele din jur
  - **culoarea_emisă = Ke, o constantă**

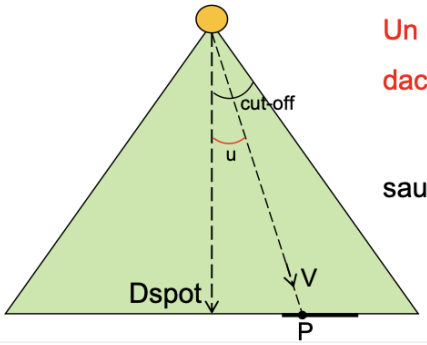
Efectul de spot

- În modelul de iluminare locală, sursa de lumină este considerată punctiformă. Lumina produsă de o sursa punctiformă este omnidirecțională.
- Efectul de spot simulează o sursă de lumină care produce un con de lumină.
- Numai suprafețele aflate în conul de lumină primesc lumina de la sursă.



Conul de împrăștiere a luminii se calculează folosind:

- Poziția spotului (sursei de lumina)
- Direcția spotului
- Împrăștierea luminii spotului (θspot sau cut-off)



Un punct P al unei suprafețe primește lumina de la spot daca u < cut-off:

cos(u) > cos(cut-off)

sau

Vu • Du > cos(cut-off)

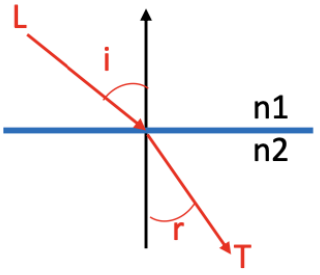
CURS 11 - TRANSPARENȚA ÎN MODELELE DE ILUMINARE LOCALĂ

Modelarea transparenței

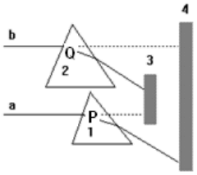
- unele obiecte ale scenei sintetizate pot fi construite din materiale transparente sau translucide
- **transmisia luminii prin obiectele transparente este specular, în timp ce prin cele translucide este difuză**
- atunci când lumina trece dintr-un mediu într-altul (de exemplu, din aer în apă) direcția sa se modifică datorită refracției
- Relația dintre unghiul razei incidente, i, și cel al razei refractate, r, este dată de **legea lui Snell**

sin(r)/sin(i) = n1/n2

n1 și n2 sunt indicii de refracție ai celor două medii (materiale) traversate de lumină.



- indicele de refracție al unui material este dependent de lungimea de undă a luminii incidente și chiar de temperatură, dar în modelele de sinteza a imaginilor el este considerat constant
- **culoarea vizibilă într-un punct al unei suprafețe transparente provine de la obiectul aflat pe direcția razei transmise**



3 și 4 sunt poligoane opace

1 si 2 sunt poligoane transparente, aflate în fața poligoanelor 3 și 4

a și b sunt raze de lumină provenind de la surse, incidente în P și Q

Ținând cont de refracție:

- Culoarea în P este dată de poligonul 4
- Culoarea în Q este dată de poligonul 3

Neglijând refracția (T are direcția razei incidente)

- Culoarea în P este dată de poligonul 3
- Culoarea în Q este dată de poligonul 4

- refracția produce, de asemenea, o distorsionare a obiectelor, asemănătoare cu aceea produsă de o proiecție perspectivă → **dacă se dorește obținerea de imagini realiste, trebuie să se țină cont de refracție**
- multe metode practice de modelare a transparenței **ignoră refracția**, astfel încât **obiectele vizibile printr-o suprafață transparentă sunt cele aflate pe direcția razei incidente**

  Motivul ignorării:
  - reducerea volumului de calcule
  - obținerea realismului fotografic în totalitate (fără deformare)

- atunci când suprafața vizibilă într-un pixel este **transparentă**, **culoarea** în care va afișat pixelul **se poate obține combinând culoarea suprafeței vizibile cu aceea a suprafeței aflată imediat în spatele său** (sau a fondului), folosind următoarea formulă de interpolare

$I_\lambda = (1-kt_1)*I_{\lambda 1} + kt_1*I_{\lambda 2}$

$I_{\lambda 1}$ este culoarea suprafeței vizibile în pixel

$I_{\lambda 2}$ este culoarea suprafeței din spatele celei vizibile, care se proiectează în același pixel

Coeficientul de transmisie, kt_1, este o măsură a transparenței suprafeței vizibile în pixel,

$0 <= kt_1 <= 1$

$kt_1 = 0$ → suprafața vizibilă este opacă si deci pixelul va fi afișat în culoarea sa, $I_{\lambda 1}$

$k_{t1} = 1$ → suprafața vizibilă este perfect transparentă si nu contribuie la culoarea pixelului, pixelul va fi afișat în culoarea suprafeței din spate, $I_{\lambda 2}$

Dacă $k_{t1}=1$ și suprafața din spatele celei vizibile este la rândul său transparentă, metoda de calcul se aplică recursiv, până când se întâlnește o suprafață opacă sau fundalul scenei.

- aproximarea liniară din model nu dă rezultate bune pentru suprafețele curbe
    - în apropierea laturilor siluetei unei suprafețe curbe (de exemplu, o vază sau o sticlă) grosimea materialului reduce transparența
    - soluția propusă de D.S. Kay ("Transparency For Computer Synthesized Images"): kt se calculează în funcție de normala la suprafață în punctul considerat

- **majoritatea algoritmilor de determinare a vizibilității suprafețelor la afișarea scenelor 3D pot fi adaptați pentru a îngloba transparența**
- în algoritmii care afișează poligoanele scenei 3D în ordinea "din spate în față" (back to front), de exemplu algoritmul Pictorului și BSP, $I\lambda_1$ corespunde poligonului care se rasterizează la un moment dat, iar $I\lambda_2$ este valoarea existentă în memoria imagine pentru pixelul considerat

- **adăugarea efectului de transparență în algoritmul Z-Buffer este mai dificilă**, deoarece **poligoanele** sunt **rasterizate în ordinea în care sunt transmise în banda grafică, neținându-se cont de distanța lor față de observator**
    - se poate realiza folosind mai multe memorii buffer-imagine și generând imaginea în mai multe etape, imaginea finală obținându-se prin combinarea imaginilor poligoanelor opace cu cele ale poligoanelor transparente

În OpenGL, transparența poate fi simulată prin amestecul dintre culoarea fragmentului curent și cea a pixelului în care se afișează, folosind modelul (R, G, B, A);

A – opacitatea: 0 – transparent; 1 – opac

glEnable(GL_BLEND) - activează amestecul culorilor

glBlendFunc(..) – specifică modul de amestec al culorilor. Exemplu:

glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA):

noua_culoare_pixel = Culoare_fragment * A_fragment + Culoare_pixel * (1 – A_fragment)

- **operația de amestec** este **o operație raster** efectuată de procesorul de rasterizare după testul de adâncime, înainte de actualizarea valorii pixelului în bufferul imagine (frame buffer)

CURS 12 - INTRODUCEREA CEȚII ÎN IMAGINI

Modelarea ceții

Ceața adaugă realism imaginii și ajută în eliminarea defectelor produse de decuparea la nivelul planului din spate al volumului vizual

- fără ceață, pe măsură ce observatorul se îndepărtează de un obiect, obiectul se apropie de planul din spate și este decupat atunci când planul din spate îl intersectează
- cu ceață, dacă intensitatea ceții crește proporțional cu distanța de la poziția observatorului, **se creează efectul de distanță pentru obiectele îndepărtate**, iar **obiectele decupate la nivelul planului din spate nu mai sunt vizibile**

Cfog – culoarea de ceață

Clocal – culoarea calculată într-un punct folosind modelul de iluminare locala

0<= Φ <= 1 – factorul de ceață, proporțional cu distanța de la observator la punct

Culoarea punctului, afectată de ceață: **Cloc_fog = (1-Φ)*Clocal + Φ*Cfog**

Fie dist distanța de la observator la un punct din scena vizualizată.

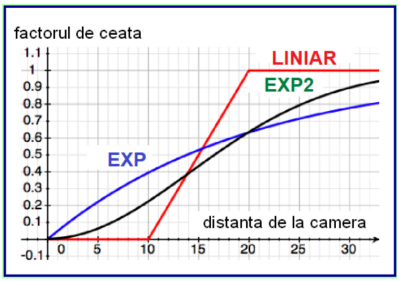
Factorul de ceață liniar

- Acționează între 2 distanțe definite în sistemul coordonatelor observator, z0 si z1, măsurate pe direcția în care privește observatorul.

$\Phi = 0$ pentru dist < z0

$\Phi = (dist - z0) / (z1 - z0)$ pentru z0 <= dist <= z1

$\Phi = 1$ pentru dist > z1



z0 = 10; z1 = 20; fog_density = 0.05

Factorii de ceață exponențiali

$\Phi = 1.0 - e^f$

EXP: $f = -(fog_density*dist)$

EXP2: $f = -(fog_density*dist)^2$

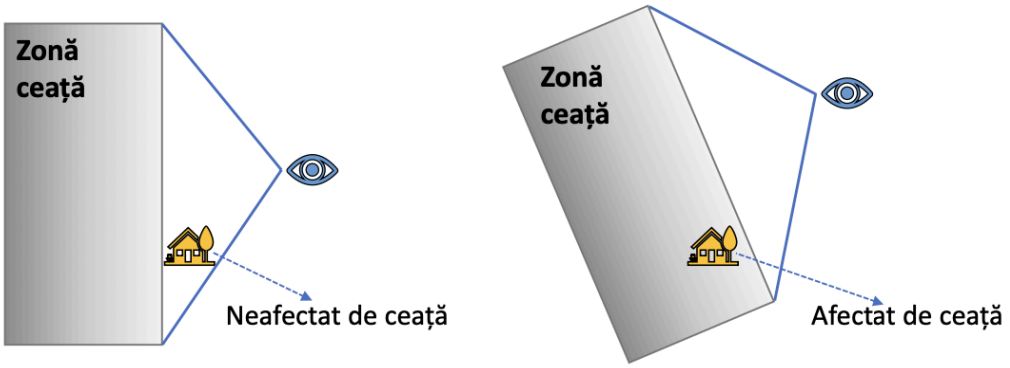
- fog_density: densitatea ceții (un număr real, de ex. 0.05) - modelează atenuarea luminii datorata ceții

Distanța - dist

Distanța față de observator a punctului în care se calculează culoarea

- Punctul este în coordonate observator
  1. **dist** este coordonata z a punctului, la fel ca distanțele z0 și z1

  Dacă se rotește camera, fără a-i modifica poziția, coordonata z a punctului se modifică → un obiect aflat la o distanță d de observator poate să intre/iasă din zona de ceață:



  2. **dist** este lungimea vectorului de la observator (0,0,0) la punct.

- Varianta 1 este mai ieftină computațional și preferată atunci când nu este rotită camera

Introducerea ceții în imagini

Includerea ceții în calculul culorii se poate efectua

- **În vertex shader (modelul Gouraud)**
  - se calculează Φ folosind distanța (dist) de la observator la poziția vârfului transformată în coordonate observator
  - se calculează culoarea vârfului afectat de ceață: Cloc_fog = (1-Φ)*Cvrf + Φ*Cfog
  - Cloc_fog este ieșire din vertex shader
  - culorile afectate de ceață ale vârfurilor sunt interpolate de GPU la nivel de fragmente
  - culoarea de ceață interpolată, Cloc_fog, este intrare pentru fragment shader

- **În fragment shader (modelul Phong)**
  Varianta (1): fragment shader primește poziția fragmentului în coordonate observator
  - se transmite de la vertex shader poziția vârfului în coordonate observator
  - pozițiile vârfurilor în coordonate observator sunt interpolate de GPU la nivel de fragmente
  - în fragment shader se calculează Φ folosind ca distanța (dist) - poziția fragmentului (pe z) în coordonate observator

  Varianta (2): poziția fragmentului în coordonate observator se poate calcula în fragment shader sau mai simplu, se poate primi (interpolata) de la VS

CURS 13 - MODELE DE "SHADING"

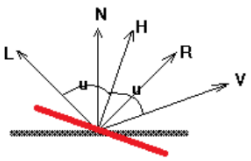
Calculul culorii fragmentelor la redarea suprafețelor 3D - Shading Models

- în vederea vizualizării, suprafețele 3D sunt descompuse în fațete poligonale/triunghiulare conectate; poligoanele procesate în banda grafică pot fi fețele unui corp poliedral (fețe plate) sau pot proveni din descompunerea unei suprafețe oarecare (fețele sunt petice de suprafață curbă)
- la momentul rasterizării nu se cunoaște din ce suprafață face parte un poligon

Modelul LAMBERT - Iluminare constantă sau plată

- Toate fragmentele unui poligon sunt afișate cu aceeași culoare, calculată folosind modelul de iluminare locală (de regulă considerând numai componenta difuză):

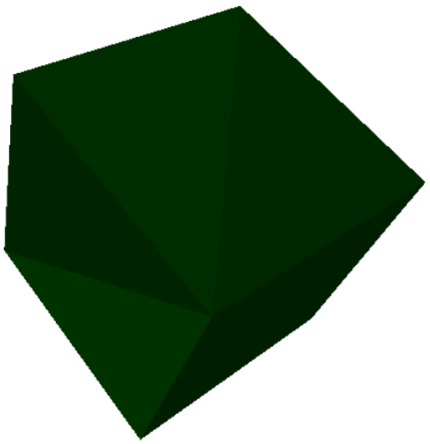
$$Id\lambda = Ia\lambda * ka\lambda + fat * Isursa\lambda * [kd\lambda * \max((Nu \bullet Lu),0) + lum * \max((Nu \bullet Hu)^n,0)]$$



Modelul se bazează pe următoarele presupuneri

1. **poligonul face parte din suprafața de vizualizat** și **nu este o aproximare** a unui petic de suprafață curbă: vectorul Nu este același în orice punct al poligonului
2. **sursa de lumină este la infinit**: produsul scalar (Nu · Lu) este atunci constant pe întreaga suprafață a poligonului
3. **observatorul este la infinit**: (Nu · Hu) este constant pe suprafața poligonului

- dacă ultimele două cerințe nu sunt satisfăcute => se poate adopta o convenție de calcul al vectorilor L și V pentru un întreg poligon(de exemplu, cei doi vectori pot fi calculați în centrul poligonului)
- dacă prima cerință nu este îndeplinită => **aproximarea unei suprafețe curbe prin rețeaua de fațete poligonale este vizibilă deoarece intensitățile calculate pentru fațete adiacente cu orientare diferită sunt diferite**





Efectul de banda Mach

- descoperit de Mach în 1865, este **o iluzie optică datorată filtrării componentei de luminozitate a imaginii formate pe retină**, de către sistemul vizual uman

- efectul **mărește contrastul dintre zone adiacente cu intensitate ușor diferită**, la frontiera dintre zone, **declanșând detecția frontierei în sistemul vizual uman**
- atunci când o suprafață curbă (exemplu: o sferă) este descompusă în fațete poligonale, este de dorit ca laturile fațetelor să nu fie vizibile în imaginea sferei
- în cazul folosirii modelului Lambert pentru calculul culorilor fragmentelor, **datorită folosirii unei singure normale pentru toată suprafața unei fațete**, **culorile fațetelor adiacente** vor fi **ușor diferite**, iar **efectul de bandă Mach mărește percepția schimbării de intensitate pe laturile fațetelor adiacente**

Modelul Gouraud

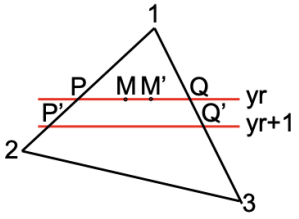
- la momentul **rasterizării** unui poligon, **fiecărui vârf al poligonului îi este asociată o culoare**
- **culorile fragmentelor poligonului** se obțin prin **interpolarea liniară a culorilor din vârfuri**

1. **culoarea într-un vârf** (ieșire din vertex shader: out vec3 color)
    a. se calculează aplicând **modelul de iluminare locală în vârf** (cu N, L, V în coordonate globale)

2. **normala unui vârf**, necesară în modelul de iluminare locală, trebuie să fie transmisă la vertex shader din programul de aplicație (este intrare pentru vertex shader)
    a. poate fi **calculată exact din ecuația analitică a suprafeței pe care se afla vârful**
    b. poate fi **aproximată ca medie aritmetica a normalelor la fețele adiacente în vârf**

3. la **rasterizarea unui poligon**, procesorul de rasterizare calculează culoarea fiecărui fragment astfel:
    a. prin **interpolarea liniară a culorilor vârfurilor**, pentru fragmentele de pe laturi
    b. prin **interpolare liniară între culorile din capetele segmentului interior** pe care se află fragmentul
    Culoarea fragmentului este intrare pentru programul fragment shader (in vec3 color)

Interpolarea liniară a culorilor



$y = y1 + t(y2 - y1) \rightarrow tP = (yr - y1)/(y2 - y1)$

$IP = I1 + tP(I2 - I1) = I1 + (I2 - I1)(yr-y1) / (y2-y1) = I1 + (yr-y1) * \mathbf{C_{1-2}}$

$\mathbf{C_{1-2} = (I2 - I1) / (y2 - y1)}$ **- constanta pentru latura 1-2**

$IQ = I1 + (I3 - I1)(yr-y1) / (y3-y1) = I1 + (yr - y1) * \mathbf{C_{1-3}}$

$\mathbf{C_{1-3} = (I3 - I1) / (y3 - y1)}$ **- constanta pentru latura 1-3**

$x = xP + t(xQ - xP) \rightarrow tM = (xM - xP)/(xQ - xP)$

$IM = IP + tM(IQ - IP) = IP + (IQ - IP)(xM - xP) / (xQ - xP) = IP + (xM-xP)* \mathbf{C_{P-Q}}$

$\mathbf{C_{P-Q} = (IQ - IP) / (xQ - xP)}$ **- constanta pentru segmentul P-Q**

Calculul incremental al culorilor

$IP' = I1 + (I2 - I1)(yr+1 - y1) / (y2 - y1) = I1 + (yr+1 - y1) * \mathbf{C_{1-2}} = IP + \mathbf{C_{1-2}}$

$\mathbf{IP' = IP + C_{1-2}}$

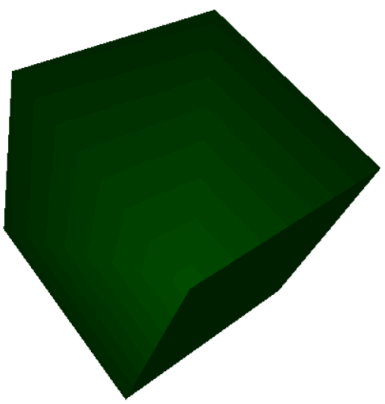
analog, $\mathbf{IQ' = IQ + C_{1-3}}$

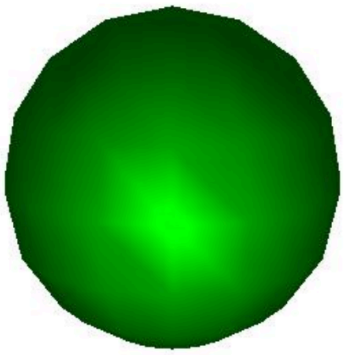
M' (xM + 1, yr)

$IM' = IP + (xM + 1 - xP) * \mathbf{C_{P-Q}} \rightarrow \mathbf{IM' = IM + C_{P-Q}}$

Aprecieri asupra modelului Gouraud

- **este integrat în algoritmul de rasterizare a poligoanelor**
- calcule simple pentru culoarea unui fragment (calcul incremental) → rapid
- nu permite calculul luminii reflectate specular pentru fragmentele interioare unui poligon - **intensitățile fragmentelor interioare nu pot fi mai mari decât cele din vârfuri**
- **reduce efectul de banda Mach, dar nu îl elimină**
  - culorile pe latura de adiacență a 2 poligoane sunt aceleași
  - nu ține cont de tangentele la fețele adiacente pe o latură (culorile de pe laturi se calculează folosind numai culorile din vârfuri)
  - efectul de banda Mach poate fi observat în vecinătatea siluetei unei suprafețe 3D și a zonelor de curbura mare





Modelul Phong

- la momentul **rasterizării unui poligon**, **fiecărui vârf îi este asociată normala vârfului**
    - normala vârfului este transmisă din programul de aplicație, ca și pentru modelul Gouraud
    - normala și poziția vârfului, în coordonate globale, sunt ieșiri din vertex shader
    - **procesorul de rasterizare interpolează liniar pozițiile și normalele vârfurilor, transmițând la fragment shader poziția și normala fragmentului**

- **în fragment shader se calculează culoarea fragmentului aplicând modelul de iluminare locală în poziția fragmentului**

- la rasterizarea unui poligon se calculează **o normală pentru fiecare fragment** astfel
    - prin **interpolarea liniara a normalelor vârfurilor, pentru fragmentele de pe laturi** (la fel ca în modelul Gouraud, pentru culori)
    - prin **interpolare liniară între normalele capetelor segmentului interior** pe care se află fragmentul (la fel ca în modelul Gouraud, pentru culori)

Dezavantaje față de modelul Gouraud

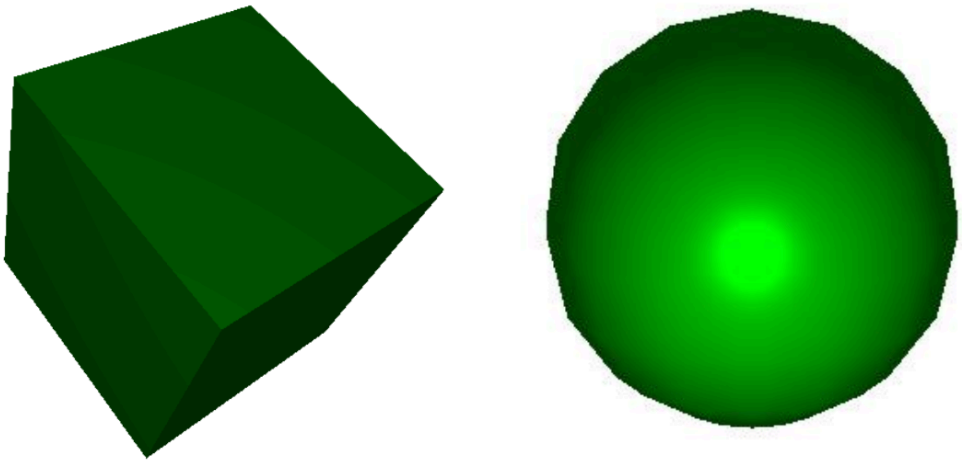
- componentele Nx, Ny, Nz ale normalei unui fragment se obțin (în algoritmul de rasterizare) printr-un calcul incremental analog cu cel folosit pentru calculul culorilor în modelul Gouraud dar, pentru folosirea lor în calculul culorii, normala trebuie să fie normalizată în fragment shader

Nu = N/|N|, unde |N| = $\sqrt{Nx^2 + Ny^2 + Nz^2}$ **(se folosește funcția GLSL *normalize()*)**

- vectorii L (către sursa de lumină) și V (către observator), folosiți în modelul de iluminare locală la nivel de fragment, sunt calculați și normalizați în programul fragment shader pentru a fi folosiți în calculul culorii fragmentului
- **calcule mai complexe la nivel de fragment decât în modelul Gouraud**

Avantaje

- permite redarea reflexiei speculare în orice fragment al unui poligon
- reduce mult efectul de bandă Mach



CURS 14 - DETALII DE IMPLEMENTARE UMBRE

Stencil Buffer

Definitie

- Stencil buffer-ul pune la dispoziție multe opțiuni pentru **a restricționa desenarea pe ecran** și se poate folosi în multe locuri acolo unde **buffer-ul de adâncime nu poate fi utilizat**; privit în modul cel mai simplist, **stencil buffer-ul este folosit pentru a bloca la desenare anumite zone din ecran**

Activare: glEnable(GL_STENCIL_TEST)

Exista 4 funcții stencil în OpenGL :

- void glClearStencil(GLint s)
- void glStencilFunc(GLenum func, GLint ref, GLuint mask)
- void glStencilMask(GLuint mask)
- void glStencilOp(GLenum fail, GLenum zfail, GLenum zpass)

glClearStencil() funcționează în mod similar cu glClearColor și glClearDepth

De asemenea inițializează stencil buffer-ul cu valoarea inițială dată ca parametru când este apelat glClear(GL_STENCIL_BIT). În mod default această valoare este 0.

În mod default doar prin activarea folosirii stencil buffer-ului nu se întâmplă nimic toate desenările pe ecran fiind permise fără nici o restricție. Pentru a lucra cu stencil buffer-ul trebuie să "marcăm" valori în acesta folosind glStencilFunc() și glStencilOp()

Funcția glStencilFunc(GLenum func, GLint ref, GLuint mask) definește :

- o funcție de comparație
- o valoare de referință cu care se face comparația
- o mască (se face ȘI logic cu valoarea de referință && valoarea din stencil buffer)

Funcțiile de comparare valide pentru stencil test sunt :

- GL_NEVER – eșuează întotdeauna (nu se va desena nimic pe ecran)
- GL_ALWAYS – reușește întotdeauna (desenarea are loc mereu - aceasta este comportarea default)
- GL_EQUAL – reușește dacă valoarea de referință este egală cu valoarea din stencil buffer
- GL_NOTEQUAL – reușește dacă valoarea de referință nu este egală cu valoarea din stencil buffer
- GL_LESS – reușește dacă valoarea de referință este mai mică decât valoarea din stencil buffer
- GL_LEQUAL – reușește dacă valoarea de referință este mai mică sau egală cu valoarea din stencil buffer
- GL_GREATER – reușește dacă valoarea de referință este mai mare decât valoarea din stencil buffer
- GL_GEQUAL – reușește dacă valoarea de referință este mai mare sau egală cu valoarea din stencil buffer

Legate de funcțiile stencil sunt operațiile stencil definite cu
glStencilOp(GLenum fail, GLenum zfail, GLenum zpass) care setează comportarea valorii stencil bufferului pentru cei trei parametri ai funcției :

- fail : testul stencil a eșuat
- zfail : testul stencil a reușit dar testul de adâncime a eșuat
- zpass : testul stencil a reușit și testul de adâncime a reușit

Operațiile valide asupra valorilor din stencil buffer sunt :

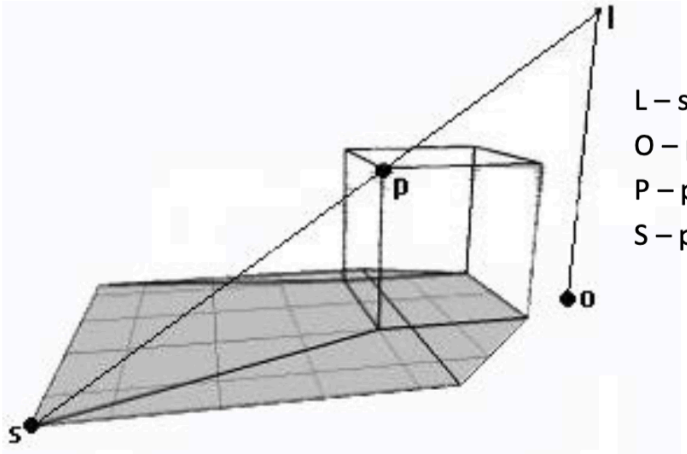
- GL_ZERO - setează valoarea din stencil pentru pixelul/fragmentul curent la 0
- GL_KEEP - păstrează valoarea din stencil pentru pixelul/fragmentul curent
- GL_REPLACE – înlocuiește valoarea din stencil pentru pixelul/fragmentul curent cu valoarea de referință
- GL_INCR – mărește valoarea din stencil pentru pixelul/fragmentul curent
- GL_DECR - scade valoarea din stencil pentru pixelul/fragmentul curent

Exemple de folosire :

- glStencilFunc(GL_ALWAYS, 1, 1); glStencilOp(GL_KEEP, GL_KEEP, GL_REPLACE);
  Se va desena tot timpul pe ecran iar în locul unde a fost desenat ceva stencil buffer-ul va avea valoarea 1
- glStencilFunc(GL_EQUAL, 0, 1); glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
  Se va desena pe ecran numai unde stencil buffer-ul este 0 și valorile din stencil buffer vor fi păstrate

Umbre planare

- o metodă simplă care **permite afișarea de umbre pe suprafețe plane** a fost dezvoltată în 1988 de James Blinn
- ideea care stă în spatele metodei propuse de Blinn este aceea de **a găsi o transformare care proiectează geometria fiecărui obiect generator de umbre (occluder) pe un plan stabilit** rezultând astfel **umbra obiectului respectiv**
- proiecția poate fi descrisă de o matrice 4x4 de transformare în coordonate omogene



L – sursa de lumină punctiformă
O – punct de pe plan
P – punct al obiectului generator de umbră
S – proiecția lui P pe plan

Pentru a simplifica lucrurile să considerăm pentru început situația din figură unde avem o sursă de lumină la poziția L = (Lx,Ly,Lz) și planul care va primi umbra este y = 0

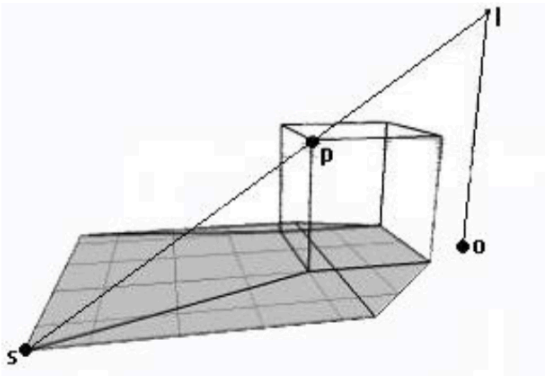
Un punct P = (Px,Py,Pz) de pe obiect văzut din sursa de lumina L va proiecta o umbră pe plan în punctul S = (Sx,0,Sz).

Punctul S poate fi exprimat ca:

$$s = L + t(P - L) \qquad (1)$$

Dar cum Sy = 0 rezultă:

$$t = \frac{L_y}{L_y - P_y}$$



Introducând în formula (1) obținem coordonatele lui S:

$$\begin{bmatrix} S_x \\ 0 \\ S_z \\ w \end{bmatrix} = \begin{bmatrix} L_y & -L_x & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -L_z & L_y & 0 \\ 0 & -1 & 0 & L_y \end{bmatrix} \begin{bmatrix} P_x \\ P_y \\ P_z \\ 1 \end{bmatrix}$$

Unde w = $L_y - P_y$

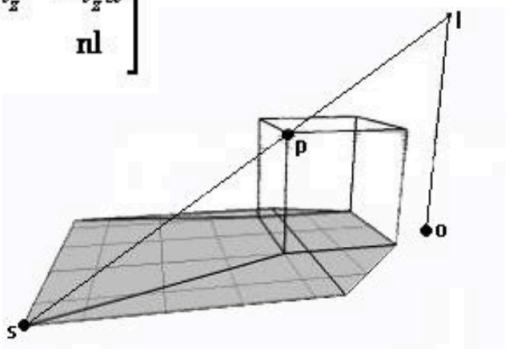
În cazul general poligonul pe care se vor proiecta umbrele nu se află în planul y = 0.

Pentru un plan oarecare **NX** + d = 0, unde **N** este vectorul normalei planului și **X** este un punct oarecare de pe plan se poate construi matricea transformării M într-un mod similar. Combinând formula (1) cu ecuația planului și introducând pe **X = S** rezultă :

$$t = \frac{NL+d}{N(L-P)}$$

Similar rezulta matricea de transformare M care satisface relația S = MP

$$\mathbf{M} = \begin{bmatrix} \mathbf{nl} + d - l_x n_x & -l_x n_y & -l_x n_z & -l_x d \\ -l_y n_x & \mathbf{nl} + d - l_y n_y & -l_y n_z & -l_y d \\ -l_z n_x & -l_z n_y & \mathbf{nl} + d - l_z n_z & -l_z d \\ -n_x & -n_y & -n_z & \mathbf{nl} \end{bmatrix}$$



Metoda propusă de Blinn este foarte simplă => duce la o serie de probleme/limitări

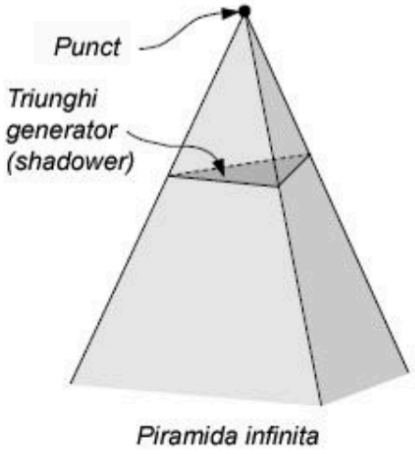
- limitarea la suprafețe plane
- datorită problemelor de precizie este posibil ca poligoanele proiectate sa fie afișate sub cele ale planului pe care se proiectează umbrele
- umbrele sunt proiectate pe planul ce conține poligonul primitor și nu direct pe acesta; astfel umbra va fi desenată și în afara poligonului -> folosire stencil buffer pentru rezolvare
- pot fi afișate umbre chiar și pentru obiectele care nu pot în mod normal să aibă umbre; dacă obiectul este situat în spatele sursei de lumină umbrele generate de acesta se numesc anti-umbre (anti-shadows) iar dacă obiectul se află în spatele planului umbrele generate se numesc umbre false (fake shadows); o soluție pentru eliminarea acestor probleme ar fi folosirea de clipping-planes

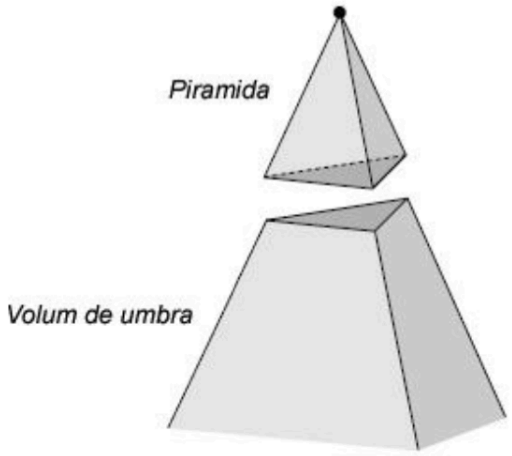
Avantaje	**Dezvantaje**
- realizează proiectarea umbrelor pe suprafețe plane cu rezultate f. bune	- nu permite obiectelor concave sa se auto-umbrească
- metodă rapidă pentru scene mici cu o singură sursă de lumină și un plan pe care se proiectează umbrele	- creează hard shadows
	- dacă se dorește realizarea de umbre pentru surse de lumină multiple și mai multe plane de proiectare, costul computațional crește foarte repede

Shadow Volumes

Introducere

- folosind această metodă se pot **proiecta umbre pe obiecte arbitrare** și se poate realiza și **auto-umbrirea obiectelor**
- pentru început sa ne imaginăm **o sursă de lumină punctiformă** și **un poligon generator (shadower)**; luând ca poligon generator un triunghi extinzând virtual liniile din sursa de lumină și care trec prin vârfurile triunghiului obținem o piramidă infinită
- putem considera partea de deasupra ca fiind o piramidă iar partea de sub poligon ca fiind un trunchi de piramidă; **orice parte a unui obiect care se află în interiorul volumului trunchiului de piramidă va fi în umbră** => denumirea acestui volum este "**volum de umbră" (shadow volume)**





- să ne imaginăm că vizualizăm scena dintr-un anumit punct din spațiu, iar pentru fiecare pixel afișat ducem o raza până când aceasta va lovi obiectul afișat pe ecran; pe măsura ce **raza este proiectată spre obiect incrementăm un contor de fiecare dată când aceasta intră în volumul de umbră** (traversează o față front-facing a acestuia) și **decrementăm același contor când raza iese din volumul de umbră** (traversează o față back-facing a acestuia).
- dacă în momentul în care raza a lovit pixelul obiectului contorul este mai mare decât 0 atunci acel pixel se află în volumul de umbră (este umbrit) iar în caz contrar se află în afara acestuia.
- Dacă această metodă ar fi aplicată geometric pentru fiecare pixel al obiectului ar necesita un număr foarte mare de calcule. De aceea în practică se folosește stencil buffer-ul care va face contorizarea.

- Folosind și stencil buffer-ul și z-buffer-ul se desenează pe rând întâi fețele front-facing ale volumului de umbră având grijă să incrementăm contorul din stencil-buffer iar apoi sunt desenate fețele back-facing ale volumului de umbră având grijă să decrementăm contorul.
- În final este afișată întreaga scenă, o valoare de 0 în stencil-buffer însemnând că acel pixel se află în afara volumului de umbră (este iluminat) iar o valoare de 1 înseamnă că acesta este umbrit.

Sinteză implementare algoritm

1. **Se dezactivează buffer-ele de culoare și adâncime referitor la scriere (acestea nu mai primesc update-uri) dar testul de adâncime rămâne activat**
2. **Se activează back-face culling-ul (vor fi desenate doar fețele front-facing)**
3. **Se setează operația stencil să incrementeze (GL_INCR) buffer-ul de marcaj dacă testul de adâncime reușește**
4. **Se desenează poligoanele volumului de umbră**
5. **Se activează front-face culling-ul (vor fi desenate doar fețele back-facing)**
6. **Se setează operația stencil să decrementeze (GL_DECR) buffer-ul de marcaj dacă testul de adâncime reușește**
7. **Se desenează poligoanele volumului de umbră**

- metoda denumită și **depth-pass** deoarece **stencil buffer-ul este modificat când testul de adâncime reușește**
- problemă: observatorul se poate afla în interiorul unuia sau mai multe volume de umbră:
    - se inițializează stencil buffer-ul cu numărul de volume de umbră în care se află observatorul;
    - calculul dacă observatorul se află în cadrul mai multor volume de umbră poate deveni foarte costisitor -> folosire metoda **depth-fail**

Metoda depth-fail

- stencil buffer-ul este modificat când testul de adâncime eșuează

1. Se dezactivează buffer-ele de culoare și adâncime referitor la scriere (acestea nu mai primesc update-uri) dar testul de adâncime rămâne activat
2. Se activează front-face culling-ul (vor fi desenate doar fețele back-facing)
3. Se setează operația stencil să incrementeze (GL_INCR) buffer-ul de marcaj dacă testul de adâncime eșuează
4. Se desenează poligoanele volumului de umbră
5. Se activează back-face culling-ul (vor fi desenate doar fețele front-facing)
6. Se setează operația stencil să decrementeze (GL_DECR) buffer-ul de marcaj dacă testul de adâncime eșuează
7. Se desenează poligoanele volumului de umbră

Practic se numără poligoanele de umbra din spatele suprafețelor considerate !

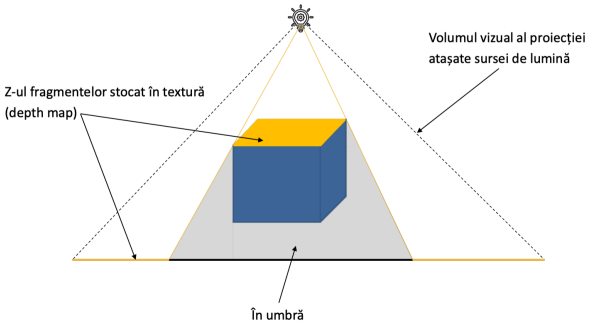
Observație: părțile din față și spate a volumului de umbră (bazele trunchiului piramidei) trebuie să fie construite (acoperite)

Cunoscută și sub numele de *Carmack's reverse* (propusă de John Carmack în dezvoltarea jocului Doom 3).

Shadow Mapping

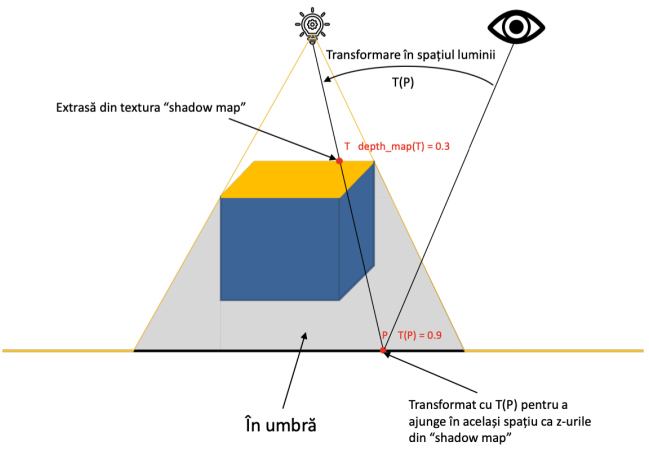
Doi pași de redare (render pass)

1. **Scena redată din poziția sursei de lumină cu redare direct într-o textură**
   a. matricea de VIEW calculată cu glm::lookat folosind ca poziție a observatorului poziția sursei de lumină
   b. se setează ca **buffer de imagine** un **buffer diferit de cel al redării normale**; acest buffer va avea atașată o textură imagine de dimensiune dată -> denumită **shadow map/depth map**
   c. se redă scena folosind **matricea VIEW** ce are **ca poziție sursa de lumină (practic scena este văzută din perspectiva sursei de lumină)** folosind ca buffer de imagine shadow map-ul
   d. se vor memora în shadow map doar coordonatele z ale fragmentelor vizibile din poziția sursei de lumină



i. Fragment shader-ul nu va scrie nimic în culoarea de output, rasterizatorul atașând automat coordonatele z interpolate în gl_FragDepth

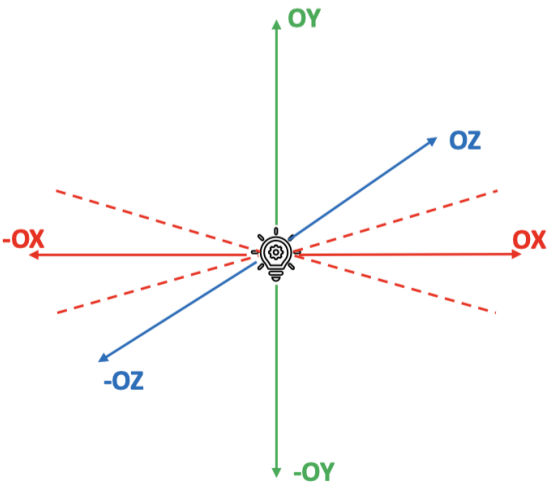
2. **Scena redată normal**
   a. în vertex shader se transformă vârfurile cu MVP
   b. suplimentar se transmite matricea VP pentru sursa de lumină ce va fi folosită pentru a transforma vârfurile în spațiul sursei de lumină (MVPS)



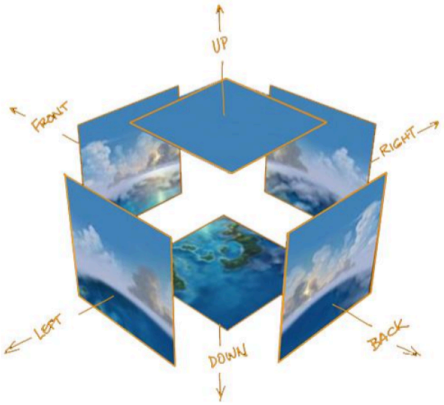
   c. în fragment shader se extrage din textura de shadow map coordonata z corespondentă fragmentului considerat ce a fost transformat în spațiul sursei de lumină
   d. se compară aceasta valoare (extrasă din shadow map) cu cea a z-ului fragmentului considerat (transformat în spațiul sursei de lumină) și dacă valoarea este mai mică înseamnă că atunci fragmentul considerat se află în umbră (sursa de lumină "vede" alt fragment care este mai aproape de ea decât fragmentul considerat)

Omni-directional shadow mapping

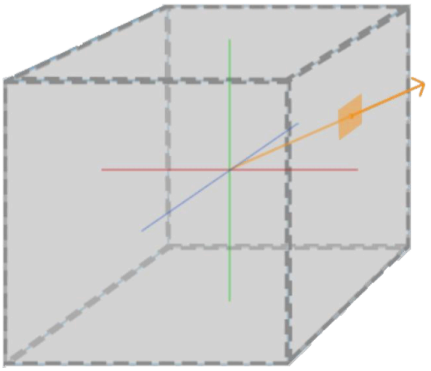
- sursa de lumină omni-direcțională (punctiformă)
- 6 volume de vizualizare din poziția sursei de lumină ce acoperă întreaga scenă: OX, -OX, OY, -OY, OZ, -OZ
- se folosește un cubemap unde fiecare față reprezintă textura de shadow map pentru fiecare din cele 6 vizualizări
- pentru a nu reda explicit de 6 ori se folosește un geometry shader care generează fiecare față a cubemap-ului, fiecare din aceste fețe cu matricea de VIEW corespunzătoare
- în rest algoritm similar cu cel de la shadow map clasic cu singura diferență că extragerea coordonatei Z din shadow map se face prin eșantionarea din cubemap pe direcția de la sursa de lumină la fragmentul considerat



Câte o vedere din poziția sursei de lumină care acoperă fiecare din cele 6 direcții



Cubemap



Eșantionare în cubemap folosind un vector direcție

Pentru omnidirectional shadow maps:

- Eșantionare pe directia R unde

  R = world_position - light_position